Insurance Claims Frequency and Severity

McQueen Insurances

Contents

1 Executive Summary **1**

2 Data Preparation **1**

2.1 Overview of the Dataset 1

2.2 Data Transformation 8

2.3 Data Exploration 32

2.4 Feature Engineering 48

3 Modelling **71**

3.1 Validation Framework 72

3.2 Frequency Modelling 79

3.3 Severity Modelling 94

4 Global Overview **95**

4.1 Importance of Features 95

4.2 Final Premium Structure 95

5 Conclusion **95**

1 Executive Summary

2 Data Preparation

2.1 Overview of the Dataset

The first step of the data exploration process is to load the insurance claims of our Belgian Motor Third Party Liability (MTPL) dataset. We will start ensuring the uniqueness of keys. Straight after that, we will split the data between training and testing sets to make sure we keep a proper holdout for model evaluation. Then, we'll have a look at our target variables and the distribution of the exposure. Finally, we will have a quick look at the predictor variables we have before diving in the data transformation, exploration, and feature engineering.

2.1.1 Uniqueness of Keys

Each row in the dataset represents a unique insurance policy (`ID`). We can verify the uniqueness of rows by checking if the number of unique `ID`s matches the total number of rows.

```
setwd(paste0(
    "C:/Users/MathieuDemarets/Documents/UCL Certificat/",
    "CodeRepositories/LACTU2150/project"
))
load("../data/mtpl_be.rda")
data <- mtpl_be

n_unique_ids <- length(unique(mtpl_be$ID))
n_rows <- nrow(mtpl_be)
cat(sprintf(
    "Number of unique IDs: %d\nNumber of rows: %d\nAre they equal? %s",
    n_unique_ids, n_rows,
    ifelse(n_unique_ids == n_rows, "Yes", "No")
))
```

```
Number of unique IDs: 163231
Number of rows: 163231
Are they equal? Yes
```

```
index_training <- createDataPartition(mtpl_be$NCLAIMS, times = 1, p = 0.8, list = FALSE) # Not
data <- mtpl_be[index_training, ]
holdout <- mtpl_be[-index_training, ]
cat(sprintf(
    "Training set rows (data): %d\nTesting set rows (holdout): %d\n",
    nrow(data), nrow(holdout)
))
```

```
Training set rows (data): 130585
Testing set rows (holdout): 32646
```

2.1.2 Target Variables

For each policy, we have an exposure (`EXP`), which represents the fraction of the year that the policy was in force. The contract last a single year at most. Over that exposure time, we have the two target variables:

- **Frequency (`NCLAIMS`)**: The number of claims filed during the policy period.
- **Severity (`AVG`)**: The average cost per claim during the policy period.

The total claim cost is derived from the frequency and severity:

- **Total Claim Cost (`NCLAIMS * AVG = AMOUNT`)**: The total cost of claims during the policy period.[1]

```
qtable <- function(data, x, n_quant) {
    data_not_na <- data[!is.na(data[[x]]), ]
    if (sum(is.na(data[[x]])) > 0) warning(sprintf("Removing %d NA values from column %s", sum
    # Get the n_quantiles of the column x
    qx <- quantile(data_not_na[[x]], probs = seq(0, 1, by = 1 / n_quant))
    quantiles <- data.frame(Quantile = names(qx), Value = as.numeric(qx))
    quantiles$Value <- format(quantiles$Value, big.mark = ",", format = "f", digits = 2, scient
    rownames(quantiles) <- quantiles$Quantile
    quantiles$Quantile <- NULL
    quantiles %>% t() %>% as.data.frame() %>% kable(
    caption = sprintf("Quantiles of %s", x),
    booktabs = FALSE
    ) %>% kableExtra::kable_styling(full_width = FALSE, position = "left")
}
```

```
qtable(data, "NCLAIMS", 10)
```

Table 1: Quantiles of NCLAIMS

	0%	10%	20%	30%	40%	50%	60%	70%	80%	90%	100%
Value	0	0	0	0	0	0	0	0	0	1	5

```
p1 <- ggplot(data, aes(x = NCLAIMS)) +
    geom_bar() +
    labs(
        title = "NCLAIMS Distribution",
        x = "Number of Claims",
        y = sprintf("Count (All: %d)", nrow(data))
    ) +
    theme_minimal()

qtable(data, "AVG", 10)
```

Table 2: Quantiles of AVG

	0%	10%	20%	30%	40%	50%	60%	70%	80%	90%	100%
Value	0.05	70.18	125.89	150.37	204.98	529.20	937.46	1,426.38	1,510.95	2,537.20	1,989,567.90

[1]For policies with no claims, `AMOUNT` is zero; `AVG` is undefined.

```r
filtered_AVG <- data %>% filter(!is.na(AVG))

p2 <- ggplot(filtered_AVG, aes(x = log(AVG))) +
    geom_histogram(bins = 30) +
    labs(
        title = "AVG Distribution",
        x = "Log of Average Claim Cost",
        y = sprintf("Count (All: %d)", nrow(filtered_AVG))
    ) +
    theme_minimal()

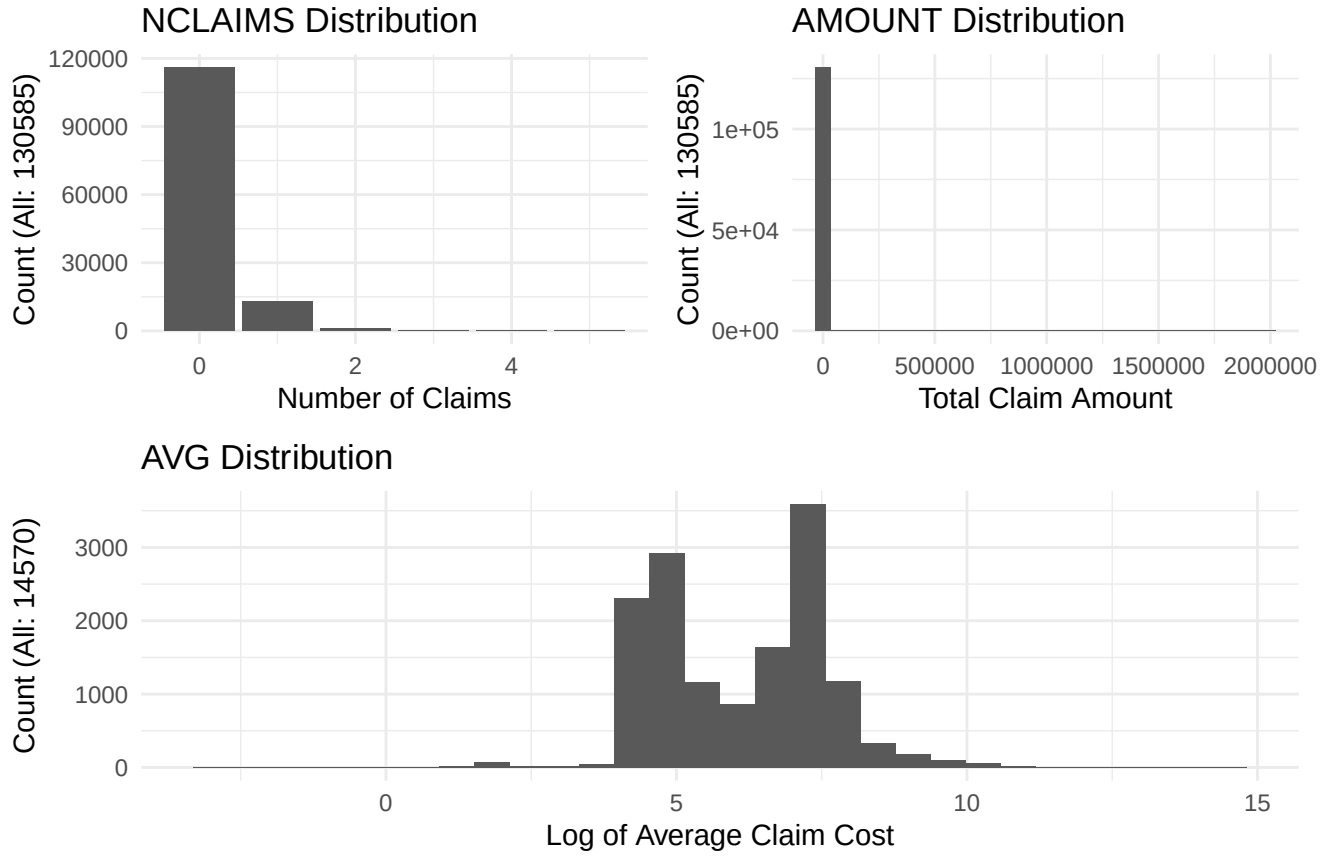
qtable(data, "AMOUNT", 10)
```

Table 3: Quantiles of AMOUNT

	0%	10%	20%	30%	40%	50%	60%	70%	80%	90%	100%
Value	0	0	0	0	0	0	0	0	0	71	1,989,568

```r
filtered_AMOUNT <- data %>% filter(!is.na(AMOUNT))
p3 <- ggplot(filtered_AMOUNT, aes(x = AMOUNT)) +
    geom_histogram(bins = 30) +
    labs(
        title = "AMOUNT Distribution",
        x = "Total Claim Amount",
        y = sprintf("Count (All: %d)", nrow(filtered_AMOUNT))
    ) +
    theme_minimal()


(p1 | p3 )/ p2
```



For both the frequency and severity of claims, we observe a large distribution mass around zero. This is quite typical in insurance data, where many policies have no claims or very low average claim costs. We also note that 116015 policies have missing average claim costs. This is simply because no claims were reported for these policies (116015 policies with no claims). The claim amount is by nature zero in this case, and the average ($\frac{AMOUNT}{NCLAIMS}$) is undefined and not informative. When we will tackle the modeling of claim severity, we will therefore focus only on policies with at least one claim. To handle unbalanced data in the frequency modelling, various techniques are available such as:

- Synthetic oversampling techniques (e.g., SMOTE) create synthetic samples for the minority class to balance the dataset.
- Hurdle models can be employed to separately model the zero and non-zero counts, effectively handling the excess zeros in the data and focusing on the non-zero records in the second part of the model.

These are outside the scope of this analysis, it is nevertheless important to note that this unbalance will limit the frequency modelling performance and that these techniques could be explored in the future.

We can also note that the distribution of average claim is more interesting to look at. Indeed, we exclude policies with no claims, as their average claim is undefined. Then, we can take the logarithm of the average claim to better visualize the scale of cost instead of the absolute values.

The quantiles indeed show us a very skewed distribution, with most average claims being relatively low but a few policies having very high average claims. We can see two peaks in the log-transformed average claim distribution which indicates the presence of different kind of claims:

- First peak corresponds to small dents and repairs: the log around 5 represents claims of approximately 148 euros
- Second peak corresponds to more significant repairs: the log around 7.5 represents claims of approximately 1808 euros
- Right tail corresponds to way bigger damages: the log around 10 represents claims of approximately 22026 euros

Even if they are not so frequent, the larger claims in the right tail can have a significant impact on the company profitability. A future report could use `Extreme Value Theory` to better model and understand these rare but impactful claims.

```
qtable(data, "EXP", 10)
```

Table 4: Quantiles of EXP

	0%	10%	20%	30%	40%	50%	60%	70%	80%	90%	100%
Value	0.0027	0.4658	0.8849	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000

Regarding the exposure, we see that a large majority of the policies have an exposure close to 1, indicating that most policies are active for nearly the entire year. About 0.23 of the policies have an exposure less than 1. This does not suggest that an analysis per exposure level is necessary.

Exposure should still be considered to correctly estimate the claim frequency as counting full exposure would underestimate the risk. The correct way to compute the mean claim frequency is

$$\frac{\sum NCLAIMS}{\sum EXP}, \text{ where } EXP \in [0, 1]$$

which will therefore be higher than the simple mean

$$\frac{\sum NCLAIMS}{nrow(data) = \sum 1}$$

```
mean_claim_frequency <- sum(data$NCLAIMS) / sum(data$EXP)
se_mean <- sqrt(mean_claim_frequency / sum(data$EXP))
ic_99 <- mean_claim_frequency + c(-1, 1) * 2.576 * se_mean
wrong_mean <- mean(data$NCLAIMS)
cat(sprintf(
    "Correct mean claim frequency: %.3f%% (99%% CI: [%.3f%%, %.3f%%])\nNaive mean: %.3f%%",
    mean_claim_frequency * 100,
    ic_99[1] * 100,
    ic_99[2] * 100,
    wrong_mean * 100
))
```

```
Correct mean claim frequency: 13.862% (99% CI: [13.581%, 14.144%])
Naive mean: 12.336%
```

This confirms that the naive mean of `NCLAIMS` falls short of the 99% confidence interval for the correct mean claim frequency. It is therefore crucial to account for the exposure through the offset[2] in the frequency modelling.

2.1.3 All Variables

```r
col_types <- sapply(data, class)

knitr::kable(
    data.frame(
        Type = col_types,
        Center = sapply(data, function(x) {
            if (is.numeric(x)) {
                mean(x, na.rm = TRUE) %>% round(4)
            } else {
                counts <- sort(table(x), decreasing = TRUE)
                names(counts)[1]
            }
        }),
        Missing = sapply(data, function(x) sum(is.na(x)))
    ),
    caption = "Overview of variable in the insurance claims dataset",
    booktabs = TRUE
) %>% kableExtra::kable_styling(full_width = TRUE, position = "left")
```

Table 5: Overview of variable in the insurance claims dataset

	Type	Center	Missing
ID	integer	81699.2272	0
NCLAIMS	integer	0.1234	0
AMOUNT	numeric	208.8667	0
AVG	numeric	1698.6839	116015
EXP	numeric	0.8899	0
COVERAGE	character	TPL	0
FUEL	character	gasoline	0
USE	character	private	0
FLEET	character	N	0
SEX	character	male	0

[2] In frequency modelling, the exposure is typically included as an offset in the model to adjust for the varying lengths of policy coverage. This ensures that the estimated claim frequency accurately reflects the risk per unit of exposure rather than per policy.

AGEPH	integer	47.01	0
BM	integer	3.2671	0
AGEC	integer	7.3796	0
POWER	integer	56.0367	0
PC	integer	5275.3966	0
TOWN	character	CHARLERO	0
LONG	numeric	4.4065	0
LAT	numeric	50.7588	0

From this basic overview, we can confirm a few key points:

- `ID`: all rows indeed have an identifier (no missing values) and we saw before that it is unique for each row.
- `NCLAIMS`: the mean value of 12.336% is **NOT** the correct mesure of risk over the exposition and should be adjusted by the exposure `EXP` to get 13.862%.
- `AMOUNT`: the average loss amount of 208.867 euros hides very different realities between no-claim, low-ticket, medium, and high-ticket claims as we established earlier.
- `AVG`: the average claim cost is higher than the average loss amount which seems counter-intuitive at first, but it reflects the fact that `AVG` is calculated only for policies with claims, whereas `AMOUNT` includes all policies, including those with no claims as we see with the number of missing values in `NCLAIMS` corresponding to policies with no claims as discussed earlier.

We can also notice that some columns have been asigned the wrong type:

- **COVERAGE, FUEL, USE, FLEET, SEX, and TOWN** should be set as unordered factor variables instead of character variables.
- **PC** should be converted from integer to an unordered factor variable.

This leads us to our next section on data transformation.

2.2 Data Transformation

2.2.1 Categorical Variables

As concluded at the end of the last section, we need to transform some character variables to factor variables. We will also make sure that the most frequent level (Center in the previous overview) is set as the reference level for unordered factor variables.

```
for (col in c("COVERAGE", "FUEL", "USE", "FLEET", "SEX", "TOWN", "PC")) {
    data[[col]] <- as.factor(data[[col]]) %>% fct_infreq()
}
```

We can then assert that the categorical variables (ordered or unordered) have the correct reference levels. Note that our function also checks for ordered factor variables, ensuring that the reference level is the lowest level in this case. This does not happen in our dataset. The variable `BM` could

be considered as unordered as we know that a higher BM score indicates a more risky driver but we don't know if the jumps between the levels are constant. We decided to treat it as numerical because the numeric variable will be binned with a weight of evidence (WoE) approach and it will yield more meaningful bins than simple singleton bins.

```
assert_ref_level <- function(data, cols) {
    for (col in cols) {
        if (!is.factor(data[[col]])) stop(paste(col, "is not a factor variable"))
        if (is.ordered(data[[col]])) {
            if (as.numeric(levels(data[[col]]))[1] != min(as.numeric(as.character(data[[col]]))
            # Pay attention to turn labels as numeric and not the factor internal codes which
                stop(paste("Reference level for", col, "is not the lowest level"))
            }
        } else {
            if (
                levels(data[[col]])[1] !=
                    fct_count(data[[col]], sort = TRUE)$f[1]
            ) {
                stop(
                    paste(
                        "Reference level for", col,
                        "is not the most frequent level"
                    )
                )
            }
    }
    }
    cat("Reference levels for", paste(cols, collapse = ", "), "are correctly set\n")
}
assert_ref_level(data,  c("COVERAGE", "FUEL", "USE", "FLEET", "SEX", "TOWN", "PC"))
```

```
Reference levels for COVERAGE, FUEL, USE, FLEET, SEX, TOWN, PC are correctly set
```

2.2.2 Numerical Variables Binning

For the initial GLM modeling, the numerical variables can be used but this assumes that:

- The effect of one increment of the numerical variable on the target is assumed to be linear (age 18 to 19 has the same effect as age 50 to 51 which may not be realistic as experience marginal returns are likely diminishing).
- The linearity assumption may not hold for all numerical variables, and discretizing them into bins can help capture non-linear relationships with the target.

For these reasons, we will discretize the continuous numerical variables into bins. Defining optimal binning for a numerical variable is a complex and important step. There are a set of naive approaches that can be used:

- Singleton bins: Each unique value of the numerical variable is treated as a separate bin.
- Equal-width bins: The range of the variable is divided into intervals of equal width.
- Equal-frequency bins: Quantiles of the variable are used to create bins with approximately the same number of observations.

For this project, we will try to implement a more complex approach based on weight of evidence (WoE) to automatically determine a suitable binning for numerical variables. The idea of WoE binning is to group values based on how strongly they are associated with the target variable (binary outcome encoded as good if claim = 0 and bad if claim >= 1):

$$\text{WoE} = \ln \frac{\text{proportion of Good Outcomes in the bin}}{\text{proportion of Bad Outcomes in the bin}}$$

Based on the WoE values, we can also compute the information value (IV) for each variable, which measures the predictive power of the variable with respect to the target. The IV is calculated as:

$$\text{IV} = \sum(\text{proportion of Good Outcomes in the bin} - \text{proportion of Bad Outcomes in the bin}) \times \text{WoE}$$

The IV value helps to assess the predictive power of a variable for the binning we have chosen. The idea of the algorithm will be to:

- Start with an adapted quantile-based binning as a baseline with a maximum number of bins specified.
- Iteratively merge bins based on bins with minimal information value (IV) contribution and assess total IV after each merge.
- Find the elbow point where the marginal contribution of one additional bin to the total IV decreases significantly.

We will take that elbow point and its associated bins boundaries for the final binning of the variable. We will detail each step of the algorithm with an example using the `AGEPH` variable.

2.2.2.1 Baseline Quantile Binning

Our function `qbinning` is a simple algorithm that will start from the maximum number of bins specified by `max_bins` and iteratively reduce the number of bins until each bin contains at least one claim. This will ensure that the weight of evidence (WoE) can be calculated for each bin, as each bin will have at least one claim. We'll then avoid having an undefined WoE.

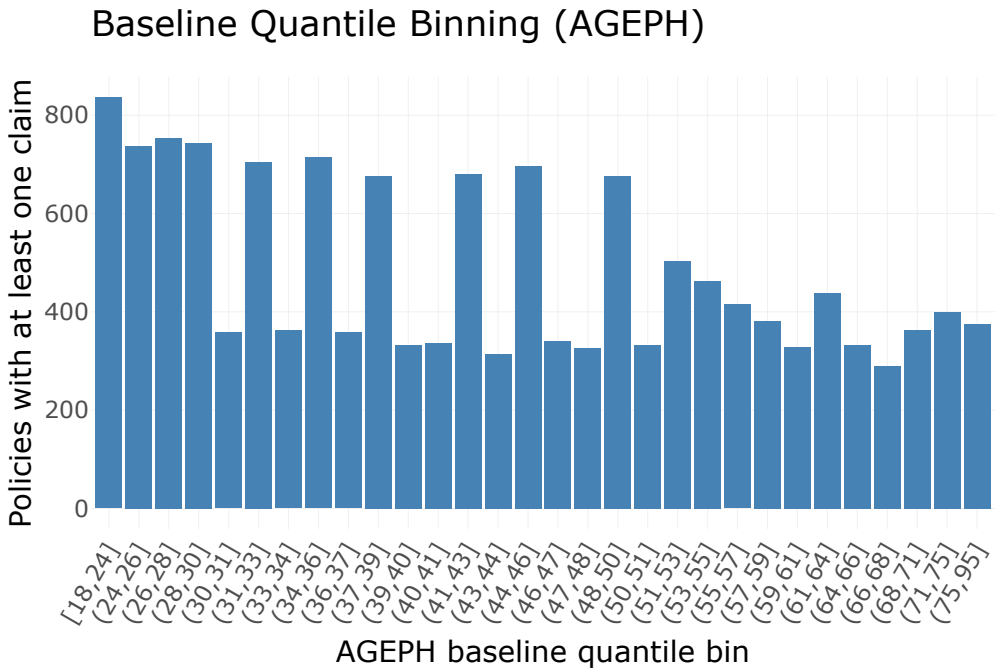
```
qbinning <- function(data, x, y, max_bins = 10) {
    "Quantile-based binning for numerical variable x with respect to binary target y"
    if (sum(data[[y]] >= 1) == 0) stop("No claims in the dataset")
    if (sum(data[[y]] == 0) == 0) stop("No non-claims in the dataset")
    if (!is.numeric(data[[x]])) stop("Variable x must be numeric")
    raw_data <- data
    temp_n <- max_bins
    temp_binning <- NULL
```

```
    baseline_bins <- NULL
    while (temp_n > 1) {
        temp_binning <- unique(
            quantile(raw_data[[x]], probs = seq(0, 1, length.out = temp_n + 1), na.rm = TRUE)
        )
        # check if there is at least one claim in each bin
        bin_claims <- tapply(
            raw_data[[y]],
            cut(raw_data[[x]], temp_binning, include.lowest = TRUE),
            function(z) sum(z >= 1))
        if (all(bin_claims > 0)) {
            baseline_bins <- temp_binning
            break
        }
        temp_n <- temp_n - 1
    }
    baseline_bins
}
```

```
bins_ageph <- qbinning(data, x = "AGEPH", y = "NCLAIMS", max_bins = 30)
counts <- tapply(
    data[["NCLAIMS"]],
    cut(data[["AGEPH"]], bins_ageph, include.lowest = TRUE),
    function(z) sum(z >= 1)
)

df_bins <- data.frame(
    bin = factor(names(counts), levels = names(counts)),
    count = as.numeric(counts)
)

p <- ggplot(df_bins, aes(x = bin, y = count)) +
ggtitle("Baseline Quantile Binning (AGEPH)") +
geom_col(fill = "steelblue") +
labs(x = "AGEPH baseline quantile bin", y = "Policies with at least one claim") +
theme_minimal() +
theme(axis.text.x = element_text(angle = 60, hjust = 1))
ggplotly(p)
```


Baseline Quantile Binning (AGEPH)
Policies with at least one claim
0
200
400
600
800
[18,24]
(24,26]
(26,28]
(28,30]
(30,31]
(31,33]
(33,34]
(34,36]
(36,37]
(37,39]
(39,40]
(40,41]
(41,43]
(43,44]
(44,46]
(46,47]
(47,48]
(48,50]
(50,51]
(51,53]
(53,55]
(55,57]
(57,59]
(59,61]
(61,64]
(64,66]
(66,68]
(68,71]
(71,75]
(75,95]
AGEPH baseline quantile bin

Starting with 30 quantile bins, we apply our tweaked function `qbinning` to the `AGEPH` variable and find these 30 bins with some policies having at least one claim.

2.2.2.2 Weight of Evidence (WoE) and Information Value (IV) Calculation

Based on the formula presented above, the weight of evidence (WoE) for each bin can be calculated as the natural logarithm of:

- proportion of Good Outcomes in the bin $= \frac{\text{Number of Good Outcomes in the bin}}{\text{Total Number of Good Outcomes}}$
- proportion of Bad Outcomes in the bin $= \frac{\text{Number of Bad Outcomes in the bin}}{\text{Total Number of Bad Outcomes}}$

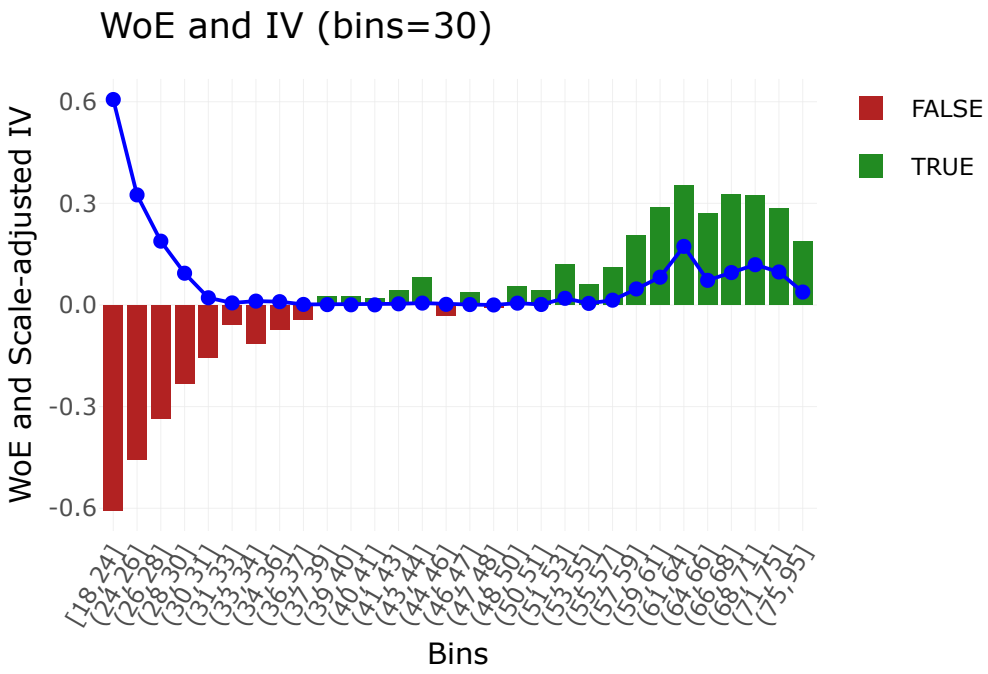
If the bin has as much good (no claim) as bad (at least one claim) outcomes, the WoE will be zero as there is very little predictive power in that bin. If the bin has more good outcomes than bad outcomes, the WoE will be positive, indicating that the bin is associated with a lower risk. Finally, if the bin has more bad outcomes than good outcomes, the WoE will be negative, indicating that the bin is associated with a higher risk. The IV quantifies the overall predictive power based on the differences in the distribution of good and bad outcomes and the corresponding WoE values.

```
woe_calc <- function(data, x, y, bins) {
    "Calculate Weight of Evidence (WoE) for each bin"
    woe_df <- data
    binned_woe <- tapply(
        woe_df[[y]],
        cut(woe_df[[x]], bins, include.lowest = TRUE),
        function(z) c(good = sum(z == 0), bad = sum(z >= 1))
    )
    binned_woe_df <- data.frame(
        bin = factor(names(binned_woe), levels = names(binned_woe)),
        good = sapply(binned_woe, function(x) x["good"]),
        bad = sapply(binned_woe, function(x) x["bad"])
    )
    rownames(binned_woe_df) <- NULL
    binned_woe_df$prop_good <- binned_woe_df$good / sum(binned_woe_df$good)
    binned_woe_df$prop_bad <- binned_woe_df$bad / sum(binned_woe_df$bad)
    binned_woe_df$woe <- log(binned_woe_df$prop_good / binned_woe_df$prop_bad)
    binned_woe_df$iv <- iv <- (binned_woe_df$prop_good - binned_woe_df$prop_bad) * binned_woe_
    return(binned_woe_df)
}

plot_woe_iv <- function(woe_df, interactive = TRUE) {
    max_woe <- max(abs(woe_df$woe))
    max_iv <- max(abs(woe_df$iv))
    # Avoid dividing by zero when all values are zero
    scale_factor <- if (max_woe > 0 && max_iv > 0) {
        max_woe / max_iv
    } else {
        1
    }
```

```
    p <- ggplot(woe_df, aes(x = bin)) +
        geom_col(
            aes(y = woe, fill = woe > 0),
            width = 0.8,
            show.legend = FALSE
        ) +
        scale_fill_manual(
            values = c("FALSE" = "firebrick", "TRUE" = "forestgreen")
        ) +
        geom_line(
            aes(y = iv * scale_factor, group = 1),
            color = "blue"
        ) +
        geom_point(
            aes(y = iv * scale_factor),
            color = "blue",
        ) +
        scale_y_continuous(
            name = "WoE and Scale-adjusted IV"
        ) +
        labs(x = "Bins") +
        theme_minimal() +
        theme(
            axis.text.x = element_text(angle = 60, hjust = 1),
        ) +
        ggtitle(
            paste0("WoE and IV (bins=", length(woe_df$bin), ")")
        )
    if (interactive) {
        ggplotly(p)
    } else {
        return(p)
    }
}
```

```
woe_ex <- woe_calc(data, x = "AGEPH", y = "NCLAIMS", bins = bins_ageph)
plot_woe_iv(woe_ex)
```

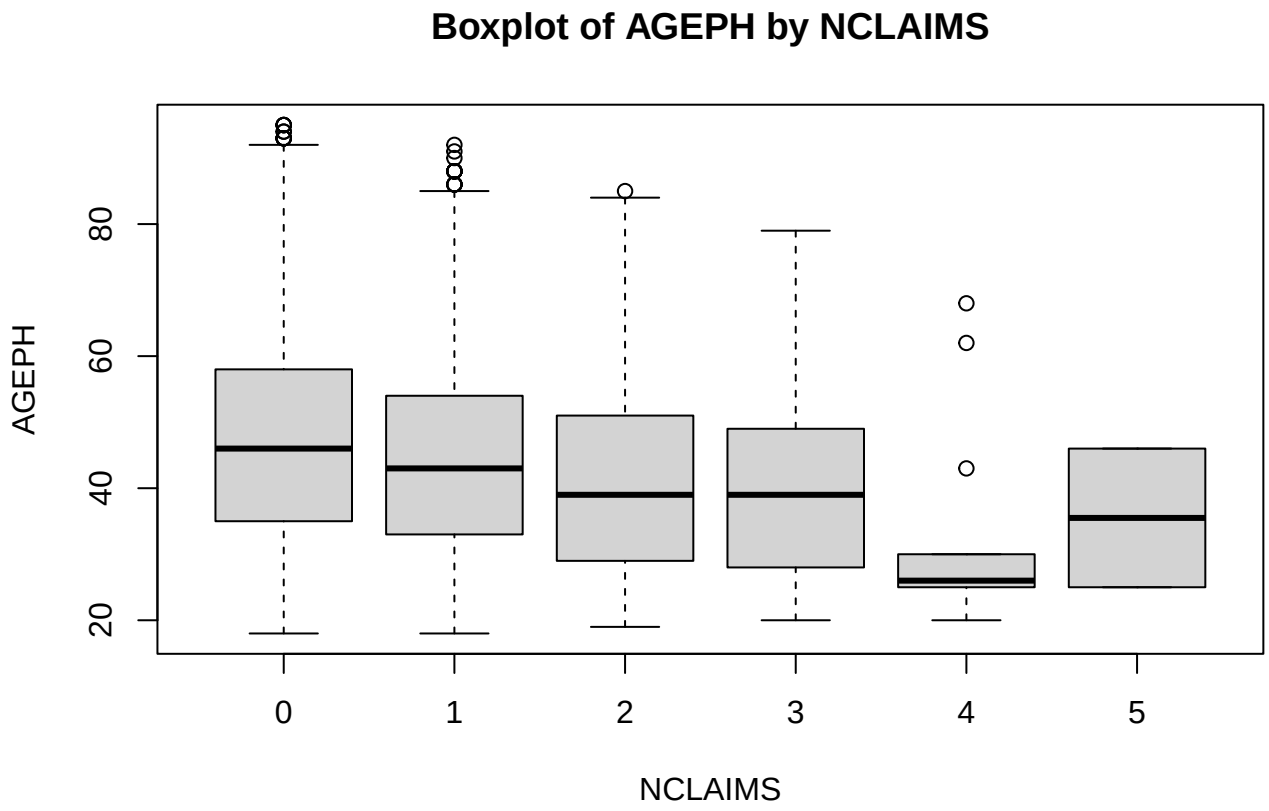

WoE and IV (bins=30)
WoE and Scale-adjusted IV
0.6
0.3
0.0
-0.3
-0.6
[18,24]
(24,26]
(26,28]
(28,30]
(30,31]
(31,33]
(33,34]
(34,36]
(36,37]
(37,39]
(39,40]
(40,41]
(41,43]
(43,44]
(44,46]
(46,47]
(47,48]
(48,50]
(50,51]
(51,53]
(53,55]
(55,57]
(57,59]
(59,61]
(61,64]
(64,66]
(66,68]
(68,71]
(71,75]
(75,95]
Bins
FALSE
TRUE

The example for AGEPH already shows very interesting patterns for our baseline binning:

- The WoE values go from negative to positive, indicating that the younger age groups have a higher likelihood of bad outcomes (at least one claim) compared to the older age groups.
- The IV line shows a very high contribution from the young and older age groups, indicating that these bins are the most predictive for the likelihood of claims.

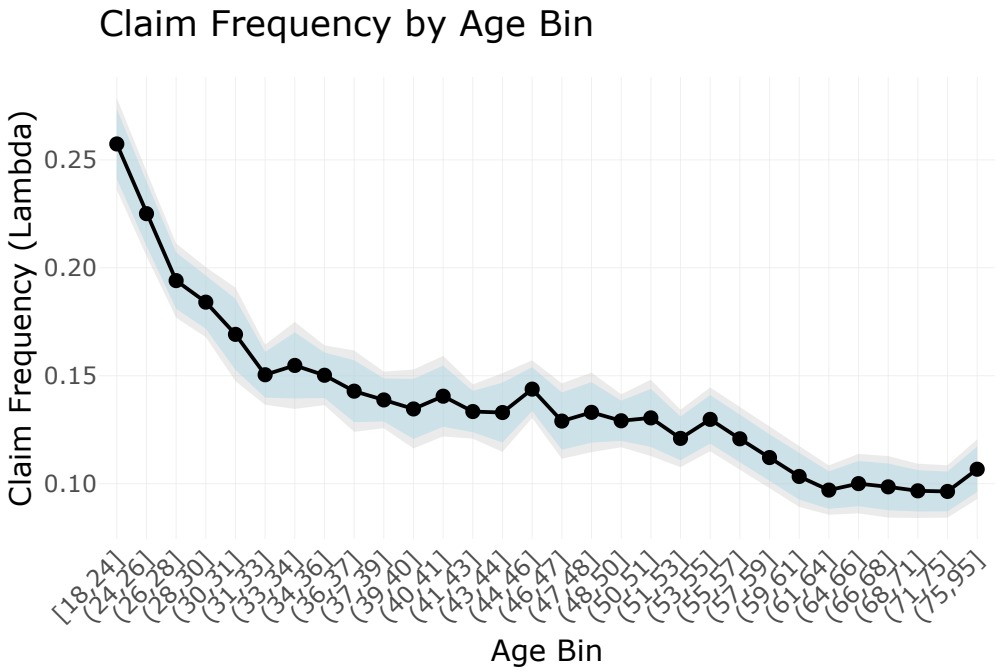
Let's confirm visuallythe relationship between age and number of claims by examining the boxplots and claim frequency by age bin.

```
# boxplot of age for each number of claims
boxplot(AGEPH ~ NCLAIMS, data = data, xlab = "NCLAIMS", ylab = "AGEPH", main = "Boxplot of AGE
```




```
# claim frequency in our example bins, group by bin
age_quantiles <- cut(data$AGEPH, breaks = bins_ageph, include.lowest = TRUE)
gpd_data <- data %>%
    mutate(age_bin = age_quantiles) %>%
    group_by(age_bin) %>%
    summarise(
```

```
        sum_exp = sum(EXP),
        sum_claims = sum(NCLAIMS),
        lambda = sum_claims / sum_exp,
        sd_lambda = sqrt(lambda / sum_exp),
        lambda_95_lower = lambda - 1.96 * sd_lambda,
        lambda_95_upper = lambda + 1.96 * sd_lambda,
        lambda_99_lower = lambda - 2.58 * sd_lambda,
        lambda_99_upper = lambda + 2.58 * sd_lambda
    )
# Plot the claim frequency (lambda) with filled error bars 95 and 99 percent confidence interv
p2 <- ggplot(gpd_data, aes(x = age_bin, group = 1)) +
    geom_ribbon(
        aes(ymin = lambda_99_lower, ymax = lambda_99_upper),
        fill = "gray", alpha = 0.3, label = "99% CI"
    ) +
    geom_ribbon(
        aes(ymin = lambda_95_lower, ymax = lambda_95_upper),
        fill = "lightblue", alpha = 0.5, label = "95% CI"
    ) +
    geom_line(aes(y = lambda), color = "black") +
    geom_point(aes(y = lambda), color = "black") +
    theme_minimal() +
    scale_y_continuous(labels = scales::comma) +
    labs(
        x = "Age Bin",
        y = "Claim Frequency (Lambda)",
        title = "Claim Frequency by Age Bin"
    ) +
    theme(axis.text.x = element_text(angle = 45, hjust = 1))
ggplotly(p2)
```


Claim Frequency by Age Bin
Claim Frequency (Lambda)
0.25
0.20
0.15
0.10
[18,24]
(24,26]
(26,28]
(28,30]
(30,31]
(31,33]
(33,34]
(34,36]
(36,37]
(37,39]
(39,40]
(40,41]
(41,43]
(43,44]
(44,46]
(46,47]
(47,48]
(48,50]
(50,51]
(51,53]
(53,55]
(55,57]
(57,59]
(59,61]
(61,64]
(64,66]
(66,68]
(68,71]
(71,75]
(75,95]
Age Bin

The boxplots show that the group of policyholders with lower amount of claims do tend to be older. It is even more striking when we look at the claim frequency by age bin with the confidence intervals[3].

2.2.2.3 Iterative Merging Based on Information Value (IV) Remember that the Information Value (IV) is a measure based on the Weight of Evidence (WoE) that quantifies the predictive power of a feature in distinguishing between good and bad outcomes. It is calculated as:

$$\text{IV} = \sum_{\text{bins}} (\text{proportion of Good Outcomes in the bin} - \text{proportion of Bad Outcomes in the bin}) \times \text{WoE of the bin}$$

We have an information value for each bin, which allows us to identify the adjacent bins that contribute the least to the overall predictive power and merge them iteratively. The goal is to recursively merge the bin with lowest IV contribution with its neighboring bin, and evaluate the resulting IV. Then we can find the elbow point where further merging would lead to a significant loss of predictive power.

```
iv_elbow <- function(data, x, y, bins) {
    "Recursively merge the bin with the lowest IV contribution with its neighboring bin with m

    temp_bins <- bins
    temp_n_bins <- length(bins) - 1
    temp_woe_df <- woe_calc(data, x, y, temp_bins)
    temp_iv <- temp_woe_df$iv
    temp_total_iv <- sum(temp_iv)

    iter_log <- data.frame(
        n_bins = temp_n_bins,
        iv = temp_total_iv,
        bins = paste(temp_bins, collapse = ","),
        diff = NA,
        diff_order = NA
    )

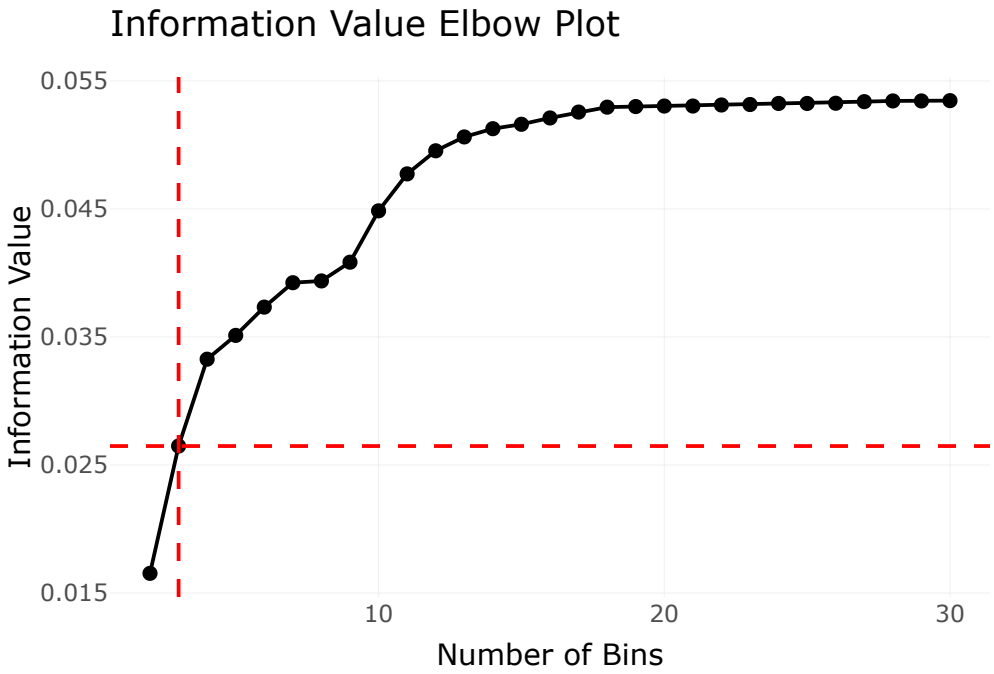
    while(temp_n_bins > 2) {
        bin_to_merge <- which.min(temp_iv)
        if (bin_to_merge == 1) {
            neighbor <- 2
        } else if (bin_to_merge == temp_n_bins) {
            neighbor <- temp_n_bins - 1
        } else {
            neighbor <- ifelse(temp_iv[bin_to_merge - 1] < temp_iv[bin_to_merge + 1], bin_to_m
        }
        to_merge <- sort(c(bin_to_merge, neighbor))
```

[3]The confidence intervals are based on the MLE estimator for the claim frequency under the assumption of a Poisson distribution for the number of claims.

```
        temp_bins <- temp_bins[-to_merge[2]]
        temp_n_bins <- length(temp_bins) - 1
        temp_woe_df <- woe_calc(data, x, y, temp_bins)
        temp_iv <- temp_woe_df$iv
        temp_total_iv <- sum(temp_iv)

        iter_log <- rbind(iter_log, data.frame(
            n_bins = temp_n_bins,
            iv = temp_total_iv,
            bins = paste(temp_bins, collapse = ","),
            diff = NA,
            diff_order = NA
        ))
    }
    iter_log <- iter_log %>% arrange(n_bins)
    iter_log$diff <- c(NA, abs(diff(iter_log$iv)))
    iter_log$diff[is.na(iter_log$diff)] <- 0
    iter_log$diff_order <- rank(-iter_log$diff)
    return(iter_log)
}
plot_iv_elbow <- function(iv_elbow_result, interactive = TRUE) {
    first_max_diff <- filter(iv_elbow_result, diff_order == 1)
    p <- ggplot(iv_elbow_result, aes(x = n_bins, y = iv)) +
        geom_line() +
        geom_point() +
        labs(title = "Information Value Elbow Plot", x = "Number of Bins", y = "Information Val
        theme_minimal() +
        geom_vline(xintercept = first_max_diff$n_bins, linetype = "dashed", color = "red") +
        geom_hline(yintercept = first_max_diff$iv, linetype = "dashed", color = "red")
    if (interactive) {
        ggplotly(p)
    } else {
        return(p)
    }
}
```

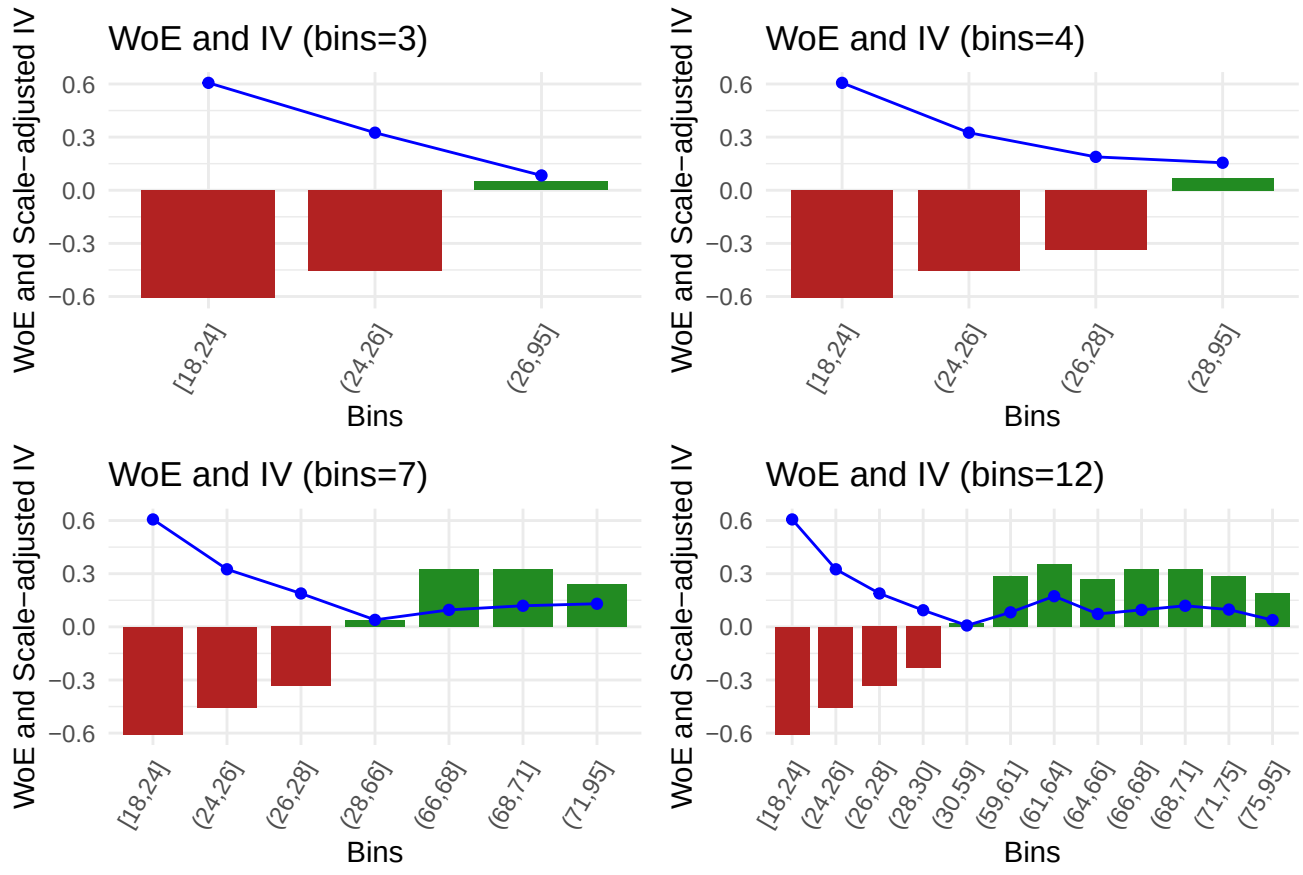
```
result_named_list <- list()
n_bins_named_list <- list()
result_ageph <- iv_elbow(data, "AGEPH", "NCLAIMS", bins_ageph)
result_named_list[["AGEPH"]] <- result_ageph
plot_iv_elbow(result_ageph, interactive = TRUE)
```


Information Value Elbow Plot
0.055
0.045
0.035
0.025
0.015
Information Value
10
20
30
Number of Bins

Looking at the IV elbow plot and taking the number of bins corresponding to the first maximum difference in IV as the optimal choice, we can rebuild the bins with only three bins. Adding a single bin seems sensible as we capture more information with a single additional split. We could also consider 7 or 12 bins that mark other levels in the IV - Number of Bins relationship.

```
rebuild_bins <- function(iv_elbow_result, n) {
    "Rebuild bins based on the IV elbow result for the specified number of bins"
    target_row <- iv_elbow_result %>% filter(n_bins == n)
    bins <- as.numeric(unlist(strsplit(target_row$bins, ",")))
    return(bins)
}
```

```
plots <- list()
for (i in c(3, 4, 7, 12)) {
    temp_bins <- rebuild_bins(result_ageph, i)
    woe_temp <- woe_calc(data, x = "AGEPH", y = "NCLAIMS", bins = temp_bins)
    plots[[i]] <- plot_woe_iv(woe_temp, interactive = FALSE)
}
(plots[[3]] | plots[[4]] ) / (plots[[7]] | plots[[12]])
```



Based on the comparison above, the optimal amount of bins seems to be 7 because:

- We capture in fine granularity the effet of young ages with higher risk.
- We skip the middle of the age distribution which brings no additional information (28 to 66).
- We capture the effect of older ages with lower risk profiles with good granularity.

The complex pattern we noticed in our initial 30 bins seems to be well captured with 7 bins only.

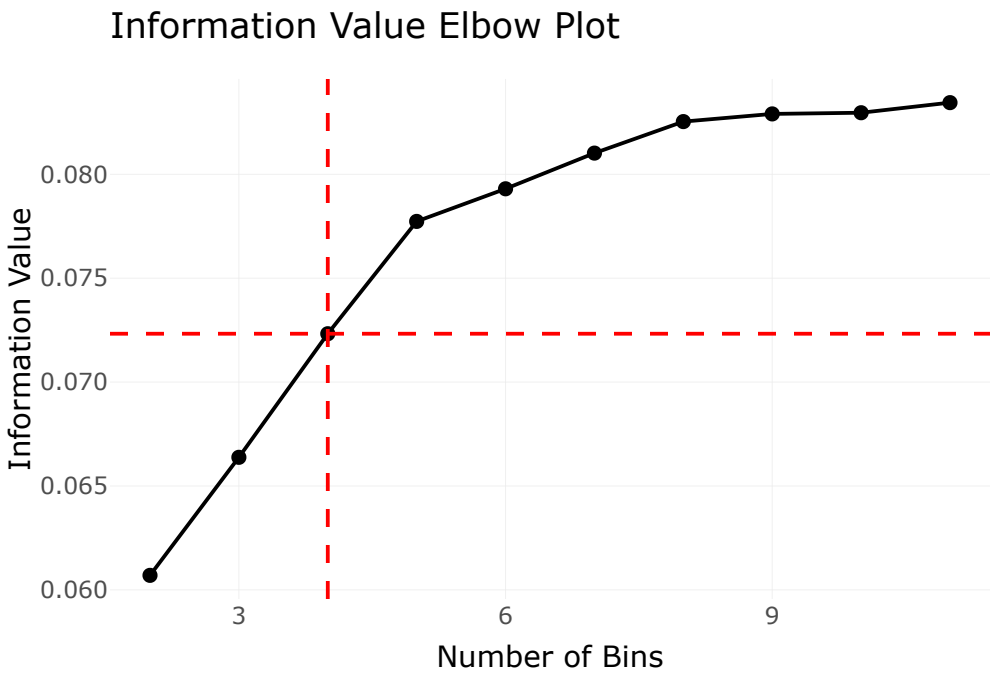
```
# We add the ordered factor variable for the binned AGEPH
data$AGEPH_binned <- cut(
    data$AGEPH,
    breaks = rebuild_bins(result_ageph, 7), include.lowest = TRUE, ordered_result = TRUE)
# Check the distribution of the binned variable
n_bins_named_list[["AGEPH"]] <- 7
summary(data$AGEPH_binned) %>%
    kable(
        caption = "Distribution of binned AGEPH",
        booktabs = TRUE
    )
```

Table 6: Distribution of binned AGEPH

	x
[18,24]	4466
(24,26]	4453
(26,28]	5052
(28,66]	100180
(66,68]	3478
(68,71]	4362
(71,95]	8594

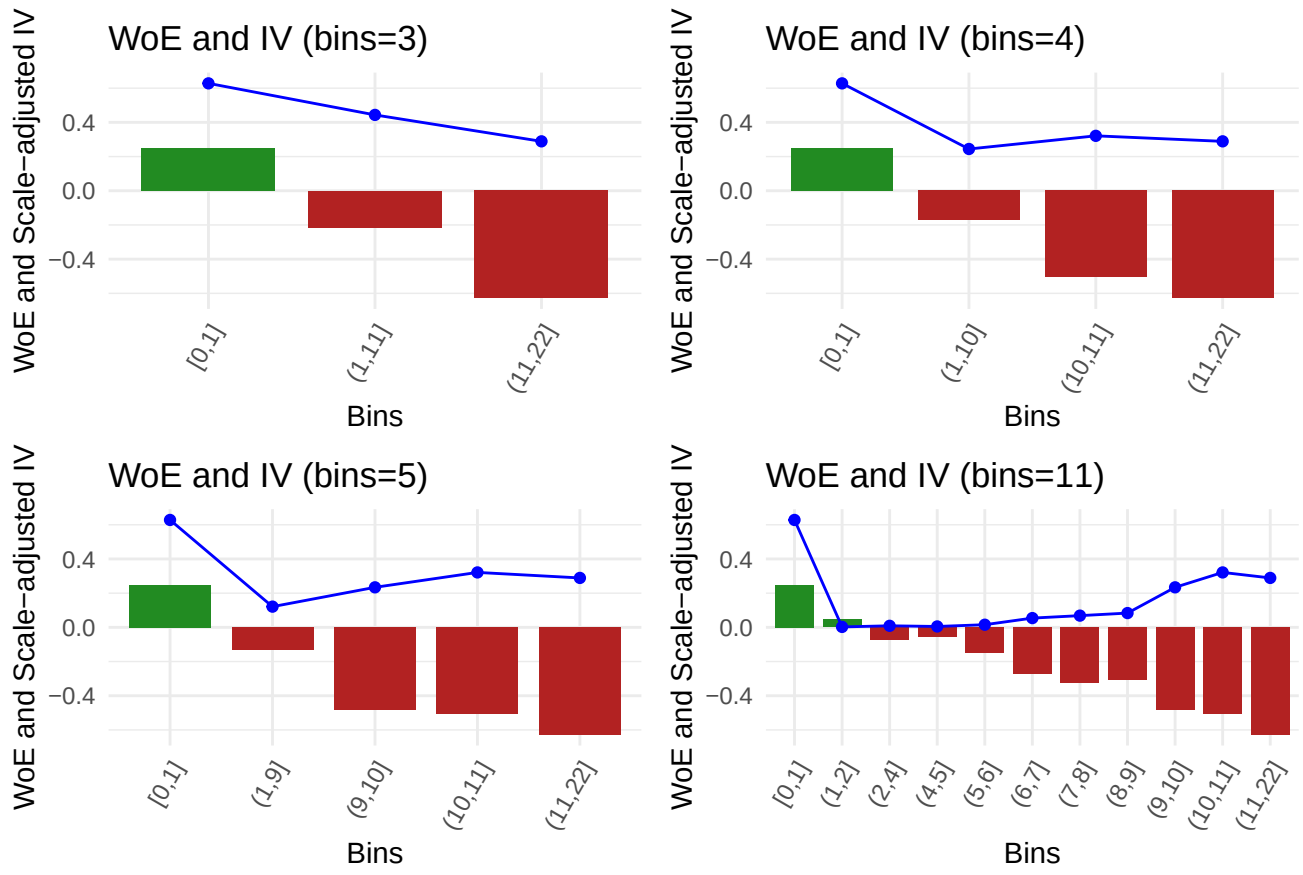
```
baseline_bm <- qbinning(data, x = "BM", y = "NCLAIMS", max_bins = 30)
result_bm <- iv_elbow(data, "BM", "NCLAIMS", baseline_bm)
result_named_list[["BM"]] <- result_bm
plot_iv_elbow(result_bm, interactive = TRUE)
```

2.2.2.4 Bonus-Malus Binning (BM)


Information Value Elbow Plot
Information Value
0.080
0.075
0.070
0.065
0.060
3
6
9
Number of Bins

Based on the Elbow plot for the Bonus-Malus (BM), let's try 3, 4, 5, and 11 bins.

```
plots_bm <- list()
for (i in c(3, 4, 5, 11)) {
    temp_bins_bm <- rebuild_bins(result_bm, i)
    woe_temp_bm <- woe_calc(data, x = "BM", y = "NCLAIMS", bins = temp_bins_bm)
    plots_bm[[i]] <- plot_woe_iv(woe_temp_bm, interactive = FALSE)
}
(plots_bm[[3]] | plots_bm[[4]] ) / (plots_bm[[5]] | plots_bm[[11]])
```

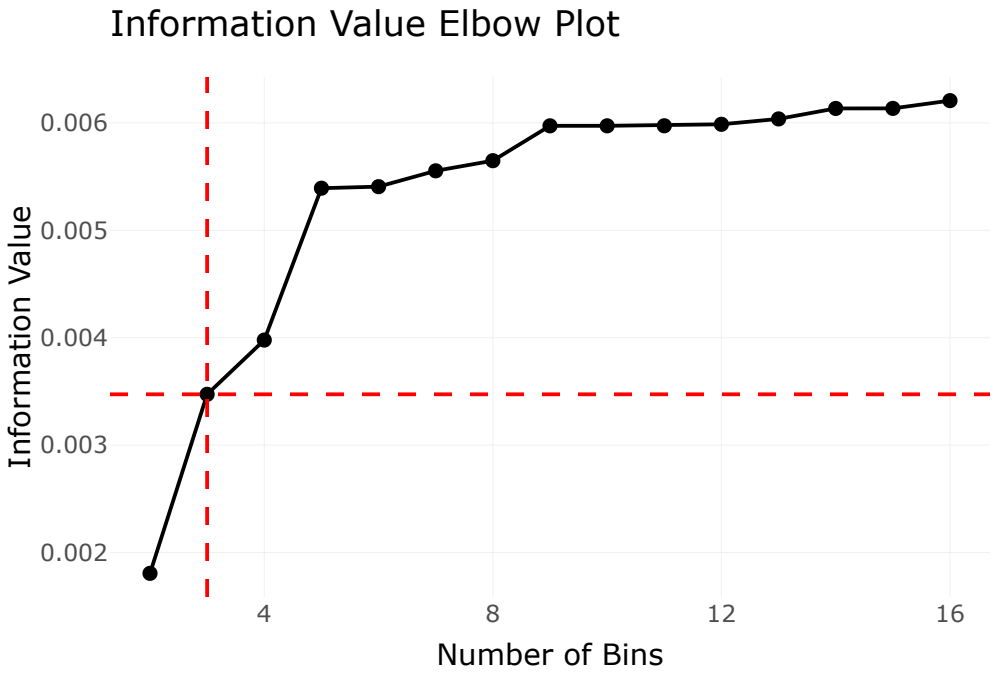


We see that most information is captured by the first bin indicating that those with a BM of 0 are significantly safer than the others. The three bins are sufficient to capture the first group at bonus-malus zero, then the first and second risk groups that are the first and second half of the BM score. Adding bins just creates bins in between with low informational value.

```
data$BM_binned <- cut(
    data$BM,
    breaks = rebuild_bins(result_bm, 3), include.lowest = TRUE, ordered_result = TRUE)
n_bins_named_list[["BM"]] <- 3
```

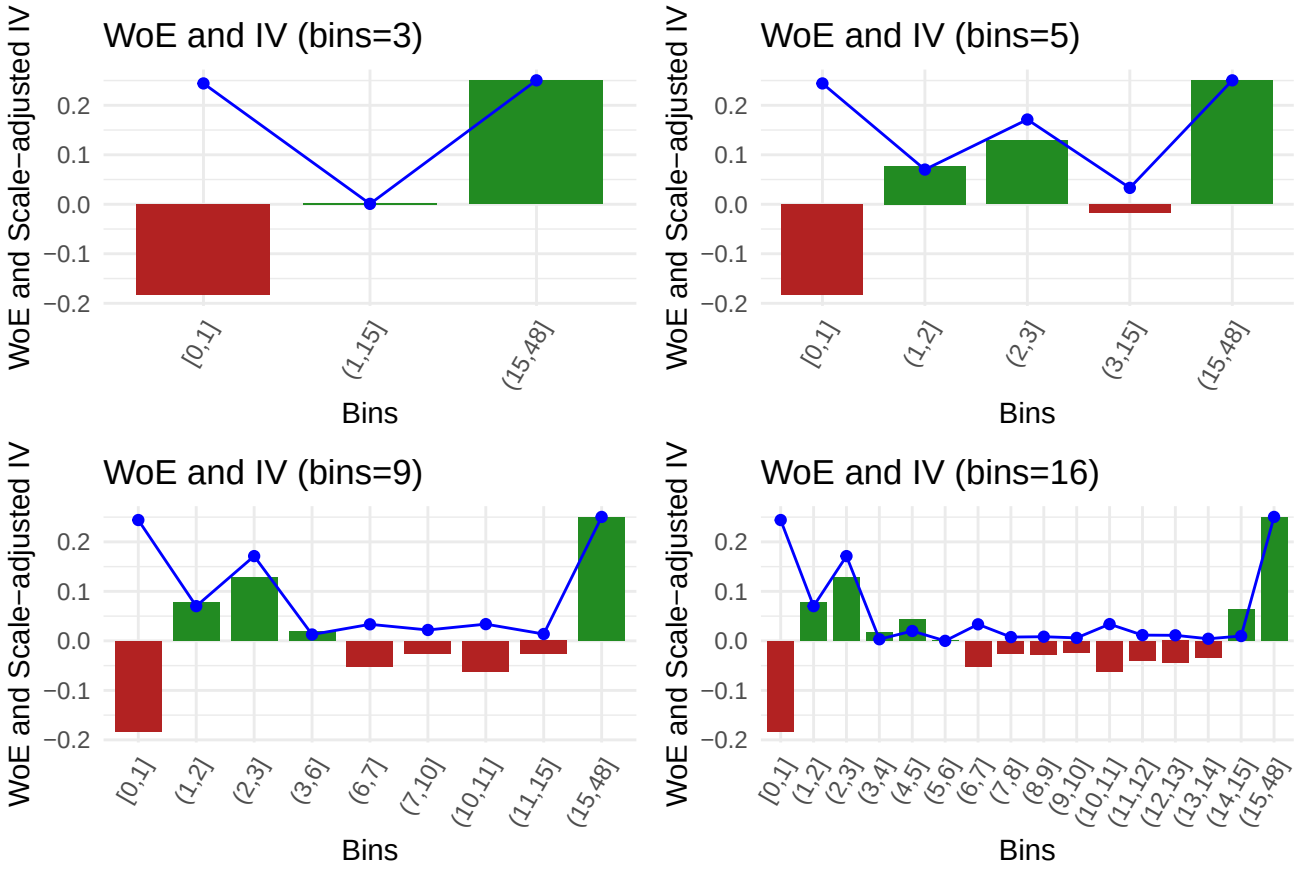
```
baseline_agec <- qbinning(data, x = "AGEC", y = "NCLAIMS", max_bins = 30)
result_agec <- iv_elbow(data, "AGEC", "NCLAIMS", baseline_agec)
result_named_list[["AGEC"]] <- result_agec
plot_iv_elbow(result_agec, interactive = TRUE)
```

2.2.2.5 Age of the Car Binning (AGEC)


Information Value Elbow Plot
Information Value
0.006
0.005
0.004
0.003
0.002
4
8
12
16
Number of Bins

For the age of the car (AGEC), we also see a plateau after around 5 and 9 bins, let's compare 3, 5, 9 and 16 (max) bins.

```
plots_agec <- list()
for (i in c(3, 5, 9, 16)) {
    temp_bins_agec <- rebuild_bins(result_agec, i)
    woe_temp_agec <- woe_calc(data, x = "AGEC", y = "NCLAIMS", bins = temp_bins_agec)
    plots_agec[[i]] <- plot_woe_iv(woe_temp_agec, interactive = FALSE)
}
(plots_agec[[3]] | plots_agec[[5]]) / (plots_agec[[9]] | plots_agec[[16]])
```



The maximum binning shows us a clear separation of risk between:

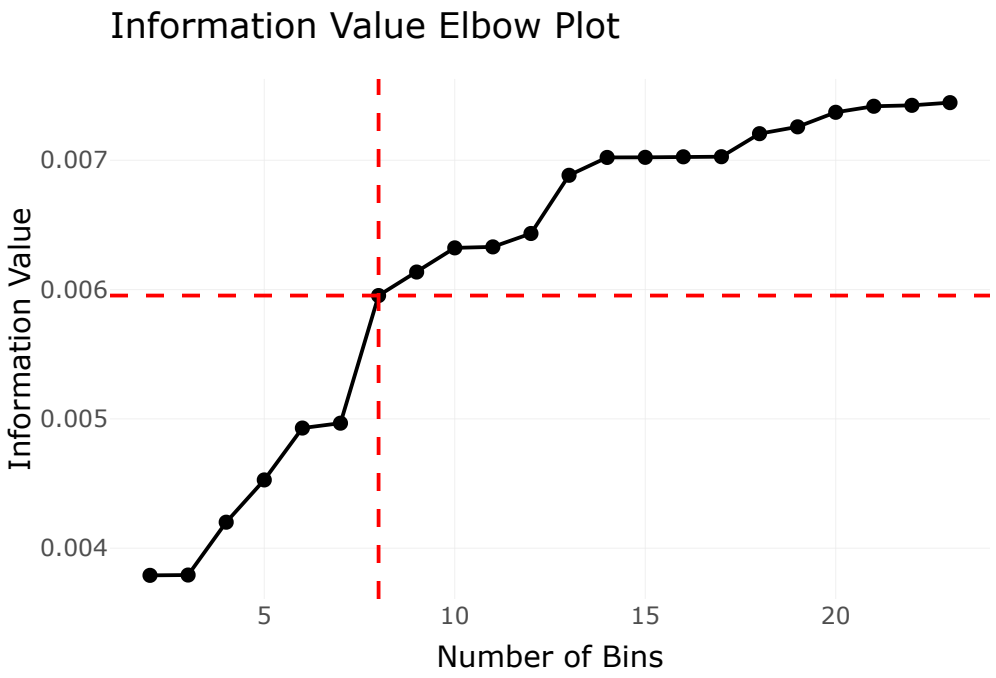
- Brand new cars (AGEC close to 0) have the highest risk.
- Older cars (AGEC over 15 years) have a lower risk.

This is captured from 3 bins onward. Using 5 bins would allow us to capture the low-risk group for cars of 1 to 3 years old separately from the older low-risk cars. These cars are typically good new cars with conductors that have had the time to get used to them. We will thereforeuse 5 bins for the age of the car.

```r
data$AGEC_binned <- cut(
    data$AGEC,
    breaks = rebuild_bins(result_agec, 5)
    , include.lowest = TRUE, ordered_result = TRUE)
n_bins_named_list[["AGEC"]] <- 5
```

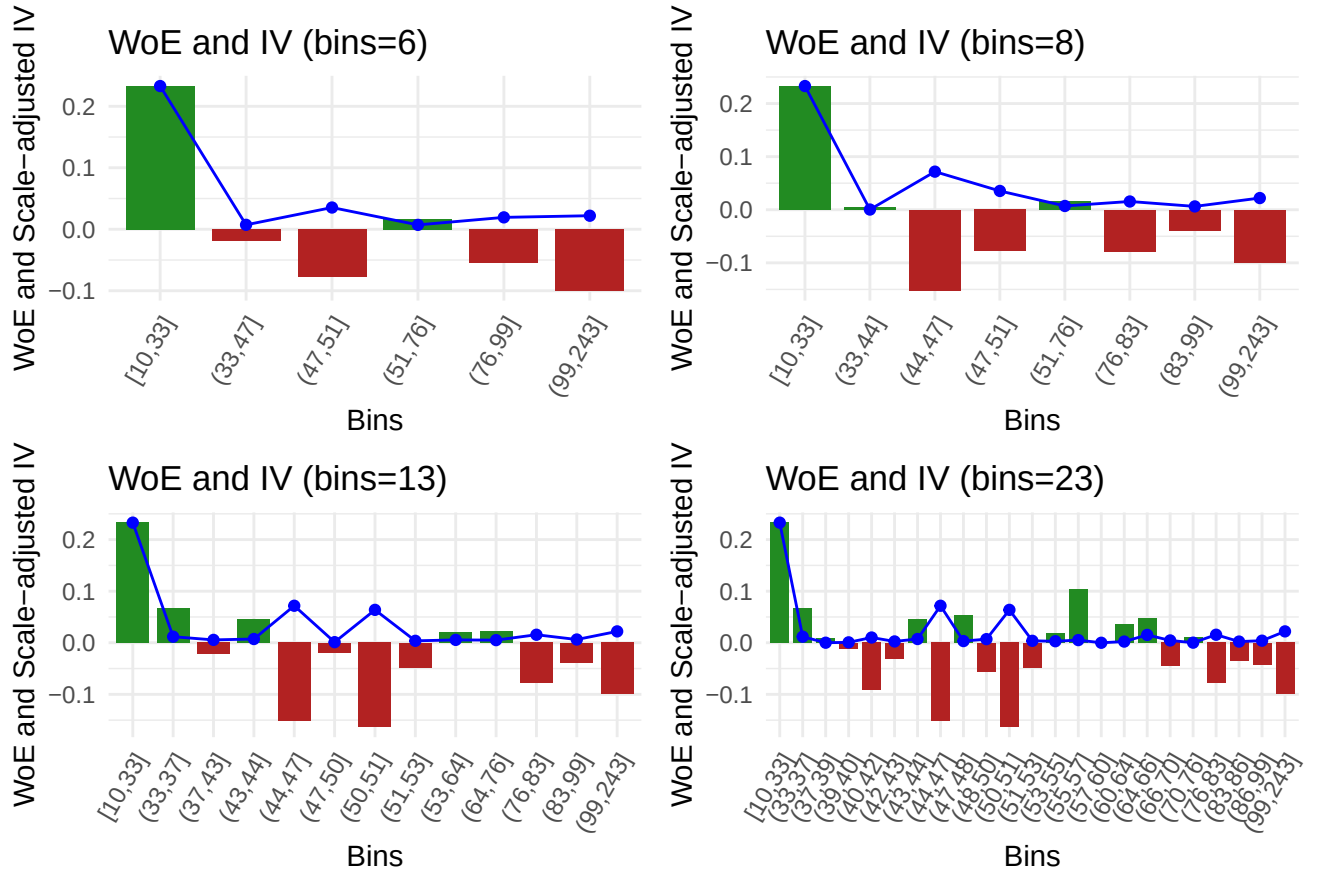
```r
baseline_power <- qbinning(data, x = "POWER", y = "NCLAIMS", max_bins = 30)
result_power <- iv_elbow(data, "POWER", "NCLAIMS", baseline_power)
result_named_list[["POWER"]] <- result_power
plot_iv_elbow(result_power, interactive = TRUE)
```

2.2.2.6 Power Binning (POWER)


Information Value Elbow Plot
Information Value
0.007
0.006
0.005
0.004
5
10
15
20
Number of Bins

For the power of the car (POWER), we have stairs at 6, 8, and 13 bins. We'll compare these three steps with the max level of bins (23).

```
plots_power <- list()
for (i in c(6, 8, 13, 23)) {
    temp_bins_power <- rebuild_bins(result_power, i)
    woe_temp_power <- woe_calc(data, x = "POWER", y = "NCLAIMS", bins = temp_bins_power)
    plots_power[[i]] <- plot_woe_iv(woe_temp_power, interactive = FALSE)
}
(plots_power[[6]] | plots_power[[8]]) / (plots_power[[13]] | plots_power[[23]])
```



The power constribution is harder to interpret. We have the low-power cars that are safer (lower claims), and high power that are more dangerous (higher claims). The middle range is not as clear. We notice two informative risk pockets (bins=12). A quick web search indicates these interpretations [4]:

- Power between 44 and 47: this is the minimum legal and practical power for modern highway-legal cars. Below this range, cars are typically underpowered for regular driving conditions.

[4]The interpretations are the result of google IA's search and should be taken with caution. A true market research and assessment of the hidden factors is outside the scope of this assignment.

- Power between 50 and 51: this is widely considered the mechanical sweet spot to maintain highway speed withouf sacrificing fuel economy in urban driving. It is typically a marker of entry-level city cars.

Based on this interpretation, the power could be a proxy for the type of car and its urban usage. We will use 8 bins to capture this bump risk pattern without overcomplicating the model.

```
data$POWER_binned <- cut(
    data$POWER,
    breaks = rebuild_bins(result_power, 8),
    include.lowest = TRUE, ordered_result = TRUE)
n_bins_named_list[["POWER"]] <- 8
```

We will end this Data Transformation section here now that we have bins for both categorical and numerical variables.

2.3 Data Exploration

2.3.1 Graphical Approach

Based on all created bins, we can visualize the claim frequency across different bins for each variable. This is what we already did in our analysis of the relation between the age of the driver and claim frequency. Remember that the MLE (Maximum Likelihood Estimator) for the claim frequency based on the hypothesis that the claims follow a Poisson distribution is the following:

$NCLAIMS_i \sim \text{Poisson}(d_i \times \lambda)$, where d_i is the exposure and λ is the claim frequency rate.

$$\hat{\lambda} = \frac{\sum_i NCLAIMS_i}{\sum_i d_i}$$

$$\hat{\lambda} \sim \text{Normal}\left(\hat{\lambda}, \frac{\hat{\lambda}}{\sum_i d_i}\right)$$

We find the variance as the inverse of the fisher information (itself equal to $-\ell''(\widehat{\lambda})$)

$$\text{Var}(\widehat{\lambda}) \approx \mathcal{I}(\widehat{\lambda})^{-1} = \frac{\widehat{\lambda}}{\sum_i d_i}.$$

Alternatively, with the gaussian approximation on $NCLAIMS_i$, we have:

$$NCLAIMS_i \sim \text{Normal}(d_i \times \lambda, d_i \times \lambda)$$

$$\hat{\lambda} = \frac{\sum_i NCLAIMS_i}{\sum_i d_i}$$

$$\hat{\lambda} \sim \text{Normal}\left(\frac{\sum_i d_i \lambda}{\sum_i d_i}, \frac{\sum_i d_i \lambda}{(\sum_i d_i)^2}\right)$$

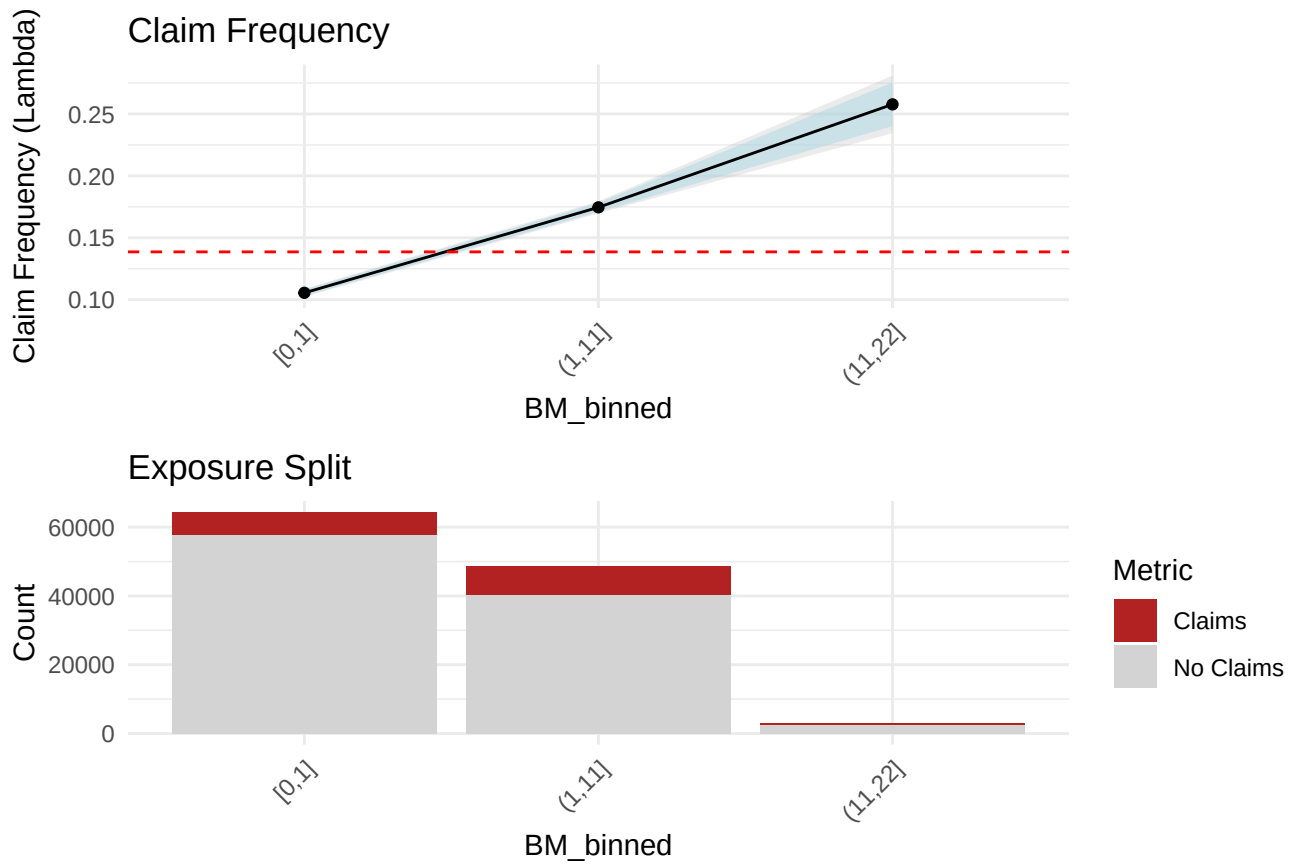
$$\hat{\lambda} \sim \text{Normal}\left(\lambda, \frac{\lambda}{\sum_i d_i}\right)$$

Based on this mathematical understanding, we can build a main plot which shows exposures, claims, and claim frequencies with their confidence intervals (95 and 99%) across different bins for each variable.

```
plot_claim_freq <- function(data, variable) {
    df_grouped <- data %>%
        mutate(bin = data[[variable]]) %>%
        group_by(bin) %>%
        summarise(
            sum_exp = sum(EXP),
            sum_claims = sum(NCLAIMS),
            lambda = sum_claims / sum_exp,
            sd_lambda = sqrt(lambda / sum_exp),
            lambda_95_lower = lambda - 1.96 * sd_lambda,
            lambda_95_upper = lambda + 1.96 * sd_lambda,
            lambda_99_lower = lambda - 2.58 * sd_lambda,
            lambda_99_upper = lambda + 2.58 * sd_lambda
        )
    df_grouped$sum_no_claims <- df_grouped$sum_exp - df_grouped$sum_claims
    df_counts <- bind_rows(
        mutate(df_grouped, bin = bin, value = sum_claims, metric = "Claims"),
        mutate(df_grouped, bin = bin, value = sum_no_claims, metric = "No Claims")
    )
    p_counts <- ggplot(df_counts, aes(x = bin, y = value, fill = metric)) +
        geom_bar(stat = "identity") + theme_minimal() +
        scale_fill_manual(values = c("Claims" = "firebrick", "No Claims" = "lightgray")) +
        labs(x = variable, y = "Count", fill = "Metric") +
        theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
        ggtitle("Exposure Split")
    p_freq <- ggplot(df_grouped, aes(x = bin, group = 1)) +
        geom_ribbon(
            aes(ymin = lambda_99_lower, ymax = lambda_99_upper),
            fill = "gray", alpha = 0.3
        ) +
        geom_ribbon(
            aes(ymin = lambda_95_lower, ymax = lambda_95_upper),
            fill = "lightblue", alpha = 0.5
        ) +
        geom_line(aes(y = lambda), color = "black") +
        geom_point(aes(y = lambda), color = "black") +
        geom_hline(yintercept = mean_claim_frequency, color = "red", linetype = "dashed") +
        theme_minimal() +
        labs(
            x = variable,
            y = "Claim Frequency (Lambda)",
```

```
        ) +
        theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
        ggtitle("Claim Frequency")
    p_all <- p_freq / p_counts
    p_all
}
```

```
plot_claim_freq(data, "BM_binned")
```



For this first example on the Bonus-Malus binned variable, we see a clear upward trend in the claim frequency as the Bonus-Malus level increases. This is expected as risky behaviour is translated in higher score on the BM scale. What is also interesting is that the claim frequency is a lot more precise for the 0 to 1 bin as the exposure is much higher compared to the higher BM levels. The confidence intervals also enable us to quickly test hypothesis that the claim frequency for a given bin is significantly different from the overall mean claim frequency (in red) or from other bins within the same variable[5].

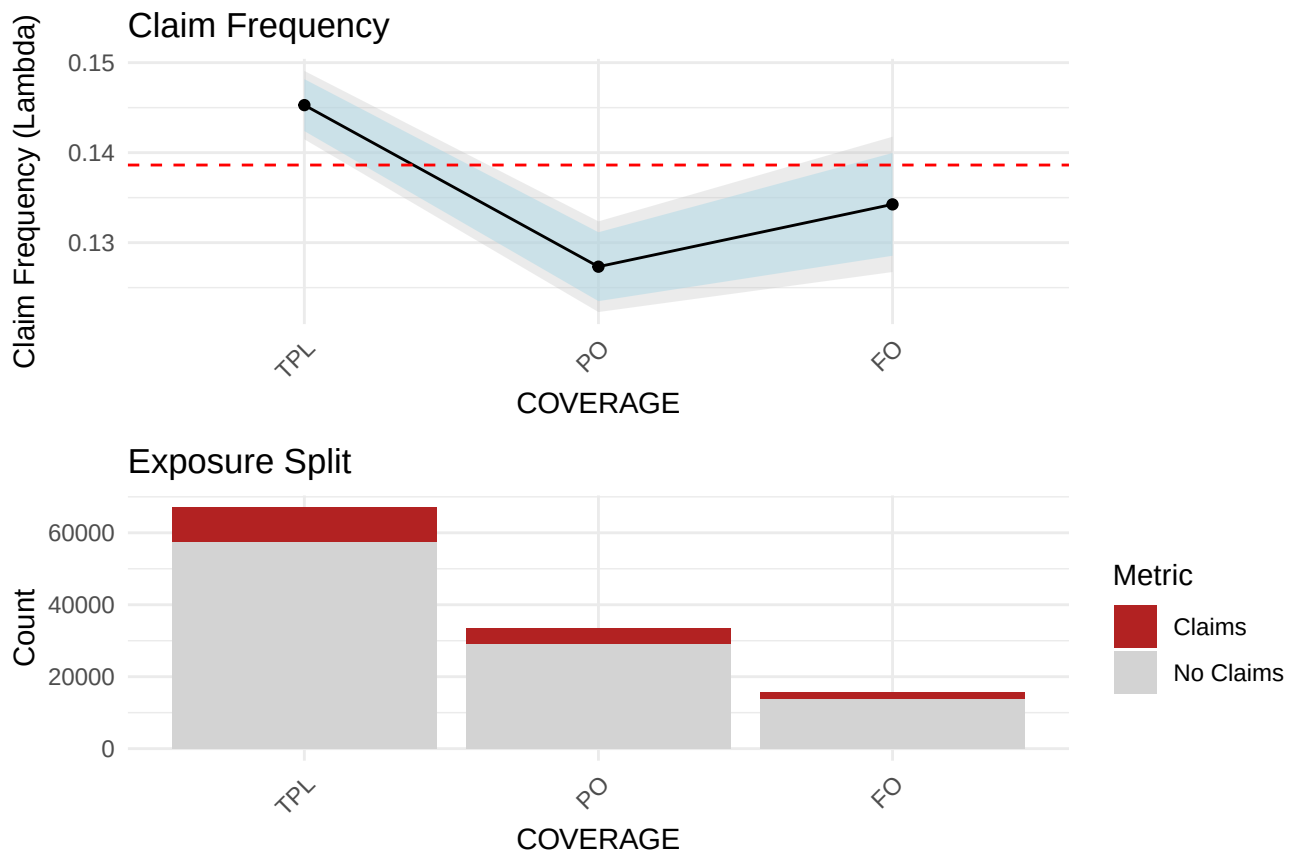
The same approach can be used for all variables:

[5]If the value is outside of the blue range, we can reject the hypothesis that the claim frequency for that bin is equal to the value with 95% certainty. For the grey interval, we can reject the hypothesis with 99% certainty.

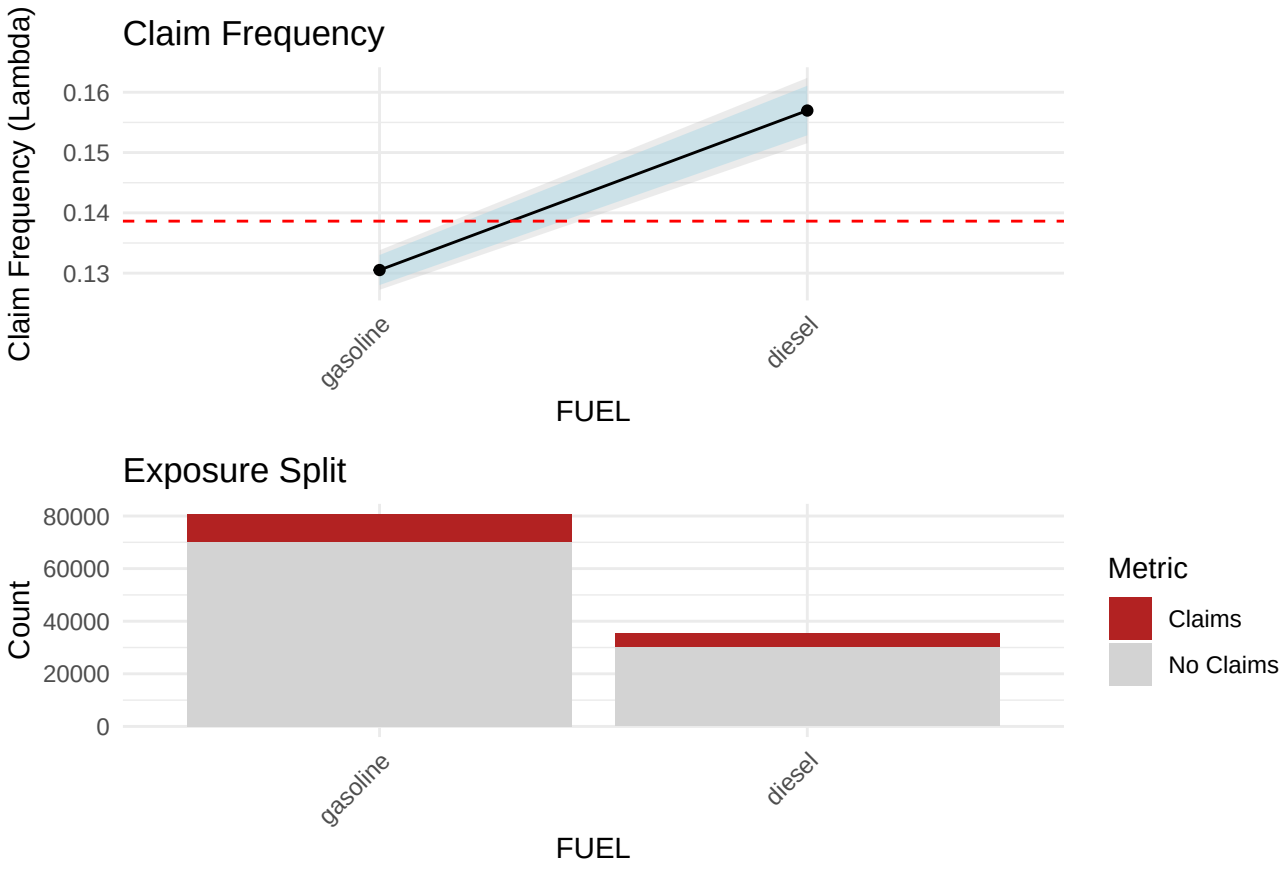
```r
for (cat_var in c("COVERAGE", "FUEL", "USE", "FLEET", "SEX")) {
    cat(paste0("> ", cat_var, "\n"))
    plot(plot_claim_freq(data, cat_var))
}
```

2.3.1.1 Categorical Variables

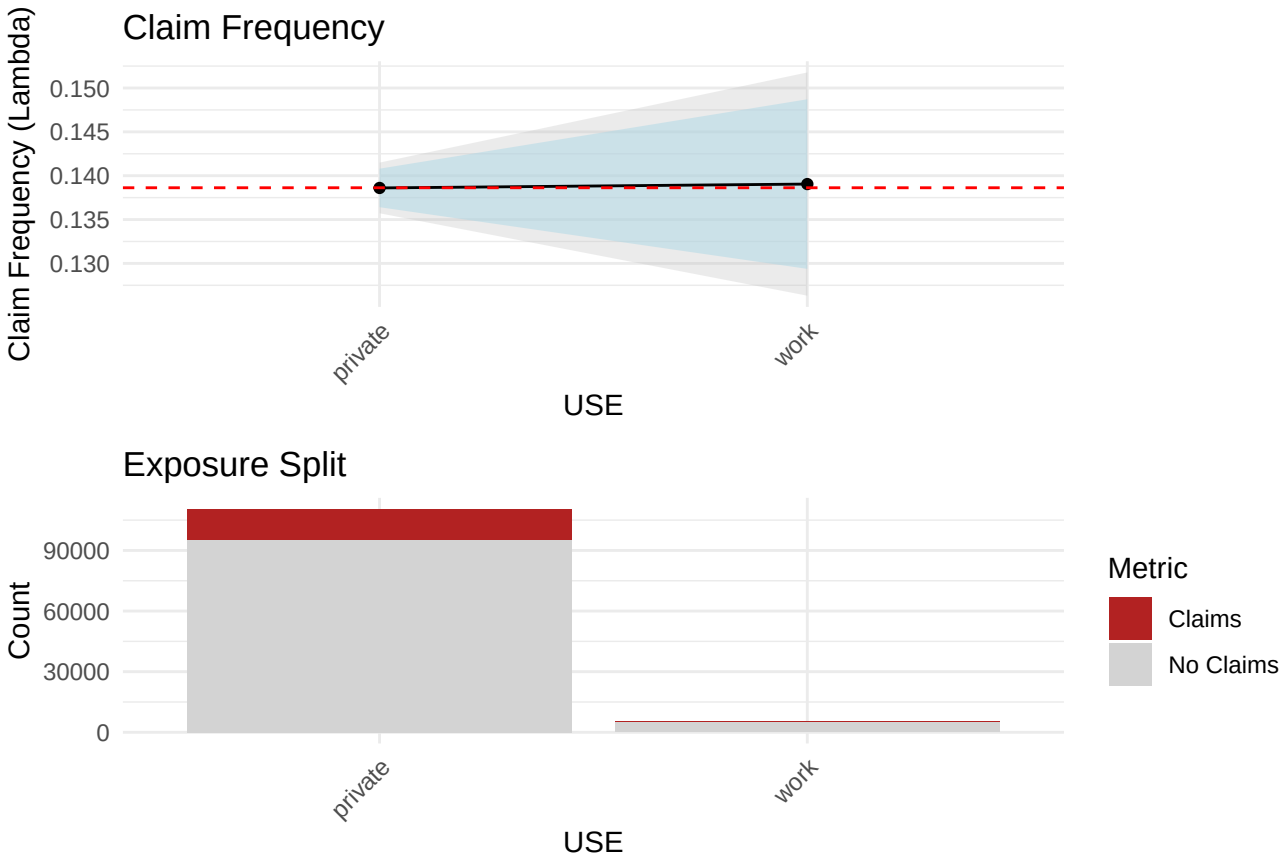
```
> COVERAGE
```



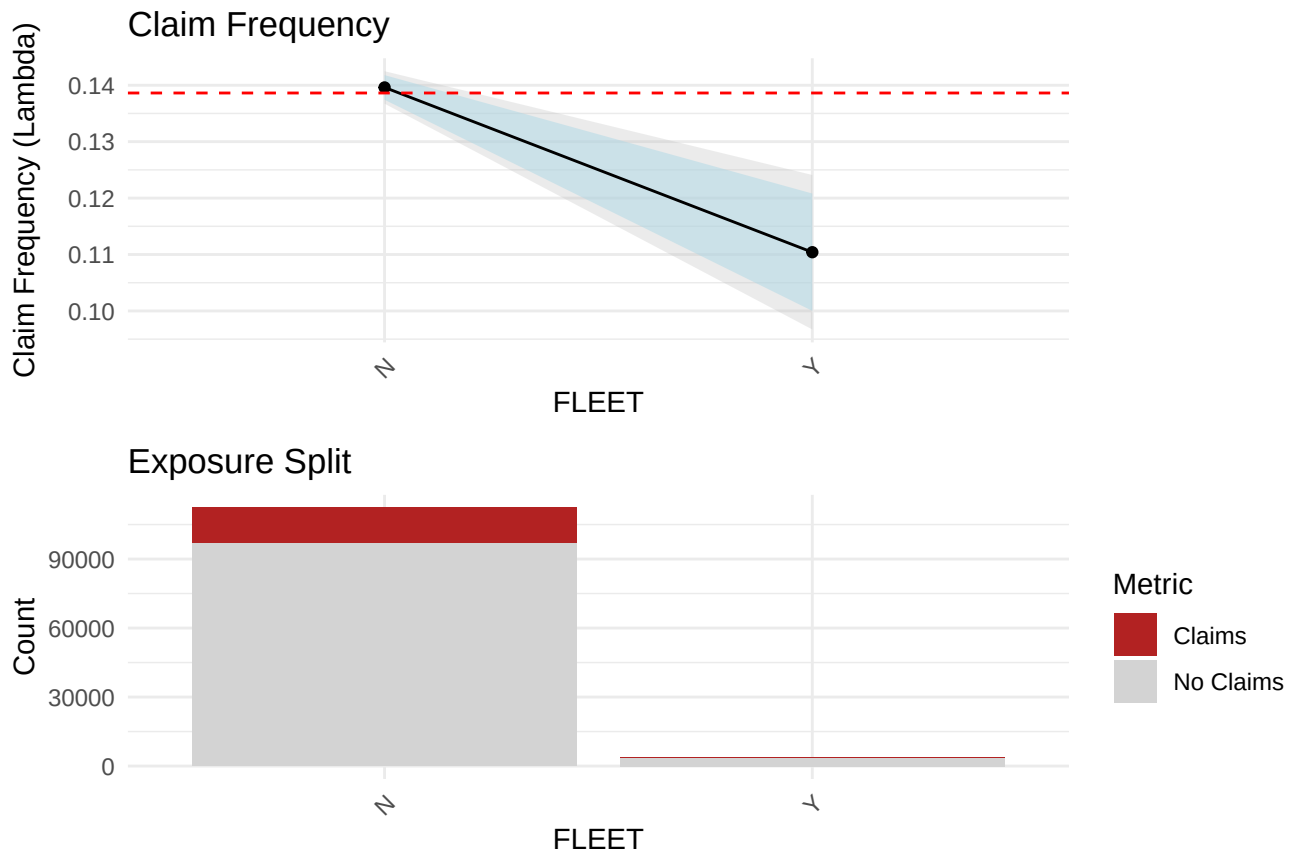

```
> FUEL
```


Claim Frequency
Claim Frequency (Lambda)
0.16
0.15
0.14
0.13
gasoline
diesel
FUEL
Exposure Split
Count
80000
60000
40000
20000
0
gasoline
diesel
FUEL
Metric
Claims
No Claims

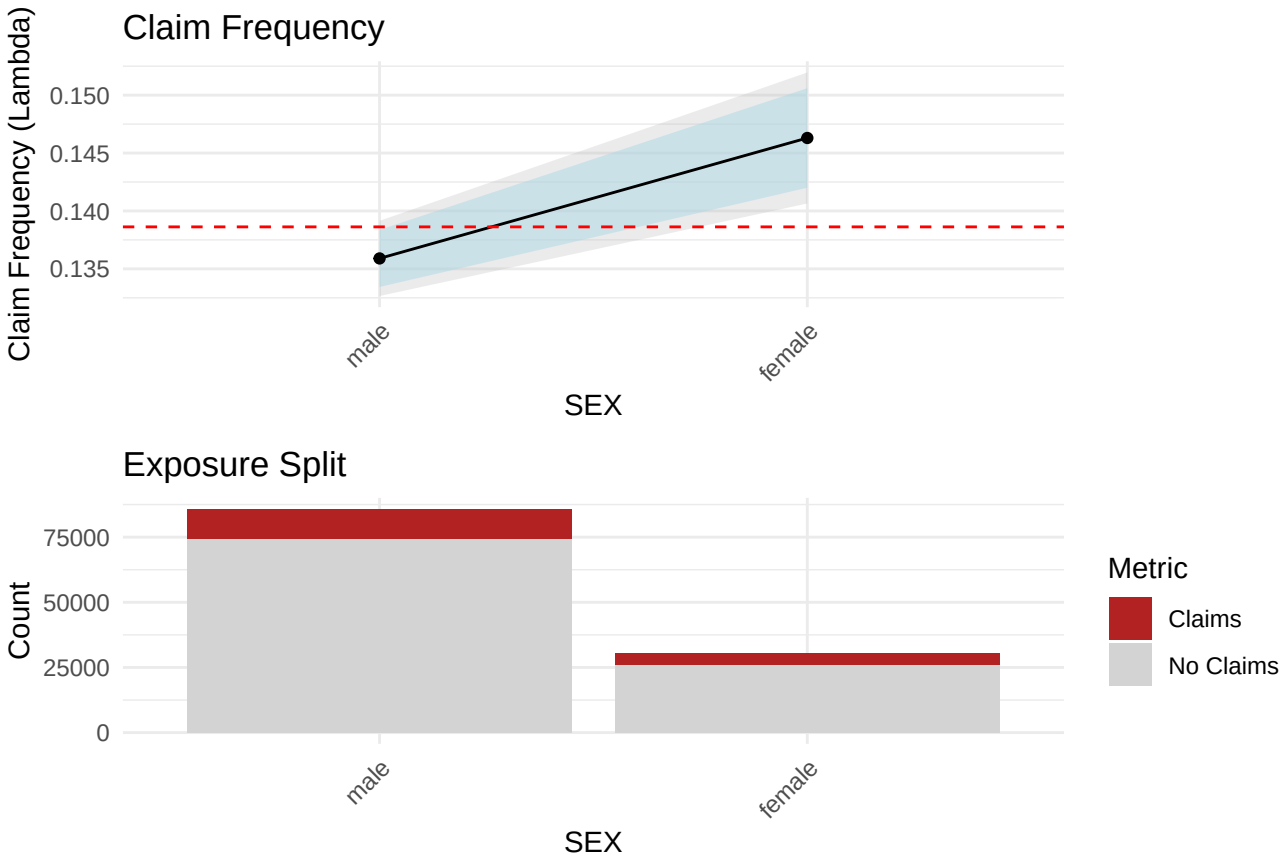
```
> USE
```


Claim Frequency
Claim Frequency (Lambda)
0.150
0.145
0.140
0.135
0.130
private
work
USE
Exposure Split
Count
90000
60000
30000
0
private
work
USE
Metric
Claims
No Claims

```
> FLEET
```


Claim Frequency
Claim Frequency (Lambda)
0.14
0.13
0.12
0.11
0.10
N
Y
FLEET
Exposure Split
Count
90000
60000
30000
0
N
Y
FLEET
Metric
Claims
No Claims

```
> SEX
```

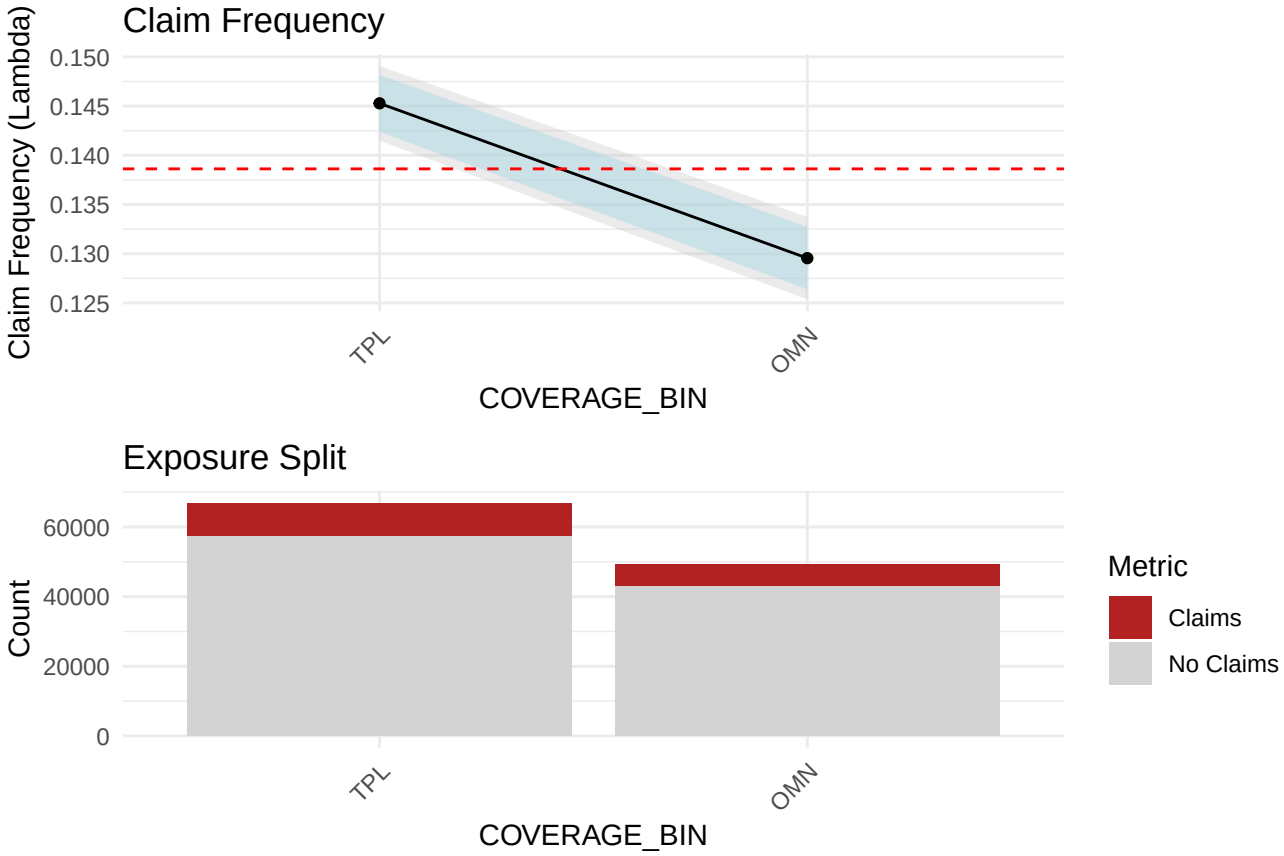


Based on the graphs, here are the preliminary conclusions for univariate analysis of categorical variables:

- `COVERAGE`

The claim frequency between partial and full omnium is not statistically different. However both are different from third-party liability only. It is interesting to see that the claim frequency is higher for these who choose lesser coverage. This can be interpreted as a proxy for risk-awareness, financial education, or even purchasing power. Based on this exploration, we will merge the partial and full omnium to focus on the significant difference.

```
data$COVERAGE_BIN <- ifelse(data$COVERAGE == "TPL", "TPL", "OMN")
data$COVERAGE_BIN <- as.factor(data$COVERAGE_BIN) %>% fct_infreq()
plot_claim_freq(data, "COVERAGE_BIN")
```



- `FUEL`

Diesel vehicles have significantly higher claim frequency compared to petrol vehicles. There is no obvious explanation for this statistical difference.

- `USE`

Private and professional use show no statistical differences from one another, nor from the mean. We will not include this variable in our models to avoid noise. Note that the proportion of policies related to work is very low, this has three consequences:

1. The private use looks very similar to the average as it represents the majority of the policies.
2. The work use is to variable to be statistically significant due to its low representation.
3. The variable is too unbalanced to provide reliable insights for modeling purposes.

- `FLEET`

Let's compare the graph for fleet to the one of `USE`. Here again, we have a very unbalanced variable.

1. The majority of policies are non-fleet, which makes the claim frequency for this category very similar to the overall average.

2. Despite its low representation, the fleet category is significantly less risky than the non-fleet category.
3. The variable has predictive potential despite the high volatility of the non-fleet category.

- `SEX`

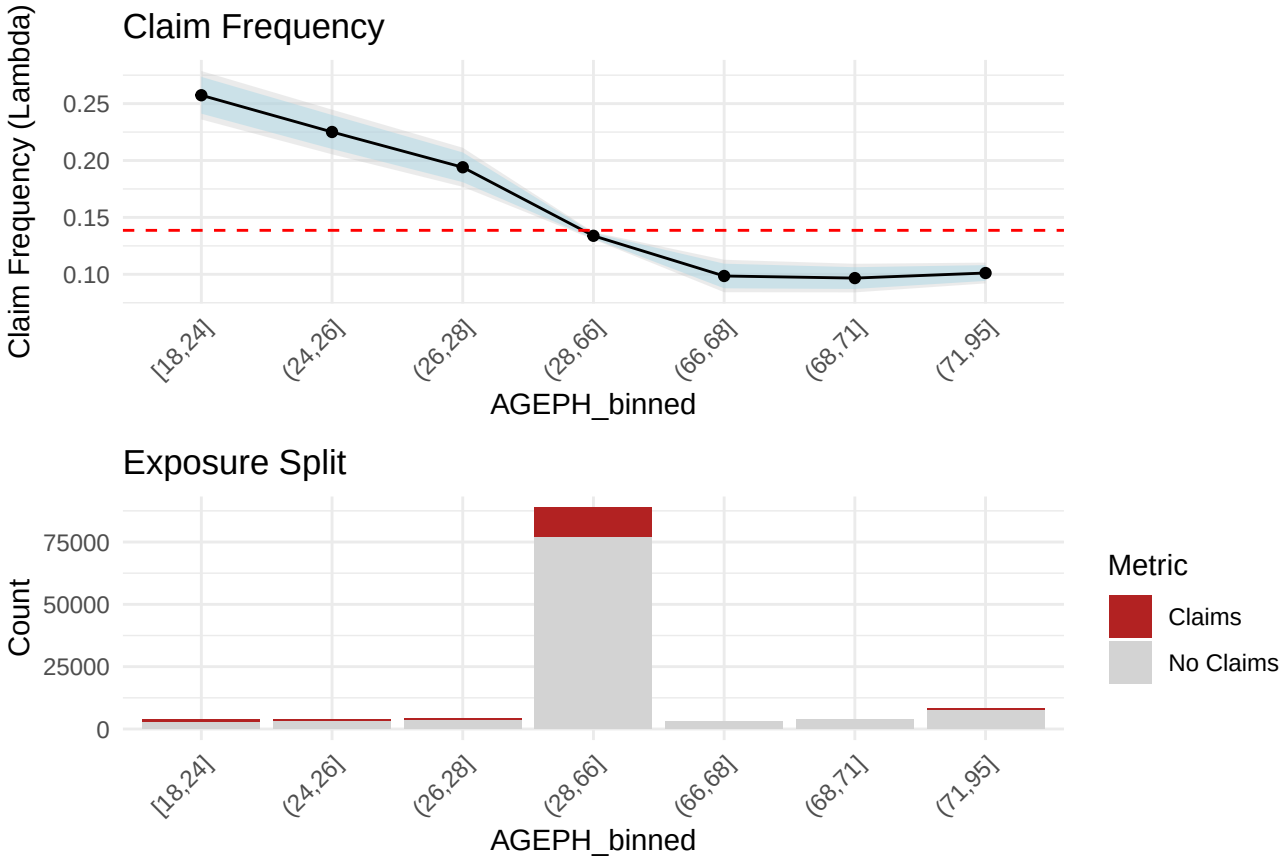
Similarly to the interpretation for the `FLEET`, there is a significant difference in claim frequency even if one category is less represented than the other. In this case, the unbalance is less strong, the males are thus slightly further from the average and females are significantly more prone to claims than males. This variable also has predictive potential for modeling purposes.

2.3.1.2 Numerical Variables

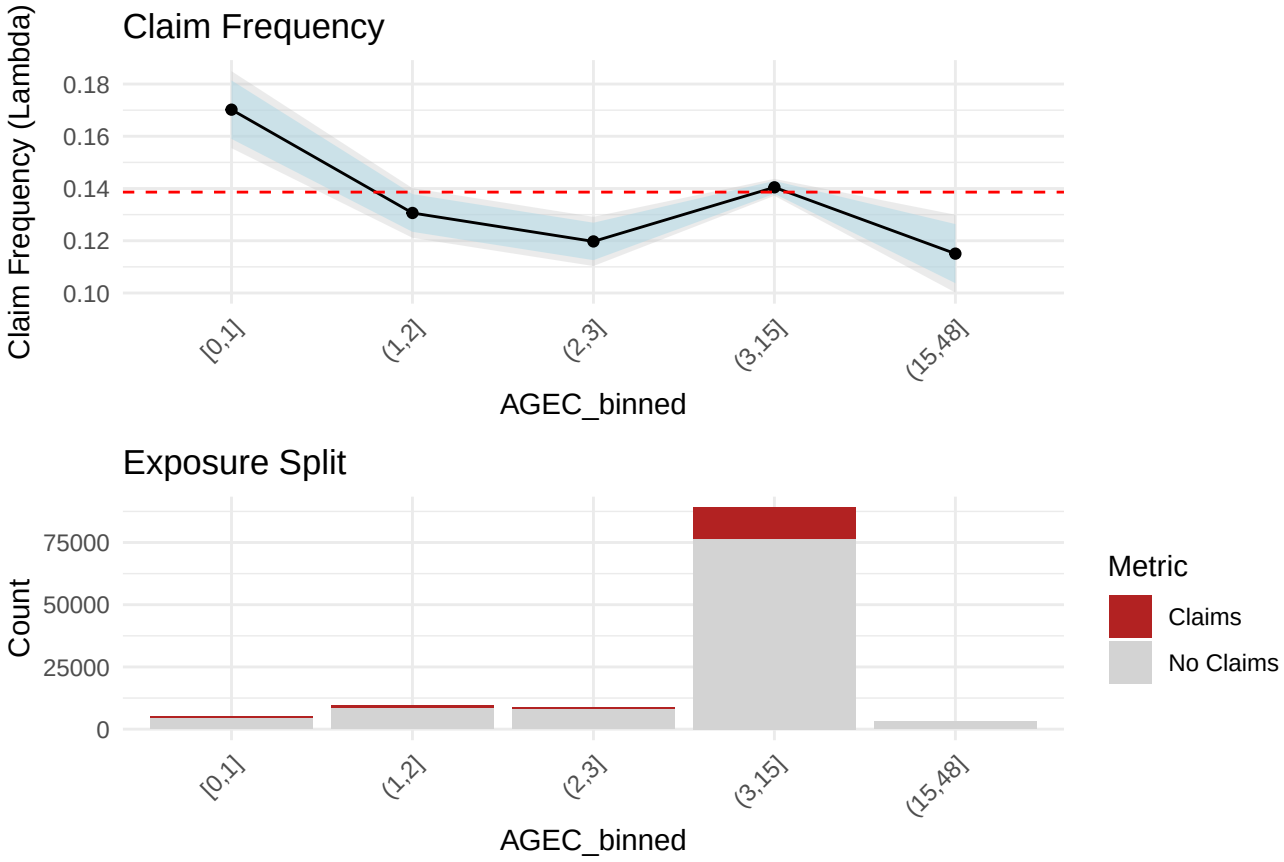
Let us now have a look at the binned numerical variables other than the bonus-malus scale that we already interpreted.

```
for (cat_var in c("AGEPH_binned", "AGEC_binned", "POWER_binned")) {
    cat(paste0("> ", cat_var, "\n"))
    plot(plot_claim_freq(data, cat_var))
}
```

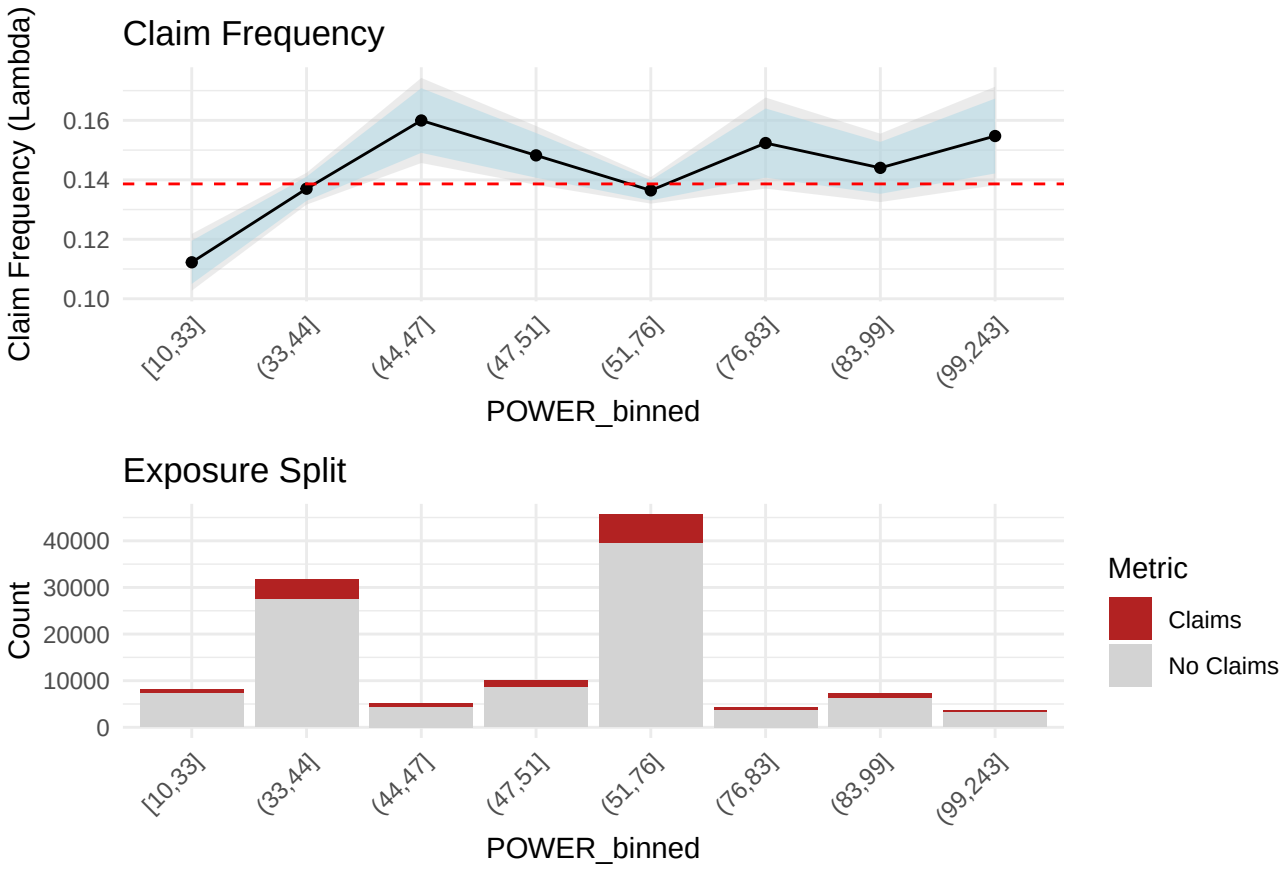
```
> AGEPH_binned
```



```
> AGEC_binned
```




```
> POWER_binned
```



- `AGEPH_binned`

We see the risk diminution as the age of the policyholder increases as we identified with our binning algorithm. However, we see that the last three are not significantly different from one another in terms of claim frequency. We will come back to this in the statistical approach section and see with an anova test if the 7 bins are significantly better than a reduced number of bins.

- `AGEC_binned`

For the age of the car, we can confirm our previous observations that brand new cars (0-1 years old) have more claims. We can also confirm the lower risk for cars between 1 and 3 years old and above 15 years. The remaining question is whether the bins 1 to 2 and 2 to 3 should be merged in a single bin. We will revisit this question in the statistical approach section to determine if merging these bins significantly affects the model performance.

- `POWER_binned`

Thanks to our binning algorithm, we correctly identified the low-risk for power lower than 33 kilowatts. We also correctly identified the very bump-shaped peak in risk for power 44 to 51 kilowatts. It is not clear whether the last 3 bins should be merged or kept separate, as the risk pattern is less pronounced in this range. We will also investigate this in the statistical approach section.

2.3.2 Statistical Approach

We already addressed the statistical significance by visually checking the confidence intervals around the claim frequencies. In this statistical data exploration, we will add two anova layers onto that visual inspection:

2.3.2.1 Global Anova Test: We will compare the deviance of the model with the variable as single predictor to the null model with only the offset term. This will allow us to test if the variable significantly improves the model fit:

$H_0 : \Delta D = D_0 - D_1 = 0$ $H_1 : \Delta D = D_0 - D_1 > 0$

For our test statistic we have $\Delta D \sim \chi^2_{df}$, where df is the difference in the number of parameters between the null and alternative models.

Our D_0 represents the deviance of the null model with only the offset term: `glm(NCLAIMS ~ offset(log(EXP)))`. The D_1 represents the deviance of the model with the variable as a single predictor: `glm(NCLAIMS ~ offset(log(EXP)) + get(var))`.

```
m0 <- glm(NCLAIMS ~ offset(log(EXP)), data = data, family = poisson(link = "log"))
cat(sprintf("Because we use Poisson with its canonic link, note exp(Beta0) = %f is equal to the
```

```
Because we use Poisson with its canonic link, note exp(Beta0) = 0.138624 is equal to the empir
```

Let's now compare this null model to the models with each predictor variable individually.

```
test_level <- 0.05
for (var in c("COVERAGE_BIN", "FUEL", "USE", "FLEET", "SEX", "BM_binned", "AGEPH_binned", "AGE
    m1 <- glm(NCLAIMS ~ offset(log(EXP)) + get(var), data = data, family = poisson(link = "log
    test_result <- anova(m0, m1, test = "Chisq")
    cat(sprintf("Is variable %s significant? %s\n", var, ifelse(test_result$`Pr(>Chi)`[2] < tes
}
```

```
Is variable COVERAGE_BIN significant? Yes
Is variable FUEL significant? Yes
Is variable USE significant? No
Is variable FLEET significant? Yes
Is variable SEX significant? Yes
Is variable BM_binned significant? Yes
Is variable AGEPH_binned significant? Yes
Is variable AGEC_binned significant? Yes
Is variable POWER_binned significant? Yes
```

With this first global anova test, we confirm our graphical approach, all variables should be kept except for `USE`.

2.3.2.2 Merged bins

Let's take the example of `COVERAGE` where we decided to merge the two omnium levels. Can we justify this statistically with the ANOVA test instead of simply with the confidence intervals?

```
m0 <- glm(NCLAIMS ~ offset(log(EXP)) + COVERAGE_BIN, data = data, family = poisson(link = "log"
m1 <- glm(NCLAIMS ~ offset(log(EXP)) + COVERAGE, data = data, family = poisson(link = "log"))
test_result <- anova(m0, m1, test = "Chisq")
pval <- test_result$`Pr(>Chi)`[2]
cat(sprintf("Was it justified to merge the COVERAGE bins? %s", ifelse(pval < 0.05, "No", "Yes"))
```

```
Was it justified to merge the COVERAGE bins? No
```

```
cat(sprintf("Are you 99%% sure that it was not justified? %s", ifelse(pval < 0.01, "Yes", "No"))
```

```
Are you 99% sure that it was not justified? No
```

Because the p-value is 0.0469242, we can conclude that we reject the hypothesis that the deviance was different between the merged and unmerged COVERAGE bins at a 95% confidence level but not at a 99% confidence level. It was therefore not a bad idea to merge the COVERAGE bins based on our graphical analysis based on confidence intervals.

We can apply the same reasoning to clarify whether or not we should merge:

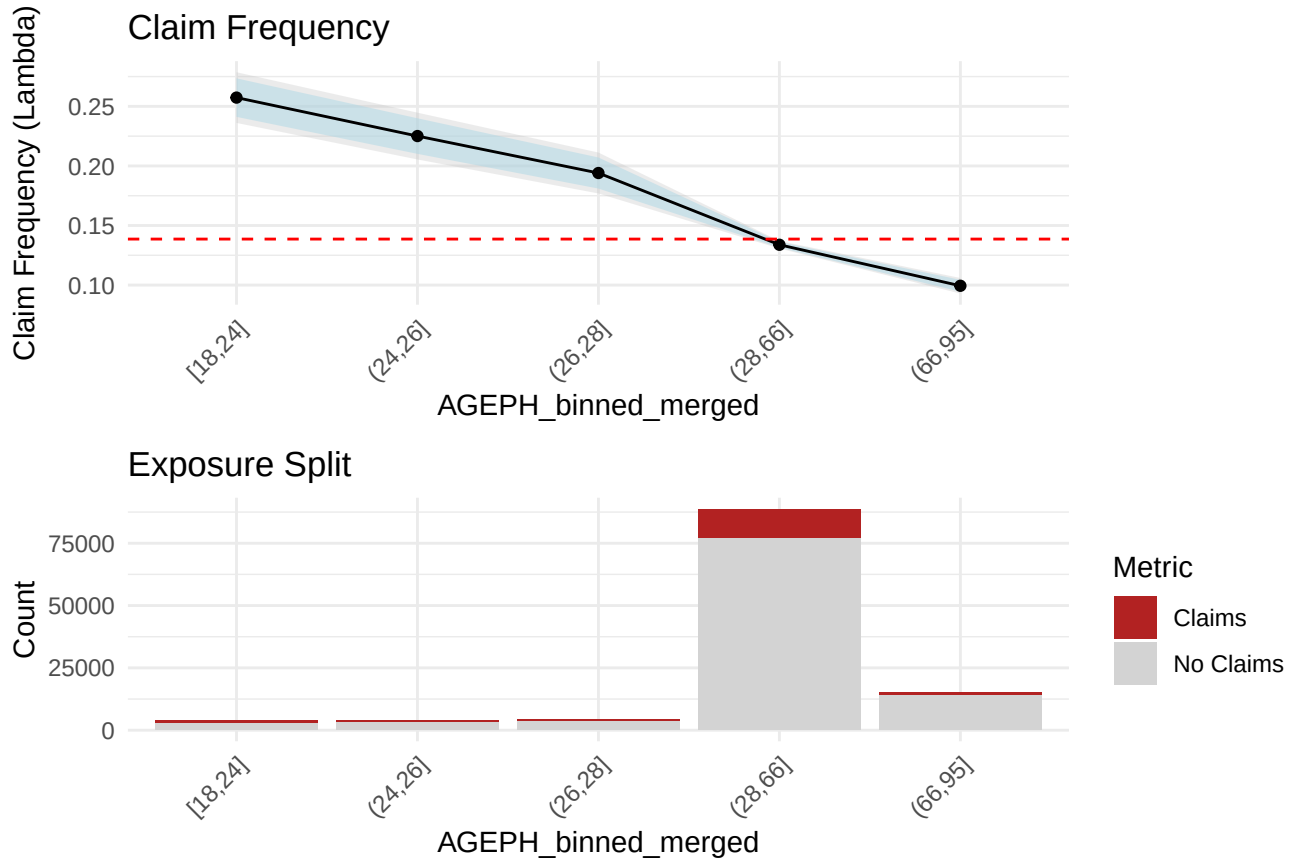
- The last 3 bins of `AGEPH_binned`

```
data$AGEPH_binned_merged <- fct_collapse(
    data$AGEPH_binned,
    "(66,95]" = c("(66,68]", "(68,71]", "(71,95]")
)

m0 <- glm(NCLAIMS ~ offset(log(EXP)) + AGEPH_binned, data = data, family = poisson(link = "log"
m1 <- glm(NCLAIMS ~ offset(log(EXP)) + AGEPH_binned_merged, data = data, family = poisson(link
test_result <- anova(m0, m1, test = "Chisq")
pval <- test_result$`Pr(>Chi)`[2]

if(pval < 0.05) {
    cat(sprintf("Merging the last 3 bins of AGEPH_binned was not justified (p-value = %f).\n",
    data$AGEPH_binned_merged <- NULL
} else {
    cat(sprintf("Merging the last 3 bins of AGEPH_binned was justified (p-value = %f).\n", pval
    plot(plot_claim_freq(data, "AGEPH_binned_merged"))
}
```

```
Merging the last 3 bins of AGEPH_binned was justified (p-value = 0.750730).
```



- The bins 1-2 and 2-3 of `AGEC_binned`

```
data$AGEC_binned_merged <- fct_collapse(
    data$AGEC_binned,
    "(1,3]" = c("(1,2]", "(2,3]")
)

m0 <- glm(NCLAIMS ~ offset(log(EXP)) + AGEC_binned, data = data, family = poisson(link = "log")
m1 <- glm(NCLAIMS ~ offset(log(EXP)) + AGEC_binned_merged, data = data, family = poisson(link =
test_result <- anova(m0, m1, test = "Chisq")
pval <- test_result$`Pr(>Chi)`[2]

if(pval < 0.05) {
    cat(sprintf("Merging the second and third bins of AGEC_binned was not justified (p-value =
    data$AGEC_binned_merged <- NULL
} else {
    cat(sprintf("Merging the second and third bins of AGEC_binned was justified (p-value = %f)
    plot(plot_claim_freq(data, "AGEC_binned_merged"))
}
```

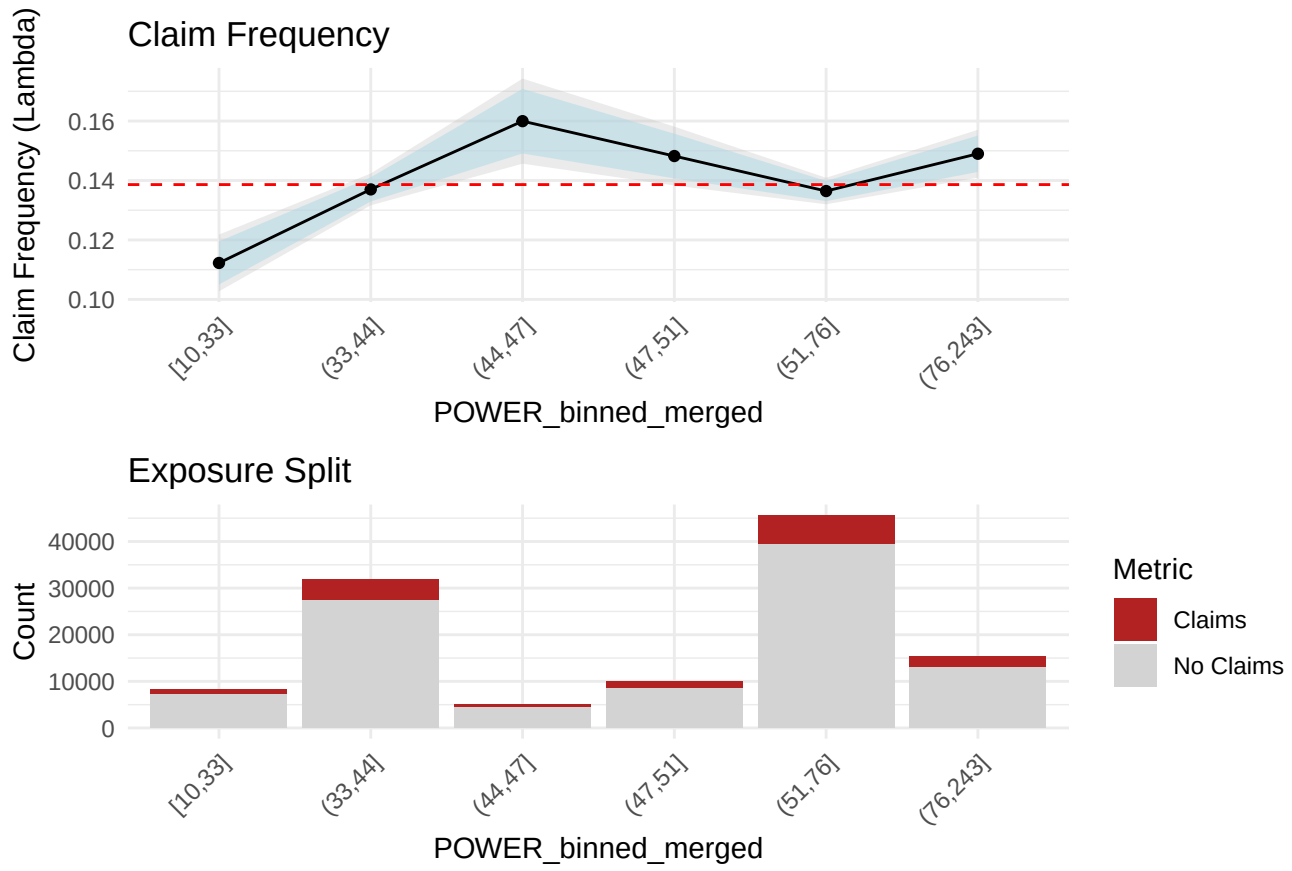
```
Merging the second and third bins of AGEC_binned was not justified (p-value = 0.036014).
```

- The last 3 bins of `POWER_binned`

```
data$POWER_binned_merged <- fct_collapse(
    data$POWER_binned,
    "(76,243]" = c("(76,83]", "(83,99]", "(99,243]")
)
m0 <- glm(NCLAIMS ~ offset(log(EXP)) + POWER_binned, data = data, family = poisson(link = "log
m1 <- glm(NCLAIMS ~ offset(log(EXP)) + POWER_binned_merged, data = data, family = poisson(link
test_result <- anova(m0, m1, test = "Chisq")
pval <- test_result$`Pr(>Chi)`[2]

if(pval < 0.05) {
    cat(sprintf("Merging the last 3 bins of POWER_binned was not justified (p-value = %f).\n",
    data$POWER_binned_merged <- NULL
} else {
    cat(sprintf("Merging the last 3 bins of POWER_binned was justified (p-value = %f).\n", pval
    plot(plot_claim_freq(data, "POWER_binned_merged"))
}
```

```
Merging the last 3 bins of POWER_binned was justified (p-value = 0.307144).
```



We have now have thoroughly examined the data we had available exced for the `TOWN` and `PC` variables. We will come back to them in the feature engineering section.

2.4 Feature Engineering

Besides the existing features, we can create additional features to add information to our design matrix and potentially improve our models.

- **Geospatial Features**: We will first explore the existing `PC` and `TOWN` variables‘. Note that these are simply categorical representations of the longitude and latitude for different granularities of location. We will then explore different granularities and use the spatial information to enrich our feature set.
- **Polynomial Features**: As we will see in the modelling section, adding polynomial features can be a good way to capture non-linear relationships in our GLM models. This is an intermediate step towards the smoothing functions of Generalized Additive Models (GAMs). We can test the second and third degree polynomial features for continuous variables and bin the statistically significant features to create new categorical variables for our GLM. Note that this is less straighforward to explain to a customer. It is best used in technical premium modeling but should likely be removed from the commercial premium models.
- **Interaction Features**: All the analysis we have done so far has focused on individual features. The statistical modelling (GLM, GAM) will allow us to capture the controlled effects of the features. However, some interactions between features are not captured by linear terms alone. We can create interaction features to explicitly model these effects.

2.4.1 Geospatial Features

Location can be considered at multiple levels of granularity. In Belgium, the `National Institute of Statistics` provides a `NIS code` nomenclature that uniquely identifies each location at various granularities, from most specific to most general:

- `Statistical Sector`:(~20,000 sectors) that are granular enough to capture neighborhood variations within a postal code. This is typically interesting for sociodemographic (socioeconomic indexes, population density, etc.) analysis.
- `Postal Code (PC)`:(~600 codes) a more aggregated level, capturing aggregated characteristics. This is a more practical level for many analyses to avoid missing data issues and respecting privacy concerns on limited datasets.
- `Administrative Arrondissement`:(~40 arrondissements) this might be the less informative level for us as it is very aggregated but is not a political division yet.
- `Province`:(~10 provinces) the political implications should be considered at this level as provinces have a certain degree of autonomy. Note that Brussels-Capital does not have this level of granularity as it is a region.
- `Region`:(3 regions) the highest level of granularity, the biggest political and administrative decision-making occurs at this level. It captures a different kind of information compared to the more localized levels.

Choosing the best level of analysis is not an easy task. It first depends on data availability and portfolio size. For smaller portfolios or when data is sparse, higher-level aggregations like provinces will allow for more robust statistical analysis. Conversely, for larger portfolios with sufficient data, more granular levels like statistical sectors or postal codes will help better differentiate risk pockets.

Besides, granularities will often capture completly different underlying risk. Let's clarify this with a climate risk example [6] using different granularities.

- `Latitude` and `Longitude`: These **numerical** features simple indicate the point on a map. If we have a large enough geographical coverage, it will help us capture meteorological patterns. For **physical risks**, such as floods or storms, we understand that a northern country will face different risk (hail breaking the windhshield, floods damaging the cars parked outdoors) compared to a southern country where heatwaves can cause damage to tires and paintwork. In a single country such as Belgium, it won't be very helpful as such but might capture other factors such as traffic density, urbanization, or quality of infrastructure. The `Statistical Sector` or `Postal Code` can also act as proxies for these underlying factors.
- `Regions`: The **categorical** features will capture administrative and political divisions, which can be important for understanding regional policies and regulations. When we look at **transition risk**, this becomes particularly relevant. For example, when the Brussels-Capital Region restricted the access to old diesel vehicles, this became stranded assets for owners of such vehicles within the region. Regions can take other decisions that impact risk exposure, such as subsidising electric vehicles, promoting public transportation, or offering advantageous schemes for company cars. This will play a role in the behavioral patterns and risk exposure of vehicle owners within the region.

We should not "choose" between granularities as they capture different underlying information. However, **multicollinearity** is a clear risk when working with various granularities of a same feature which can lead to unstable model with hard-to-interpret individual feature effects. In our dataset, we already have the postcode (583 levels) and town (578 levels) which seem to capture the same underlying geographical information and the `LAT` and `LONG` features.

To avoid redundancy, we will drop the `TOWN` feature and keep the `PC` which is part of the official NIS nomenclature. `LAT` and `LONG` will also be kept as numerical input for the smoothing functions we will use in the GAM models. To capture political and administrative underlying factors, we will add the `Regions` feature as a categorical variable. Finally, we will fetch the demographic density for each postcode which will serve as a proxy for urbanization and traffic exposure. We expect higher frequency of claims and lower severity in areas with higher demographic density, as traffic congestion may lead to more minor accidents rather than severe ones. Conversely, we expect lower frequency and higher severity in more rural areas. It will be interesting to control the predictive power of postcode with demographic density to see if it has predictive power beyond what is captured by urbanization and traffic exposure.

2.4.1.1 Dataset Enrichment

Before the enrichment, let us drop the `TOWN` feature as it is redundant but not perfecly aligned with the postcode, we'll replace it with postcode labels.

[6] This is not in the scope of our analysis but it is an important topic worth a small introduction nowadays.

```r
data <- data %>% select(-TOWN)
```

There are two main options for enriching our dataset with geographical information:

Geospatial Enrichment

- Geocode the `LAT` and `LONG` features to obtain precise geographical coordinates. We will use the Belgian Lambert 2008 (3812) projection[7] to be in line with the NIS standards.
- Download the shapefiles for Belgian administrative boundaries.
- Perform spatial matching between the geocoded points and the administrative boundaries to enrich the dataset with region, postcode, and statistical sector information.

Code Hierarchy Enrichment

- We can take a shortcut by finding matching tables that link postcodes to refnis, and use the code to assign regions, avoiding the geocoding and spatial matching steps.
- Then we merge these tables with our dataset based on the postcode to enrich it with the corresponding region.

After this step, we can look for national statistics of population density for 1997 (year of the dataset) in each municipality and merge them with our dataset to further enrich it.

```r
conv <- read.csv("../data/postcode_refnis_conversion.csv", sep = ";") %>% data.frame()
refnis <- read.csv("../data/refnis_matching.csv", sep = ";") %>% data.frame()
pop <- read.csv("../data/pop_1997.csv", sep = ";") %>% data.frame() %>%
    mutate(NIS.code = as.integer(NIS.code)) %>%
    filter(!is.na(NIS.code) & NIS.code != "")

dup_conv <- conv[duplicated(conv$Postal.code), ]
if(nrow(dup_conv) > 0) {
    warning("There are duplicate postal codes in the conversion table.")
    conv %>% filter(Postal.code %in% dup_conv$Postal.code) %>% data.frame() %>%
    kable(booktabs = TRUE, width = 100, caption = "Duplicate Postal Codes in Conversion Table")
}
```

Table 7: Duplicate Postal Codes in Conversion Table

Postal.code	Refnis.code	Gemeentenaam	Nom.commune
1040	21004	Brussel	Bruxelles
1040	21005	Etterbeek	Etterbeek
1050	21004	Brussel	Bruxelles
1050	21009	Elsene	Ixelles

[7]Coordinate Reference System are identifiers to explain how the projection from a 3D sphere to a 2D plane is performed. The Belgian Lambert 2008 (3812) projection is commonly used for visualizing geographical data in Belgium.

Postal.code	Refnis.code	Gemeentenaam	Nom.commune
1804	23088	Vilvoorde	Vilvorde
1804	23096	Zemst	Zemst

```
dup_refnis <- refnis[duplicated(refnis$NIS_MUN), ]
if(nrow(dup_refnis) > 0) {
    warning("There are duplicate NIS_MUN codes in the refnis table.")
    refnis %>% filter(NIS_MUN %in% dup_refnis$NIS_MUN) %>% data.frame() %>%
    kable(booktabs = TRUE, width = 100, caption = "Duplicate NIS_MUN Codes in Refnis Table")
}

dup_pop <- pop[duplicated(pop$NIS.code), ]
if(nrow(dup_pop) > 0) {
    warning("There are duplicate NIS codes in the population table.")
    pop %>% filter(NIS.code %in% dup_pop$NIS.code) %>% data.frame() %>%
    kable(booktabs = TRUE, width = 100, caption = "Duplicate NIS Codes in Population Table")
}
```

Before proceeding with the matching, we need to make sure that our matching keys are unique. We see that in the conversion table, there are duplicate postal codes, 1040 and 1050 can also be matched to Bruxelles but the other matched provide more insight (Etterbeek and Ixelles). For the postcode 1804, Vilvoorde is the corresponding municipality with some overlap with nearby Zemst which we will drop.

```
conv_matching <- conv %>% filter(!(
    (Postal.code == 1040 & Nom.commune == "Bruxelles") |
    (Postal.code == 1050 & Nom.commune == "Bruxelles") |
    (Postal.code == 1804 & Nom.commune == "Zemst")
))
if(any(duplicated(conv_matching$Postal.code))) {
    stop("There are still duplicate postal codes in the filtered conversion table.")
}

matched <- data %>% select(c("ID", "PC")) %>%
    mutate(PC = as.integer(as.character(PC))) %>%
    left_join(select(conv_matching, c("Postal.code","Refnis.code")), by = c("PC" = "Postal.code
    left_join(select(refnis, c("NIS_MUN", "MUNICIPALITY", "REGION")), by = c("Refnis.code" = "
    left_join(select(pop, c("NIS.code", "Totaal")), by = c("Refnis.code" = "NIS.code")) %>%
    rename(
        REFNIS = Refnis.code,
        MUNICIPALITY = MUNICIPALITY,
        REGION = REGION,
        POPULATION = Totaal
    )

if(any(duplicated(matched$ID))) stop("There are duplicate IDs in the matched dataset.")
```

```r
if(nrow(matched) != nrow(data)) stop("The number of matched rows does not match the original da

data_enriched <- data %>%
    left_join(select(matched, c("ID", "REFNIS", "MUNICIPALITY", "REGION", "POPULATION")), by =
data_enriched <- data_enriched %>%
    mutate(
        REFNIS = factor(REFNIS) %>% fct_infreq(),
        MUNICIPALITY = factor(MUNICIPALITY) %>% fct_infreq(),
        REGION = factor(REGION) %>% fct_infreq(),
        POPULATION = as.numeric(gsub("\\.", "", POPULATION)),
        POPULATION = ifelse(is.na(POPULATION), mean(POPULATION, na.rm = TRUE), POPULATION)
    )

if(nrow(data) != nrow(data_enriched)) stop("The number of rows in the enriched data does not ma
if(any(is.na(data_enriched$ID))) stop("There are missing IDs in the enriched data.")

summary(data_enriched %>% select(c("PC", "REFNIS", "MUNICIPALITY", "REGION", "POPULATION"))) %>
kable(booktabs = TRUE, width = 100, caption = "Summary of Enriched Data")
```

Table 8: Summary of Enriched Data

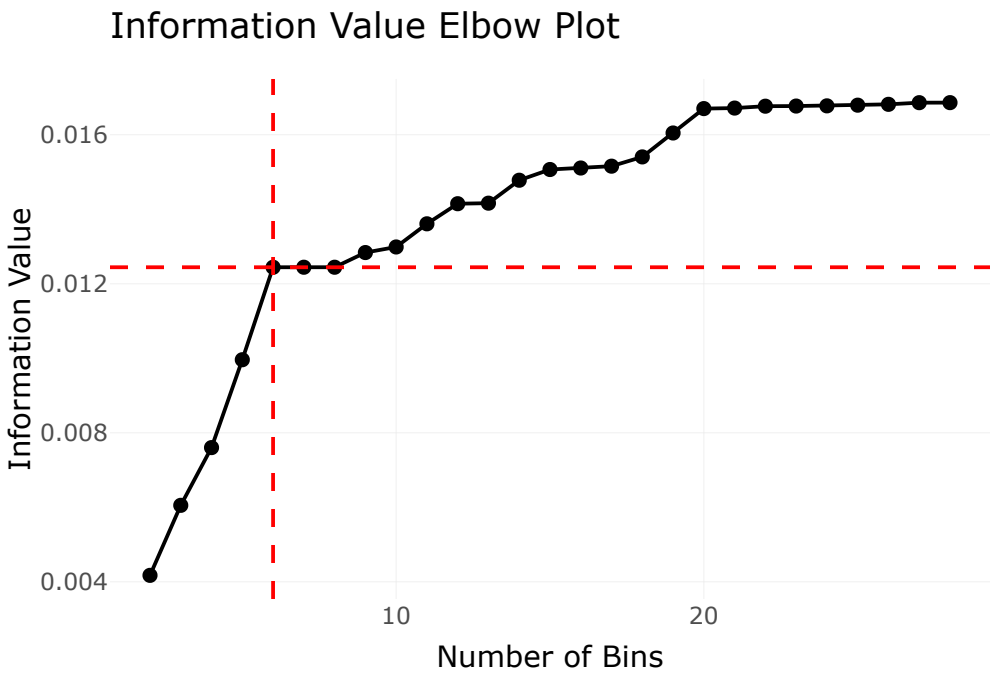
PC	REFNIS	MUNICIPALITY	REGION	POPULATION
6000 : 3992	52011 : 3992	Charleroi : 3992	RÉGION WALLONNE :62973	Min. : 995
2000 : 3716	11002 : 3837	Anvers : 3837	RÉGION FLAMANDE :59750	1st Qu.: 13222
9000 : 2770	44021 : 2770	Gand : 2770	RÉGION DE BRUXELLES-CAPITALE: 7862	Median : 24586
4000 : 2115	62063 : 2115	Liège : 2115	NA	Mean : 56488
7100 : 1806	58001 : 1806	La Louvière: 1806	NA	3rd Qu.: 56488
7000 : 1649	53053 : 1649	Mons : 1649	NA	Max. :453030
(Other):114537	(Other):114416	(Other) :114416	NA	NA

Now that we have the enriched data, we can bin the population density into categories and assess the significance of municipality, region, and population density. Note that we have 0 missing values in the population column. We'll use mean imputation to fill these missing values before binning.

```r
baseline_pop <- qbinning(data_enriched, "POPULATION", "NCLAIMS", max_bins = 30)
result_pop <- iv_elbow(data_enriched, "POPULATION", "NCLAIMS", baseline_pop)
```

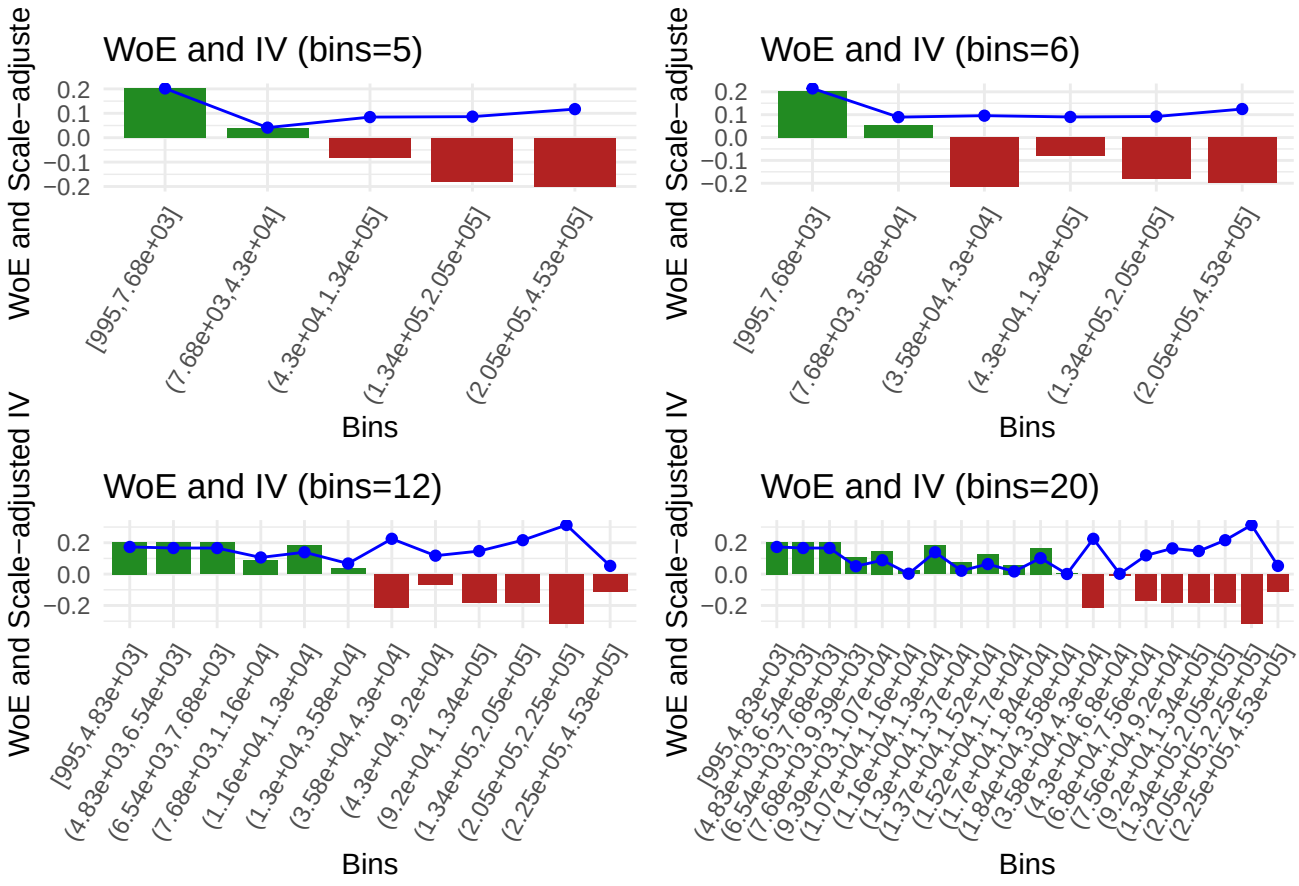
```
result_named_list[["POPULATION"]] <- result_pop
plot_iv_elbow(result_pop)
```

2.4.1.2 Population Density


Information Value Elbow Plot
Information Value
0.016
0.012
0.008
0.004
10
20
Number of Bins

Based on the elbow plot, we can look at the WoE and IV for 5, 6, 12 and 20 bins:

```
plots_pop <- list()
for (i in c(5, 6, 12, 20)) {
    temp_bins_pop <- rebuild_bins(result_pop, i)
    woe_temp_pop <- woe_calc(data_enriched, x = "POPULATION", y = "NCLAIMS", bins = temp_bins_p
    plots_pop[[i]] <- plot_woe_iv(woe_temp_pop, interactive = FALSE)
}
(plots_pop[[5]] | plots_pop[[6]]) / (plots_pop[[12]] | plots_pop[[20]])
```



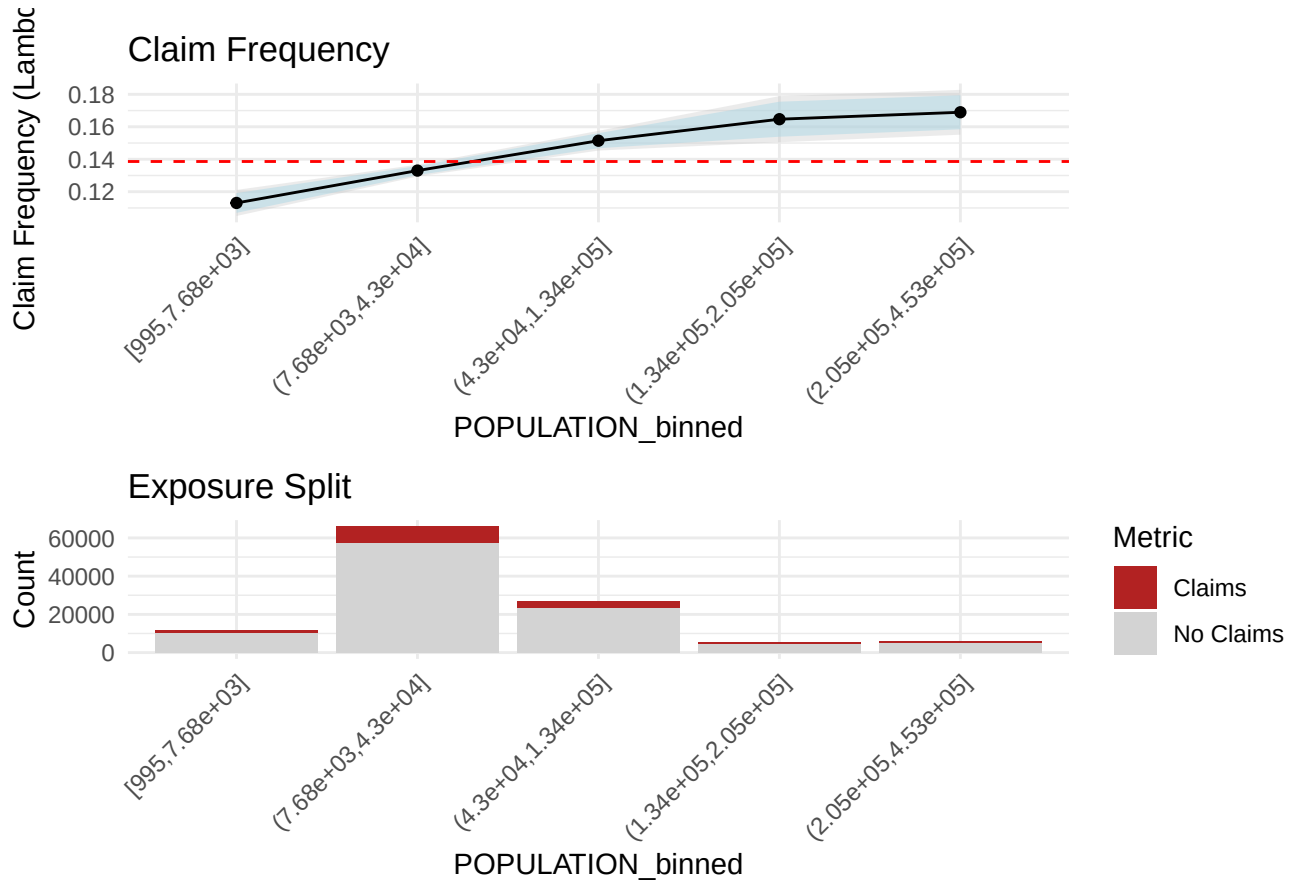
We see a linear diminution of the WoE as the population increases, indicating that higher population areas tend to have higher claim frequencies as we expected. The linear trend seems sufficiently captured with 5 bins. We'll create the new binned variable with 5 levels.

```
data_enriched$POPULATION_binned <- cut(
    data_enriched$POPULATION,
    breaks = rebuild_bins(result_pop, 5),
    include.lowest = TRUE,
    ordered_result = TRUE
)
```

```r
n_bins_named_list[["POPULATION"]] <- 5
m0 <- glm(NCLAIMS ~ offset(log(EXP)), data = data_enriched, family = poisson(link = "log"))
m1 <- glm(NCLAIMS ~ POPULATION_binned + offset(log(EXP)), data = data_enriched, family = poiss
chisq_result <- anova(m0, m1, test = "Chisq")
cat(sprintf("Is the effect of POPULATION_binned significant at 95%% confidence? %s", ifelse(ch
```

```
Is the effect of POPULATION_binned significant at 95% confidence? Yes
```

```r
plot_claim_freq(data_enriched, "POPULATION_binned")
```



The variable is significant at 95% confidence, however we see that the 3 largest population bins have very similar claim frequencies, we can try merging them to get have a simple three-level binning (Low, Medium, High Density).

```r
initial_bins_merged <- data_enriched$POPULATION_binned

three_bins_merged <- fct_collapse(
    data_enriched$POPULATION_binned,
    "Low Density" = c("[995,7.68e+03]"),
```

```
    "Medium Density" = c("(7.68e+03,4.3e+04]"),
    "High Density" = c("(4.3e+04,1.34e+05]", "(1.34e+05,2.05e+05]", "(2.05e+05,4.53e+05]")
)

m1 <- glm(NCLAIMS ~ offset(log(EXP)) + initial_bins_merged, data = data_enriched, family = pois
m0 <- glm(NCLAIMS ~ offset(log(EXP)) + three_bins_merged, data = data_enriched, family = poisso
test_pop <- anova(m0, m1, test = "Chisq")
cat(sprintf("Are we loosing significant information by merging the population bins? %s (p-value
```

```
Are we loosing significant information by merging the population bins? Yes (p-value: 0.0020)
```

```
four_bins_merged <- fct_collapse(
    data_enriched$POPULATION_binned,
    "(1.34e+05,4.53e+05]" = c("(1.34e+05,2.05e+05]", "(2.05e+05,4.53e+05]")
)

m0 <- glm(NCLAIMS ~ offset(log(EXP)) + four_bins_merged, data = data_enriched, family = poisson
test_pop_four <- anova(m0, m1, test = "Chisq")
cat(sprintf("Are we loosing significant information by merging the last two population bins? %s
```

```
Are we loosing significant information by merging the last two population bins? No (p-value: 0
```

Based on the result of the anova test, it seems that merging the bins into three levels does not significantly increase the residual deviance. We'll therefore adopt the three-level binning for the population variable which is a lot easier to interpret and work with.

```
data_enriched$POPULATION_binned <- four_bins_merged
plot_claim_freq(data_enriched, "POPULATION_binned")
```



We get a linear trend in claim frequency with respect to population density bins (ordinal variable).

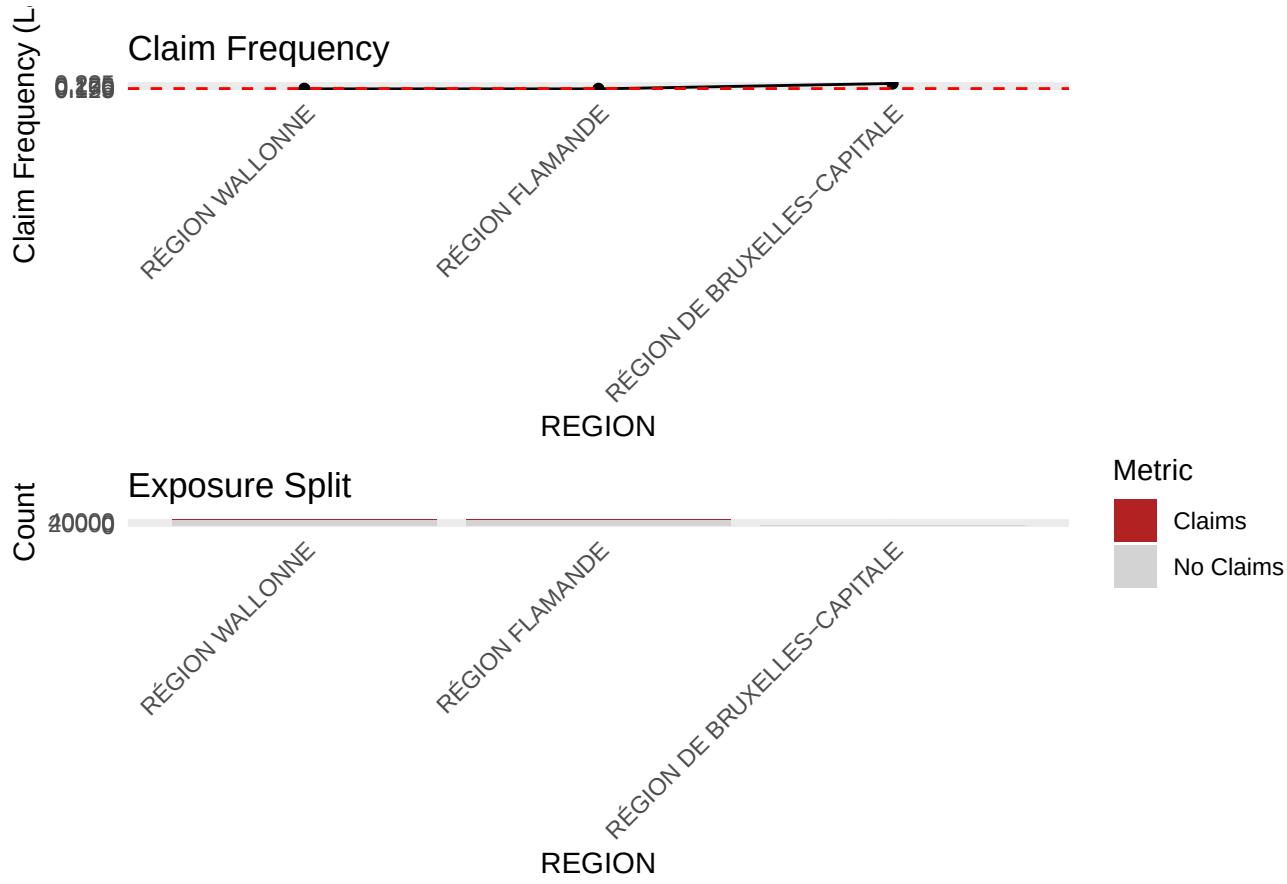
2.4.1.3 Regions

We will check the significance and claim frequency across regions

```
m0 <- glm(NCLAIMS ~ offset(log(EXP)), data = data_enriched, family = poisson(link = "log"))
m1 <- glm(NCLAIMS ~ offset(log(EXP)) + REGION, data = data_enriched, family = poisson(link = "
    test_result <- anova(m0, m1, test = "Chisq")
    cat(sprintf("Is variable %s significant? %s\n", "REGION", ifelse(test_result$`Pr(>Chi)`[2]
```

```
Is variable REGION significant? Yes
```

```
plot_claim_freq(data_enriched, "REGION")
```

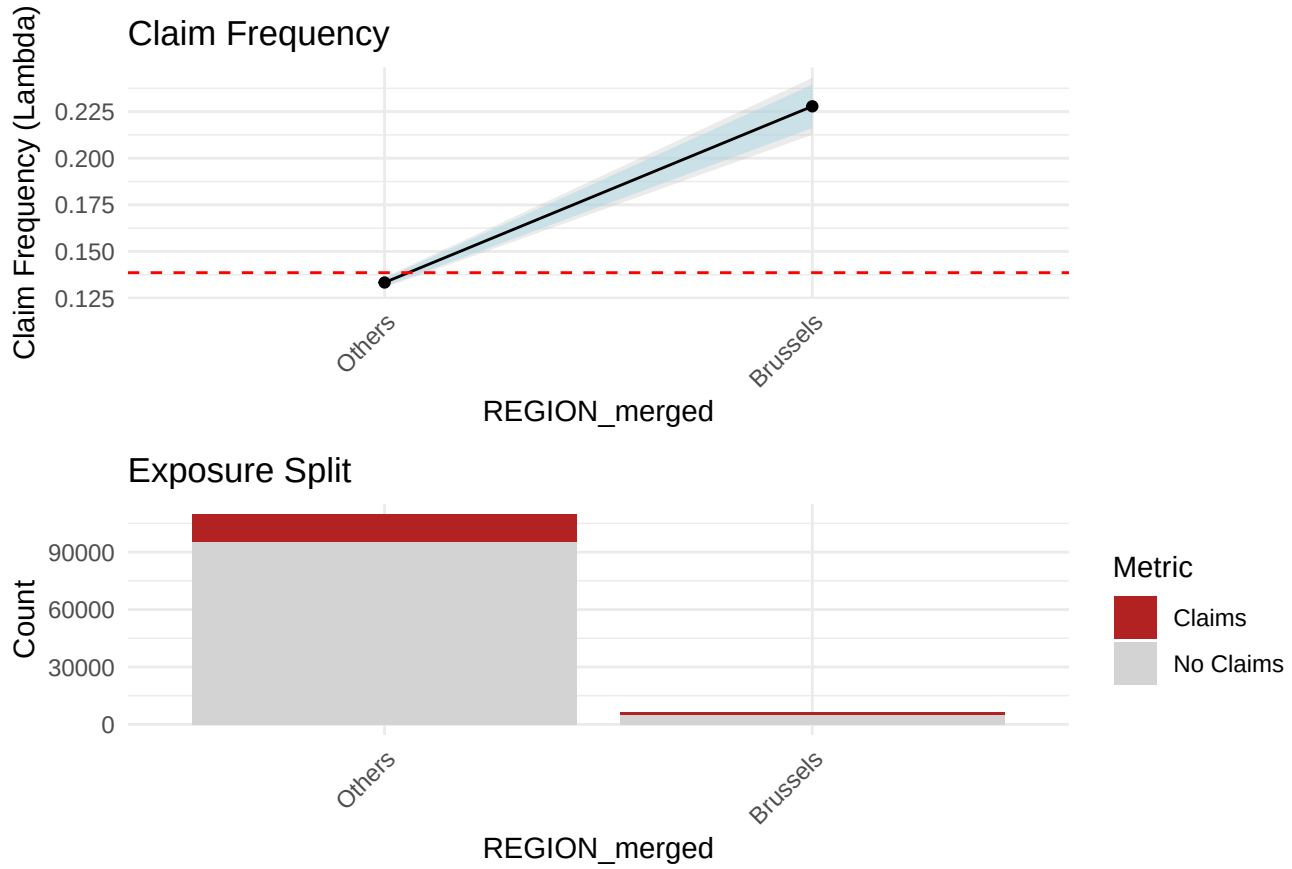


It is clear that the Brussels region has a distinct claim frequency pattern compared to the other regions. We'll test if it makes sense to keep the Walloon and Flemish regions separate as well or not.

```
merged_regions <- fct_collapse(
    data_enriched$REGION,
    "Brussels" = c("RÉGION DE BRUXELLES-CAPITALE"),
    "Others" = c("RÉGION WALLONNE", "RÉGION FLAMANDE"),
)
data_enriched$REGION_merged <- merged_regions
m0 <- glm(NCLAIMS ~ offset(log(EXP)) + REGION, data = data_enriched, family = poisson(link = "
m1 <- glm(NCLAIMS ~ offset(log(EXP)) + REGION_merged, data = data_enriched, family = poisson(li
test_result <- anova(m0, m1, test = "Chisq")
cat(sprintf("Was it justified to merge the REGION bins? %s", ifelse(test_result$`Pr(>Chi)`[2] <
```

```
Was it justified to merge the REGION bins? Yes
```

```
plot_claim_freq(data_enriched, "REGION_merged")
```



We see that despite a much lower exposure in Brussels, the claim frequency is significantly higher from the other regions. This confirms our hypothesis that urban areas, such as Brussels, tend to have a higher claim frequency compared to less densely populated regions. We can quickly check our second hypothesis that despite higher frequencies, the severity is lower in urban areas like Brussels.

```
urb_rur_comp <- data_enriched %>%
    group_by(REGION_merged) %>%
    summarise(
        total_claims = sum(NCLAIMS),
        total_exposure = sum(EXP),
        total_claims = sum(NCLAIMS),
        total_claim_amount = sum(AMOUNT),
        frequency = total_claims / total_exposure,
        severity = total_claim_amount / total_claims,
        expected_claims = frequency * severity
    ) %>%
    arrange(desc(frequency))
urb_rur_comp %>%
    kable(
        caption = "Expected claims by merged region\nSplit by frequency and severity",
```

```
        align = "l", booktabs = TRUE
    )
```

Table 9: Expected claims by merged region Split by frequency and severity

REGION_merged	total_claims	total_exposure	total_claim_amount	frequency	severity	expected_claims
Brussels	1477	6482.844	1776783	0.2278321	1202.968	274.0746
Others	14632	109723.584	25498069	0.1333533	1742.624	232.3846

With this quick descriptive analysis, we can confirm that in our sample:

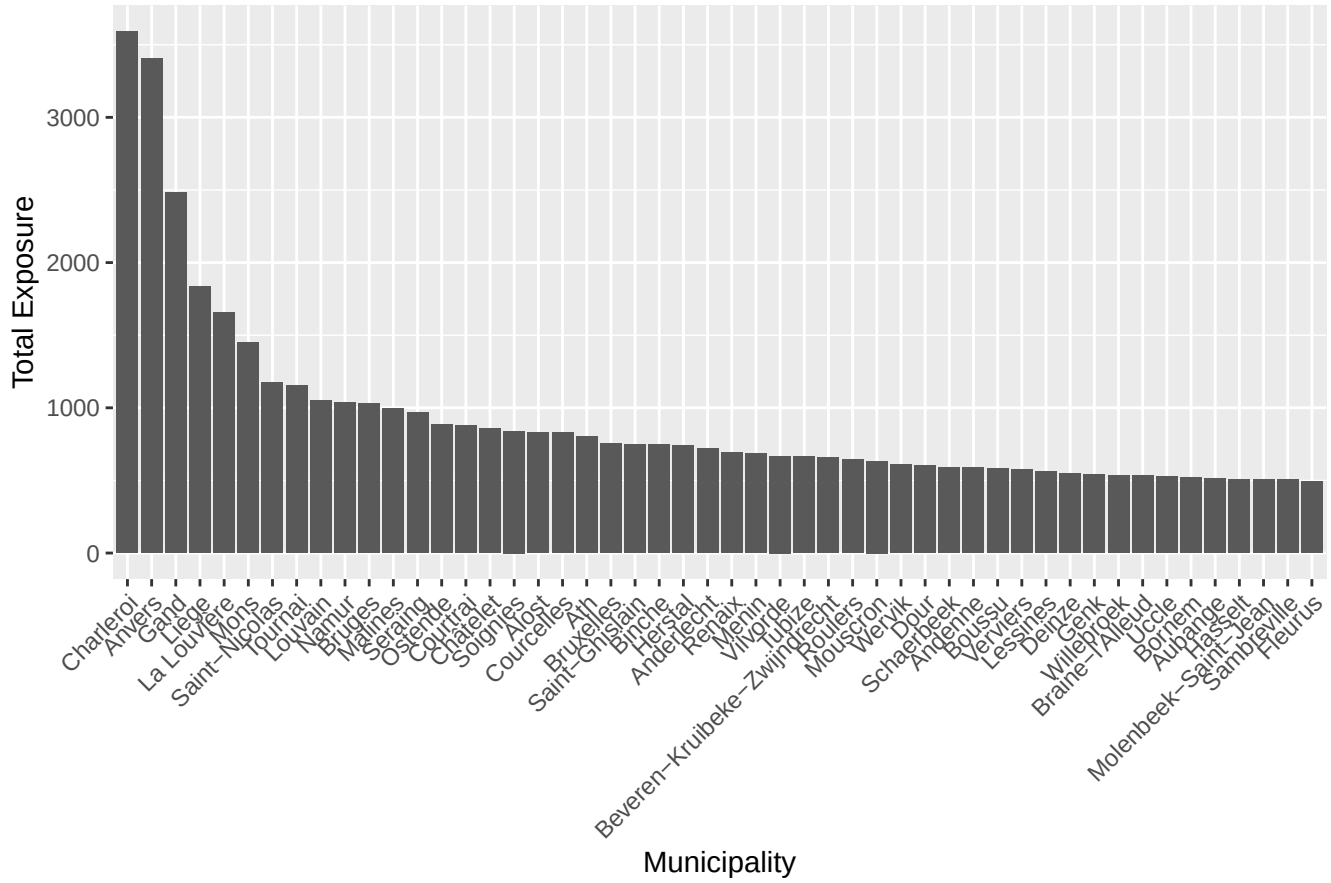
- The claim frequency is higher in urban areas like Brussels compared to other regions (factor 1.7084852)
- The claim severity is lower in urban areas like Brussels compared to other regions (factor 0.6903198)
- The frequency effect outweighs the severity effect in determining the expected claims between urban and other regions (factor 1.1794012)

We expect to see similar effects for other granularities, such as municipalities, where urban areas may exhibit higher claim frequencies but lower severities compared to less densely populated areas.

2.4.1.4 Municipalities

There are far too many municipalities to use it as a simple categorical variable in our models. We can thus focus on the n largest municipalities by exposure and group the remaining ones into an "Other" category.

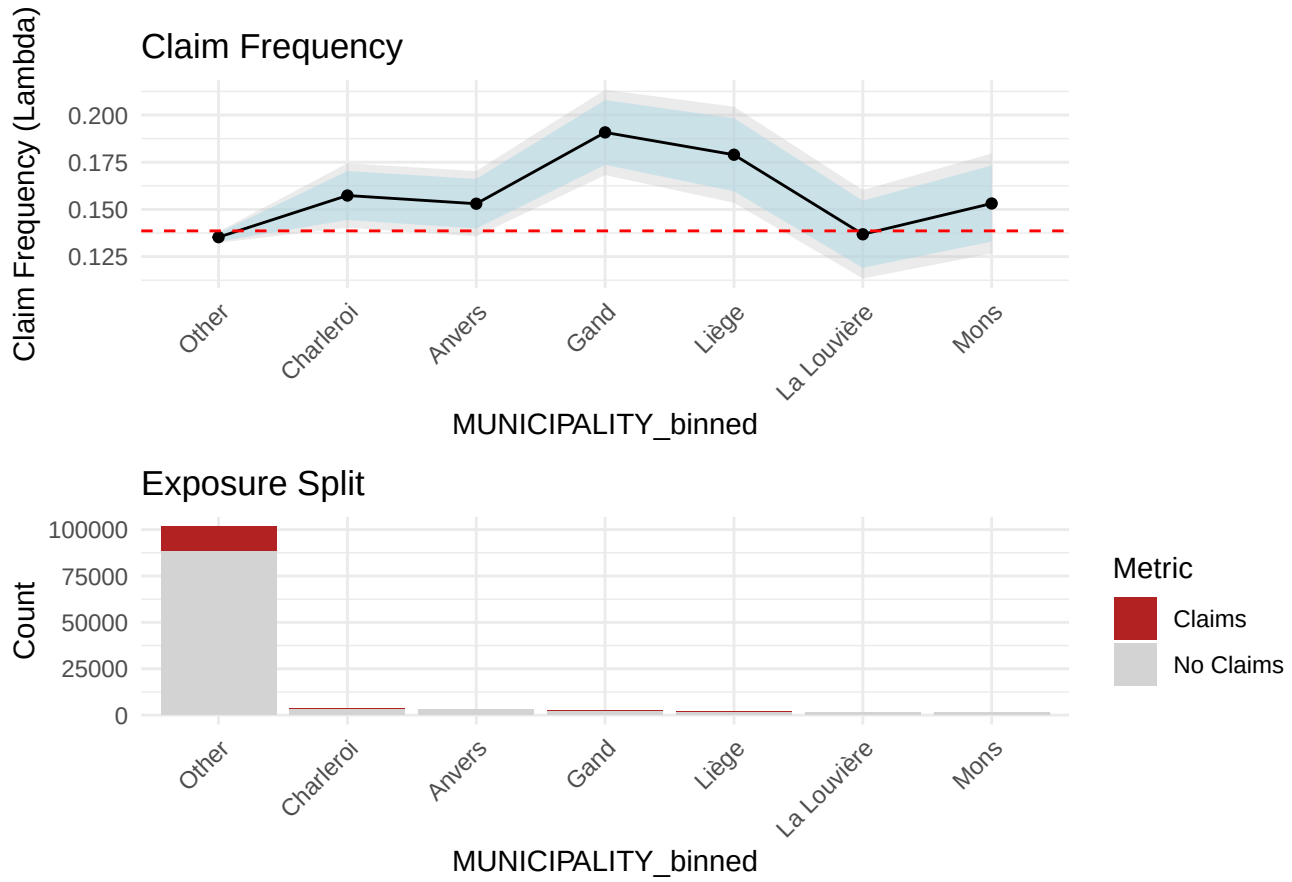
```
data_enriched %>%
    group_by(MUNICIPALITY) %>%
    summarise(
        total_exposure = sum(EXP)
    ) %>%
    arrange(desc(total_exposure)) %>% head(n=50) %>%
    ggplot(aes(x = reorder(MUNICIPALITY, -total_exposure), y = total_exposure)) +
    geom_bar(stat = "identity") +
    labs(x = "Municipality", y = "Total Exposure") +
    theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



We can focus on the 6 largest municipalities by exposure before the exposure distribution flattens out.

```
exp_breakpoint <- data_enriched %>%
    group_by(MUNICIPALITY) %>%
    summarise(
        total_exposure = sum(EXP)
    ) %>%
    arrange(desc(total_exposure)) %>%
    head(n=6)
data_enriched$MUNICIPALITY_binned <- ifelse(data_enriched$MUNICIPALITY %in% exp_breakpoint$MUNI
data_enriched$MUNICIPALITY_binned <- factor(data_enriched$MUNICIPALITY_binned) %>% fct_infreq()

plot_claim_freq(data_enriched, "MUNICIPALITY_binned")
```




```
m0 <- glm(NCLAIMS ~ offset(log(EXP)), data = data_enriched, family = poisson(link = "log"))
m1 <- glm(NCLAIMS ~ MUNICIPALITY_binned + offset(log(EXP)), data = data_enriched, family = pois
results <- anova(m0, m1, test = "Chisq")
cat(sprintf("Is variable %s significant? %s\n", "MUNICIPALITY_binned", ifelse(results$`Pr(>Chi
```

```
Is variable MUNICIPALITY_binned significant? Yes
```

We now have a binned version of the MUNICIPALITY variable which is significant and usable in our model. Of course, the exposure is limited even in the bigger municipalities and that causes some variability in the frequency estimates. However, for the sake of interpretation, we'll keep the cities separated even though Charleroi and Antwerp are similar in terms of frequency and so are Gand and Liège. We seem to capture a similar phenomenon with cities having a higher claim frequency compared to smaller municipalities even if the effect is less clear than for the regions.

```
data_enriched %>%
    group_by(MUNICIPALITY_binned) %>%
    summarise(
        total_claims = sum(NCLAIMS),
        total_exposure = sum(EXP),
        total_claim_amount = sum(AMOUNT),
```

```
        frequency = total_claims / total_exposure,
        severity = total_claim_amount / total_claims,
        expected_claims = frequency * severity
    ) %>%
    arrange(desc(frequency)) %>%
    kable(
        caption = "Expected claims by municipality\nSplit by frequency and severity",,
        align = "l", booktabs = TRUE
    )
```

Table 10: Expected claims by municipality Split by frequency and severity

MUNICIPALITY_binned	total_claims	total_exposure	total_claim_amount	frequency	severity	expected_claims
Gand	474	2484.419	756248.0	0.1907891	1595.4599	304.3963
Liège	329	1838.449	311331.2	0.1789552	946.2954	169.3445
Charleroi	565	3590.329	2681716.8	0.1573672	4746.4015	746.9279
Mons	222	1449.710	266246.7	0.1531341	1199.3094	183.6552
Anvers	521	3405.019	637351.8	0.1530094	1223.3240	187.1801
La Louvière	227	1658.945	319355.9	0.1368339	1406.8543	192.5054
Other	13771	101779.556	22302602.0	0.1353022	1619.5340	219.1265

Even if some cities follow the high frequency, low severity pattern, we do find outliers such as Charleroi where the severity is unusually high or Liège where it falls unusually low. We then seem to capture something more than the simple urban-rural divide which justifies keeping the `MUNICIPALITY_binned` variable in our model.

2.4.2 Polynomial Features

For the five numerical variables, we will consider the polynomial features up to the third degree to capture potential non-linear relationships with the response variable. This is an intermediate step towards the GAM models where we will explicitly model non-linear effects. Note that: - We will use Pearson correlation (continuous variables) instead of Spearman correlation because the rank-rank approach won't help us differentiate between the polynomial features of the same positive variable (ranks are maintained). - We will look at the p-values of the correlation tests instead of the correlation coefficients themselves to assess the significance of the relationships. - For the correlation with average claim amount (`AVG`), we will only consider the claims where `NCLAIMS` is greater than zero. - Orthogonal polynomial features will be considered separately to avoid multicollinearity issues inherent in raw polynomial features. - To compute orthogonal polynomial features, we will filter out the NA values from the data.

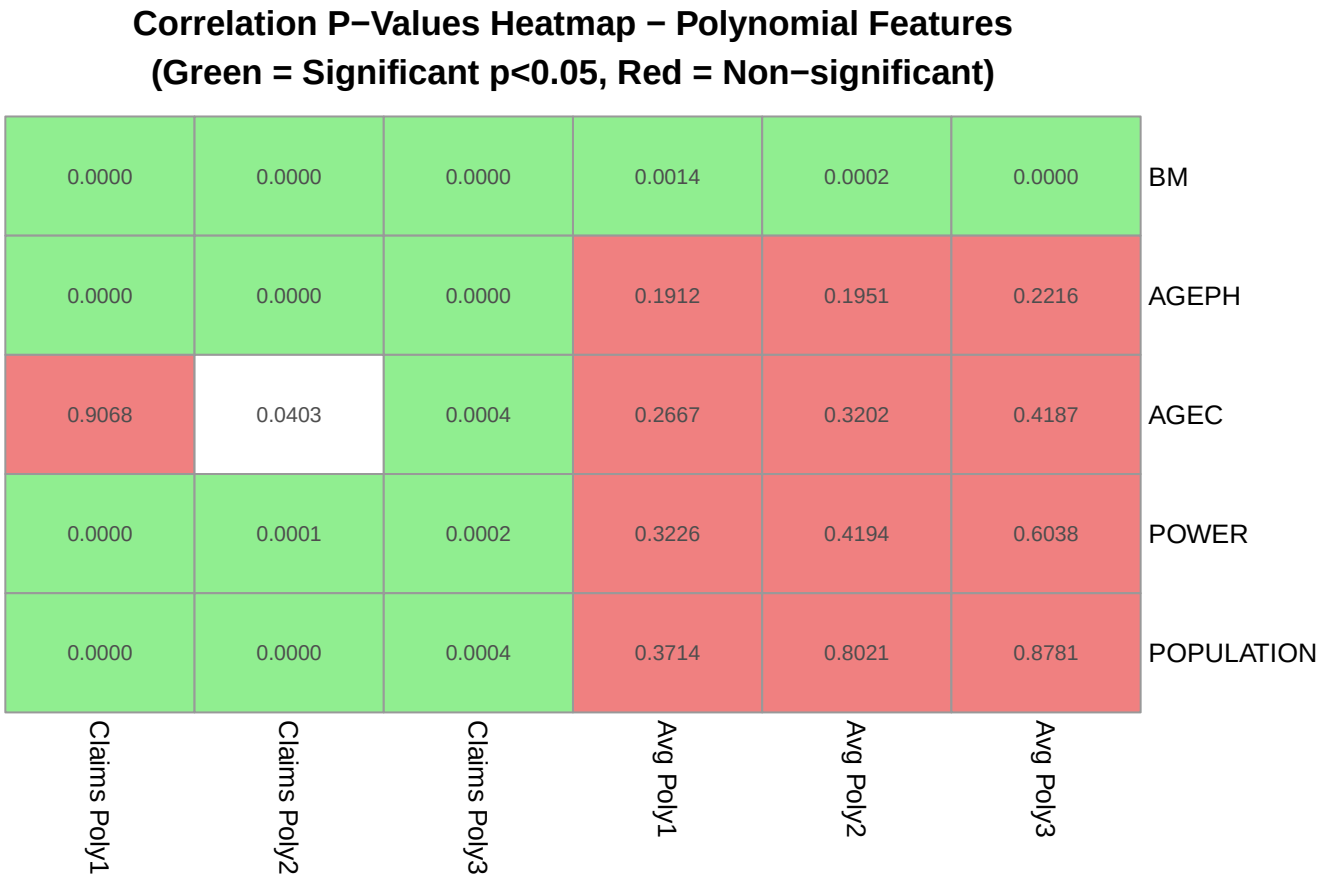
```
numerical_continuous <- c("BM", "AGEPH", "AGEC", "POWER", "POPULATION")

polynomial_features <- lapply(numerical_continuous, function(var) {
    poly(data_enriched[[var]], degree = 3, raw = TRUE)
})
```

```r
names(polynomial_features) <- numerical_continuous
# Initialize data frame with proper structure
correlation_matrix <- data.frame(
    matrix(NA, nrow = length(numerical_continuous), ncol = 6)
)
rownames(correlation_matrix) <- numerical_continuous
colnames(correlation_matrix) <- c(
    "Claims Poly1", "Claims Poly2", "Claims Poly3",
    "Avg Poly1", "Avg Poly2", "Avg Poly3")

# Fill the matrix
for (var in numerical_continuous) {
    poly_values <- polynomial_features[[var]]
    for (j in 1:3) {
        correlation_matrix[var, j] <- cor.test(data_enriched[["NCLAIMS"]], poly_values[, j], me
        claims_only_avg <- data_enriched[["AVG"]][data_enriched[["NCLAIMS"]] > 0]
        claims_only_poly_values <- poly_values[data_enriched[["NCLAIMS"]] > 0, j]
        correlation_matrix[var, j + 3] <- cor.test(claims_only_avg, claims_only_poly_values, me
    }
}

pheatmap(correlation_matrix,
    main = "Correlation P-Values Heatmap - Polynomial Features\n(Green = Significant p<0.05, Re
    display_numbers = TRUE,
    number_format = "%.4f",
    breaks = c(0, 0.01, 0.05, 1),
    color = c("lightgreen", "white", "lightcoral"),
    legend = FALSE,
    cluster_rows = FALSE,
    cluster_cols = FALSE
)
```



We first notice that `AGEC`is not significantly correlated with `NCLAIMS` when considered with the first and second-degree polynomial features. As we saw before, the non-linear effect is captures in our binning. An alternative approach could have been to use the third-degree polynomial. We would still have the issue of fixed effects being imposed by the polynomial terms, which might not fully capture the underlying non-linear relationship. This is a good example of why binning can sometimes be more flexible in capturing non-linear relationships compared to fixed polynomial terms.

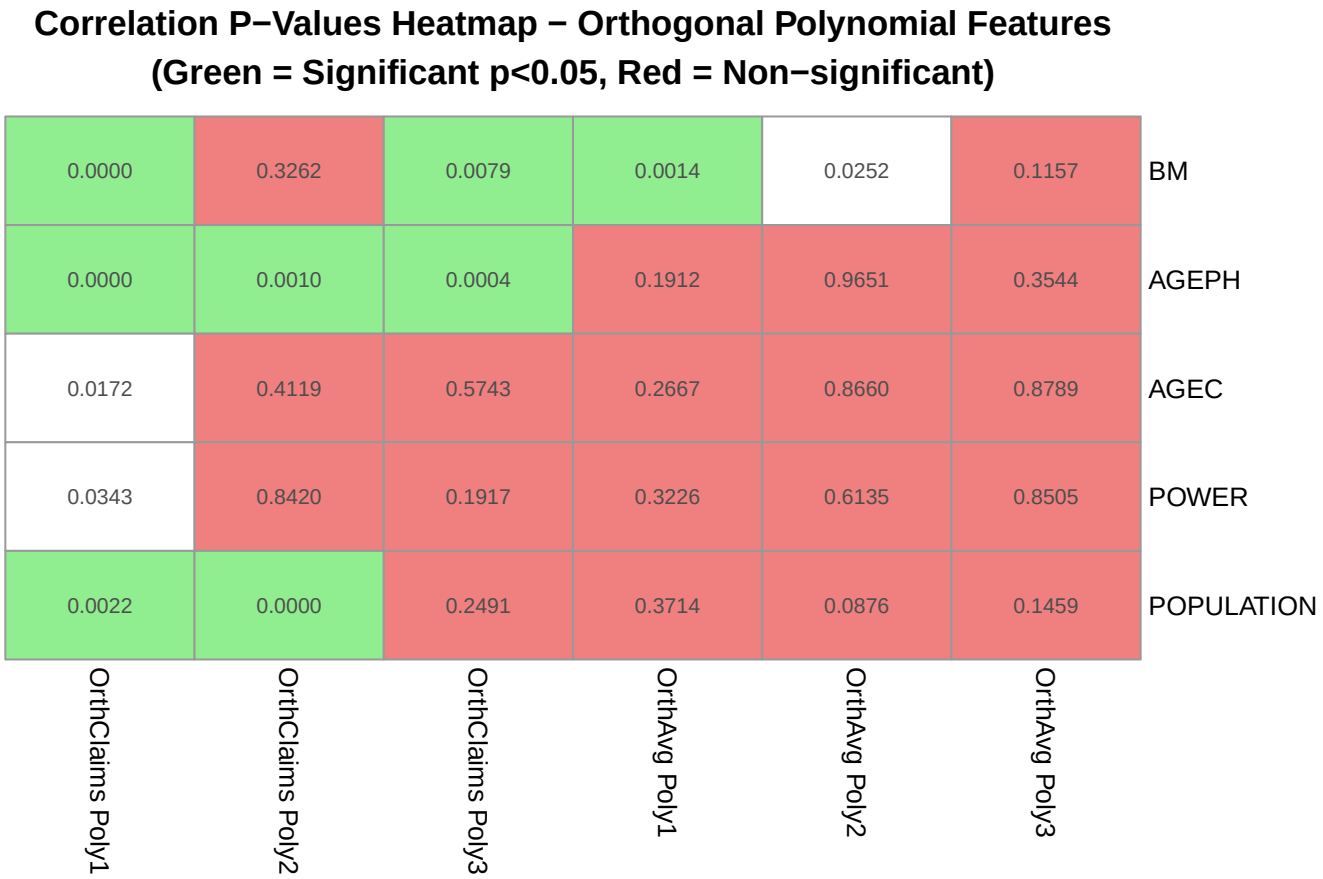
We notice that the correllations between the polynomial features of a same variable are similar, this is foreseeable given that higher-degree polynomials are derived from the lower-degree ones. We are here exposed to multicollinearity issues. A solution is to check the **orthogonal polynomial features** instead of the raw ones, as they are uncorrelated by construction. This also enables us to differentiate the individual contributions of each polynomial term more clearly in the modeling process (linear, quadratic, cubic).

```
no_na <- na.omit(data_enriched)
orthogonal_polynomial_features <- lapply(numerical_continuous, function(var) {
    poly(no_na[[var]], degree = 3, raw = FALSE)
})
names(orthogonal_polynomial_features) <- numerical_continuous
orth_correlation_matrix <- data.frame(
    matrix(NA, nrow = length(numerical_continuous), ncol = 6)
```

```
)
rownames(orth_correlation_matrix) <- numerical_continuous
colnames(orth_correlation_matrix) <- c(
    "OrthClaims Poly1", "OrthClaims Poly2", "OrthClaims Poly3",
    "OrthAvg Poly1", "OrthAvg Poly2", "OrthAvg Poly3")

# Fill the matrix
for (var in numerical_continuous) {
    poly_values <- orthogonal_polynomial_features[[var]]
    for (j in 1:3) {
        orth_correlation_matrix[var, j] <- cor.test(no_na[["NCLAIMS"]], poly_values[, j], metho
        claims_only_avg <- no_na[["AVG"]][no_na[["NCLAIMS"]] > 0]
        claims_only_poly_values <- poly_values[no_na[["NCLAIMS"]] > 0, j]
        orth_correlation_matrix[var, j + 3] <- cor.test(claims_only_avg, claims_only_poly_value
    }
}

pheatmap(orth_correlation_matrix,
    main = "Correlation P-Values Heatmap - Orthogonal Polynomial Features\n(Green = Significant
    display_numbers = TRUE,
    number_format = "%.4f",
    breaks = c(0, 0.01, 0.05, 1),
    color = c("lightgreen", "white", "lightcoral"),
    legend = FALSE,
    cluster_rows = FALSE,
    cluster_cols = FALSE
)
```



From this orthogonal polynomial feature exploration we can conclude that:

- Polynomial features are more significantly correlated with the number of claims (`NCLAIMS`) than with the average claim amount (`AVG`).
- Orthogonal polynomial features help assess whether each polynomial term contributes significantly to the relationship with the target variables, without being confounded by multicollinearity.
- There is no clear rule to know if first, second, and/or third-degree polynomial terms are significant. The WoE-based binning approach provides a more flexible and interpretable way to capture non-linear effects.

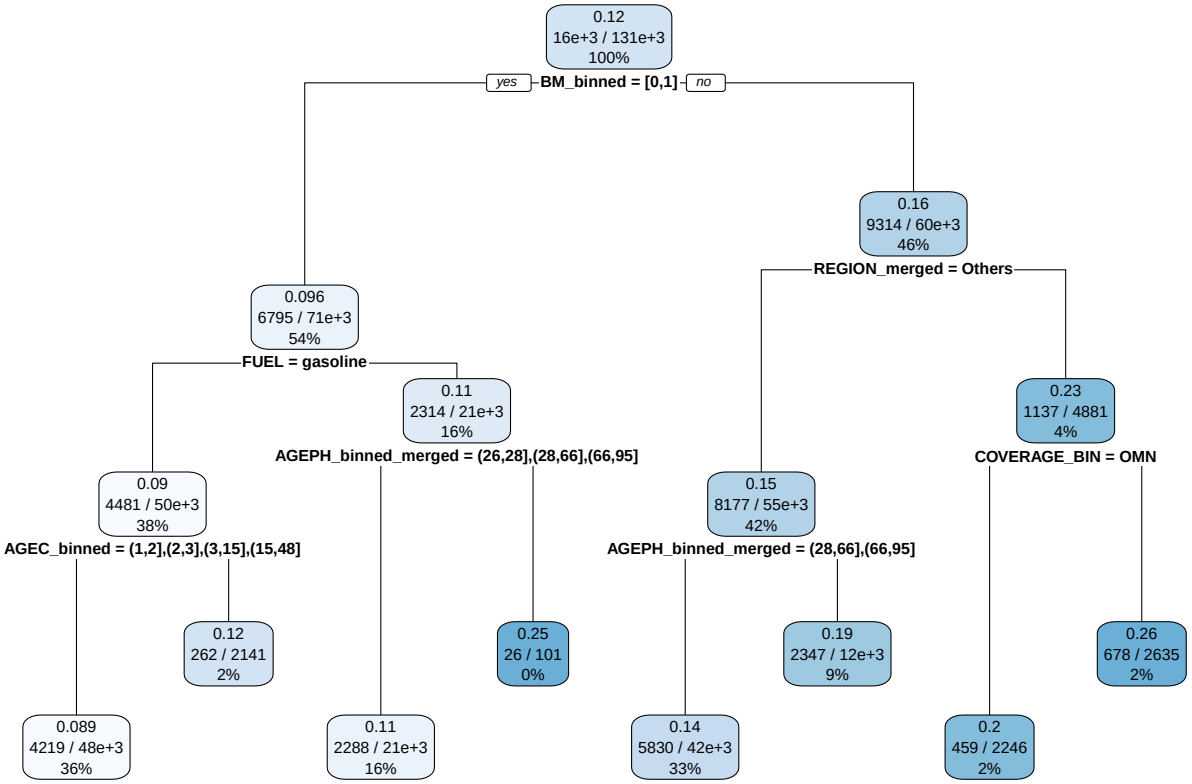
In conclusion, polynomials are a way to capture non-linear relationships. However, the incremental effect of these polynomial terms is still fixed in a GLM. We therefore prefer working with binned features rather than relying solely on polynomial terms. We will nevertheless see if the smoothing functions of the GAM models confirm the conclusions drawn from our exploratory analysis.

2.4.3 Interaction Features

Interactions are usually added as a result of business knowledge. In our case, we know from a theorical background that the age of the policyholder interacts with the power of the car. We can

add this interaction effect and test its significance in our models. Instead of manually testing all possible combinations. We'll use a decision tree approach which nature is to look for interactions and splits that best explain the variability in the target variable. With a shallow decision tree (depth of 3), we can identify the most relevant interactions without overfitting the data. Watch out the decision trees are not robusts and can be sensitive to small changes in the data. We'll do a 10-fold cross-validation to mitigate this issue.

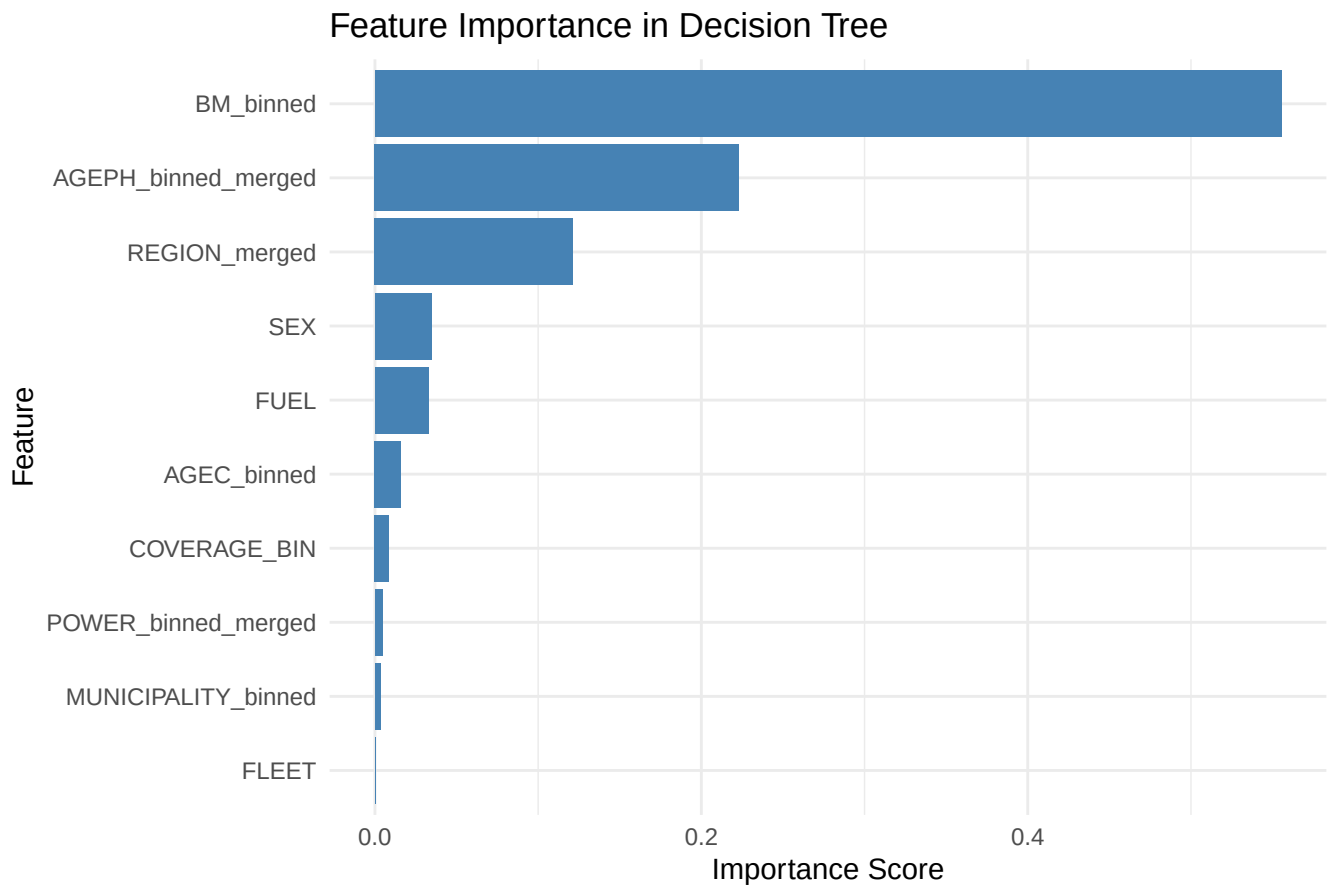
```
tree_model <- rpart(
    NCLAIMS ~ COVERAGE_BIN + FUEL + FLEET + SEX + # Categorical features
    BM_binned + AGEPH_binned_merged + AGEC_binned + POWER_binned_merged + # Binned numerical f
    POPULATION_binned + REGION_merged + MUNICIPALITY_binned, # Enriched features
    data = data_enriched,
    method = "poisson",
    control = rpart.control(maxdepth = 3, xval = 10, cp = 0)
)
rpart.plot(tree_model)
```




```
importance_df <- data.frame(
    feature = names(tree_model$variable.importance),
    importance = tree_model$variable.importance %>% `/`(sum(.))
```

```r
)
importance_df <- importance_df %>% arrange(desc(importance))

ggplot(importance_df, aes(x = reorder(feature, importance), y = importance)) +
    geom_bar(stat = "identity", fill = "steelblue") +
    coord_flip() +
    labs(title = "Feature Importance in Decision Tree",
            x = "Feature",
            y = "Importance Score") +
    theme_minimal()
```



Based on this machine learning approach, we can identify both specific interactions and the most important features that influence the target variable. When adding interaction terms to our models, we can thus consider:

Specific Interactions: representing the two first splits identified by the decision tree.

- `BM_binned.[0,1]` x `FUEL.gasoline`
- `BM_binned.[0,1]` x `!FUEL.gasoline`
- `!BM_binned.[0,1]` x `REGION_merged.Others`: will not add it as it is captured below already

- `!BM_binned.[0,1]` x `!REGION_merged.Others`: will not add it as it is captured below already

```
data_enriched <- data_enriched %>%
    mutate(
        BM_01_FUEL_gasoline = BM_binned == "[0,1]" & FUEL == "gasoline",
        BM_01_FUEL_not_gasoline = BM_binned == "[0,1]" & FUEL != "gasoline"
    )
```

Variable Interactions: representing the 3 most important features identified by the decision tree.

- `BM_binned` x `AGEPH_binned_merged`
- `REGION_merged` x `POWER_binned_merged`
- `BM_binned` x `REGION_merged`

3 Modelling

Now that we have explored, transformed, and enriched our data. We are ready to proceed with the modelling phase. We have a rich set of features to gradually incorporate into our models to improve predictive performance. We will also clean all intermediary objects that won't be used for modelling.

```
# Clean intermediary objects not used for modelling
functions <- ls()[sapply(ls(), function(x) is.function(get(x)))]
rm(list = setdiff(ls(), c(
    "data", "data_enriched", "holdout", "mtpl_be",
    "conv_matching", "refnis", "pop",
    "result_named_list", "n_bins_named_list",
    functions)))
cat(sprintf("Remaining objects and functions: %s", paste(ls(), collapse = ", ")))
```

```
Remaining objects and functions: assert_ref_level, conv_matching, data, data_enriched, holdout
```

```
original_categorical_features <- c("COVERAGE_BIN", "FUEL", "FLEET", "SEX")
continuous_features <- c("BM", "AGEPH", "AGEC", "POWER")
binned_continuous_features <- c("BM_binned", "AGEPH_binned_merged", "AGEC_binned", "POWER_binne
enriched_features <- c("REGION_merged", "MUNICIPALITY_binned", "POPULATION")
binned_enriched_features <- c("REGION_merged", "MUNICIPALITY_binned", "POPULATION_binned")
interaction_features <- c("BM_01_FUEL_gasoline", "BM_01_FUEL_not_gasoline", "BM_binned:AGEPH_bi

levels_named_list <- list()
for (col in c(original_categorical_features, binned_continuous_features, binned_enriched_featu
    if (!col %in% colnames(data_enriched)) {stop(sprintf("Column %s is missing from data_enricl
    levels_named_list[[col]] <- levels(data_enriched[[col]])
}
```

Original Categorical Features: Note that `USE` was excluded because it was not statistically significant in preliminary analyses. And `COVERAGE` was transformed into `COVERAGE_BIN` after merging the partial and full omnium levels that did not significantly add information.

- `COVERAGE_BIN`
- `FUEL`
- `FLEET`
- `SEX`

Continuous Features: The continuous features as such will be included in the GAM models only. We binned and merged them into categorical variables for the GLM modelling.

- `BM` -> `BM_binned`
- `AGEPH` -> `AGEPH_binned_merged`
- `AGEC` -> `AGEC_binned`
- `POWER` -> `POWER_binned_merged`

Enriched Features: Based on geospation information, we have created, binned and merged both categorical and continuous features to capture urban/rural distinctions and municipality-level characteristics.

- `REGION_merged`
- `MUNICIPALITY_merged`
- `POPULATION` -> `POPULATION_binned`

Interaction Features: We have also created specific and variable interaction features based on expert knowledge and the decision tree analysis.

- `POWER_binned_merged` x `AGEPH_binned_merged`: expert knowledge
- `BM_01_FUEL_gasoline`: specific interaction based on decision tree analysis
- `BM_01_FUEL_not_gasoline`: specific interaction based on decision tree analysis
- `BM_binned` x `AGEPH_binned_merged`: variable interaction based on feature importance
- `REGION_merged` x `AGEPH_binned_merged`: variable interaction based on feature importance
- `BM_binned` x `REGION_merged`: variable interaction based on feature importance

Polynomial Features: We also considered raw and orthogonal polynomial features for the continuous variables to capture potential non-linear relationships in the GAM models. We decided not to include them in the GLM models for the sake of model simplicity and interpretability. These polynomial effects will be naturally captured in the GAM models through the smooth functions. We will then be able to see whether the GAM results confirm our exploratory analysis.

3.1 Validation Framework

Remember that at the start of the Data Preparation chapter, we kept a holdout set separate from the training data for model validation purposes. We will now create a workflow including:

- **Data Preparation**: enriching the dataset and transforming features as described in the previous sections.
- **Model Training**: fitting the selected models to the prepared training data with a 5-fold cross-validation.
- **Model Validation**: evaluating the performance of the trained models on the holdout set.

3.1.1 Data Preparation Workflow

```
# Data enrichment

data_enrichment <- function(input_data) {
    if (!("PC" %in% colnames(input_data))) {stop("PC column is missing from raw_data")}
    if (!("ID" %in% colnames(input_data))) {stop("ID column is missing from raw_data")}
    if (!exists("conv_matching")) {stop("conv_matching object is missing")}
    if (!exists("refnis")) {stop("refnis object is missing")}
    if (!exists("pop")) {stop("pop object is missing")}
    output_data <- input_data %>%
        mutate(TEMP_PC = as.integer(PC)) %>%
        left_join(select(conv_matching, c("Postal.code","Refnis.code")), by = c("TEMP_PC" = "Po
        left_join(select(refnis, c("NIS_MUN", "MUNICIPALITY", "REGION")), by = c("Refnis.code"
        left_join(select(pop, c("NIS.code", "Totaal")), by = c("Refnis.code" = "NIS.code")) %>%
        rename(
            REFNIS = Refnis.code,
            MUNICIPALITY = MUNICIPALITY,
            REGION = REGION,
            POPULATION = Totaal
        ) %>%
        select(-TEMP_PC) %>%
        mutate(
            POPULATION = as.numeric(gsub("\\.", "", POPULATION)),
        ) %>%
        mutate(
            POPULATION = ifelse(is.na(POPULATION), mean(POPULATION, na.rm = TRUE), POPULATION)
        )
    if (any(duplicated(output_data$ID))) {stop("Duplicated IDs found in enriched_data")}
    if (nrow(output_data) != nrow(input_data)) {stop("Row count mismatch between enriched_data
    return(output_data)
}

# Step 2: Data Preparation

data_preparation_original_categorical <- function(input_data) {
    # Convert original categorical features to factors and validate their levels
    if (!exists("original_categorical_features")) {stop("original_categorical_features object
    if (!exists("levels_named_list")) {stop("levels_named_list object is missing")}
    temp_data <- input_data
```

```
    for (col in original_categorical_features) {
        base_col <- col %>% gsub("_binned", "", .) %>% gsub("_merged", "", .) %>% gsub("_BIN",
        if (!base_col %in% colnames(temp_data)) {stop(sprintf("Column %s is missing from input_
        temp_data[[base_col]] <- as.factor(temp_data[[base_col]])
    }
    temp_data$COVERAGE_BIN <- temp_data$COVERAGE %>% fct_collapse(
        "TPL" = c("TPL"),
        "OMN" = c("PO", "FO")
    )
    for (col in original_categorical_features) {
        temp_data[[col]] <- factor(temp_data[[col]], levels = levels_named_list[[col]])
    }
    output_data <- temp_data
    return(output_data)
}

data_preparation_binned_continuous <- function(input_data) {
    if (!exists("result_named_list")) {stop("result_named_list object is missing")}
    if (!exists("n_bins_named_list")) {stop("n_bins_named_list object is missing")}
    if (!exists("binned_continuous_features")) {stop("binned_continuous_features object is miss
    temp_data <- input_data
    for (col in binned_continuous_features) {
        base_col <- col %>% gsub("_binned", "", .) %>% gsub("_merged", "", .)
        if (!base_col %in% colnames(temp_data)) {stop(sprintf("Column %s is missing from input_
        result <- result_named_list[[base_col]]
        n_bins <- n_bins_named_list[[base_col]]
        bins <- rebuild_bins(result, n_bins)
        temp_data[[paste0(base_col, "_binned")]] <- cut(temp_data[[base_col]], breaks = bins, c
        if (length(unique(temp_data[[paste0(base_col, "_binned")]])) != n_bins) {stop(sprintf('
    }
    temp_data$AGEPH_binned_merged <- temp_data$AGEPH_binned %>% fct_collapse(
        "(66,95]" = c("(66,68]", "(68,71]", "(71,95]"))
    temp_data$POWER_binned_merged <- temp_data$POWER_binned %>% fct_collapse(
        "(76,243]" = c("(76,83]", "(83,99]", "(99,243]")
    )
    for (col in binned_continuous_features) {
        temp_data[[col]] <- factor(temp_data[[col]], levels = levels_named_list[[col]], ordered
    }
    output_data <- temp_data
    return(output_data)
}

data_preparation_enriched <- function(input_data) {
    temp_data <- input_data
    if (!"POPULATION" %in% colnames(temp_data)) {stop("Column POPULATION is missing from input_
    if (!"REGION" %in% colnames(temp_data)) {stop("Column REGION is missing from input_data")}
    if (!"MUNICIPALITY" %in% colnames(temp_data)) {stop("Column MUNICIPALITY is missing from in
```

```
    temp_data$POPULATION_binned <- cut(temp_data$POPULATION, breaks = rebuild_bins(
        result_named_list[["POPULATION"]], n_bins_named_list[["POPULATION"]]
    ), ordered_result = TRUE, include.lowest = TRUE) %>%
    fct_collapse(
        "(1.34e+05,4.53e+05]" = c("(1.34e+05,2.05e+05]", "(2.05e+05,4.53e+05]")
    )
    temp_data$REGION_merged <- temp_data$REGION %>% as.factor() %>%
        fct_collapse(
            "Brussels" = c("RÉGION DE BRUXELLES-CAPITALE"),
            "Others" = c("RÉGION WALLONNE", "RÉGION FLAMANDE")
        )
    temp_data$MUNICIPALITY_binned <- ifelse(temp_data$MUNICIPALITY %in% levels_named_list[["MU
        as.factor()
    for (col in binned_enriched_features) {
        temp_data[[col]] <- factor(temp_data[[col]], levels = levels_named_list[[col]], ordere
    }
    output_data <- temp_data
    return(output_data)
}

data_preparation_interaction <- function(input_data) {
    if (!"BM_binned" %in% colnames(input_data)) {stop("Column BM_binned is missing from input_
    if (!"FUEL" %in% colnames(input_data)) {stop("Column FUEL is missing from input_data")}
    temp_data <- input_data %>%
        mutate(
            BM_01_FUEL_gasoline = BM_binned == "[0,1]" & FUEL == "gasoline",
            BM_01_FUEL_not_gasoline = BM_binned == "[0,1]" & FUEL != "gasoline"
        )
    output_data <- temp_data
    return(output_data)
}

data_preparation_workflow <- function(input_data) {
    temp_data <- input_data %>%
        data_enrichment %>%
        data_preparation_original_categorical %>%
        data_preparation_binned_continuous %>%
        data_preparation_enriched %>%
        data_preparation_interaction
    output_data <- temp_data
    return(output_data)
}
```

```
test_data <- mtpl_be %>%
    filter(ID %in% data$ID) %>%
    data_preparation_workflow()
```

```r
# Set ordered factors for binned continuous features to unordered
for (col in names(levels_named_list)) {
    data_enriched[[col]] <- factor(data_enriched[[col]], levels = levels_named_list[[col]], or
}

for (col in c(original_categorical_features, binned_continuous_features, enriched_features)) {
    if (!col %in% colnames(test_data)) {
        stop(paste("Column", col, "is missing from test_data"))
    }
    if (!identical(test_data[[col]], data_enriched[[col]])) {
        stop(paste("Column", col, "does not match the enriched data"))
    }
}

cat(sprintf("All columns in test_data match the enriched data.\n(%s)", paste(c(original_categor
```

```
All columns in test_data match the enriched data.
(COVERAGE_BIN, FUEL, FLEET, SEX, BM_binned, AGEPH_binned_merged, AGEC_binned, POWER_binned_merg
```

3.1.2 Cross-Validation

To better understand cross-validation, let's illustrate it with a simple manual example. We create 5 data sets (folds) for cross-validation. Each fold will be used once as a validation set while the remaining folds form the training set.

```r
cv_split <- function(input_data, kfold, strata) {
    folds <- input_data %>% vfold_cv(strata = strata, v = kfold)
    return(folds)
}
base_formula <- "NCLAIMS ~ offset(log(EXP))"
folds <- mtpl_be %>%
    filter(ID %in% data$ID) %>%
    data_preparation_workflow() %>%
    cv_split(kfold = 5, strata = "NCLAIMS")
models <- vector("list", length(folds$splits))
poisson_log_loss_list <- list()
beta0 <- list()
for (i in seq_along(folds$splits)) {
    train_data <- analysis(folds$splits[[i]])
    val_data <- assessment(folds$splits[[i]])
    models[[i]] <- glm(formula = as.formula(base_formula), data = train_data, family = poisson
    log_loss <- -mean(dpois(val_data$NCLAIMS, lambda = predict(models[[i]], newdata = val_data
    poisson_log_loss_list[[i]] <- log_loss
    beta0[[i]] <- coef(models[[i]])[1]
}
m0 <- glm(formula = as.formula(base_formula), data = mtpl_be %>% filter(ID %in% data$ID) %>% da
```

```r
beta0_full <- coef(m0)[1]
log_loss_full <- -mean(dpois(holdout$NCLAIMS, lambda = predict(m0, newdata = holdout, type = "
cat(sprintf("For the simple GLM model:\n  Log loss (-log-likelihood/n): %s\n  Intercept (beta0)
```

```
For the simple GLM model:
  Log loss (-log-likelihood/n): 0.397287337307078
  Intercept (beta0): -1.97599003410665
  95% CI for beta0: [-1.9914326792921, -1.96054738892119]
```

```r
cat(sprintf("Cross-validated log loss: min = %s, mean = %s, max = %s\n", min(unlist(poisson_log
```

```
Cross-validated log loss: min = 0.384639629129922, mean = 0.389544236202061, max = 0.393721901
```

```r
cat(sprintf("Cross-validated beta0: min = %s, mean = %s, max = %s\n", min(unlist(beta0)), mean
```

```
Cross-validated beta0: min = -1.9793873830536, mean = -1.97599273053119, max = -1.973368215286
```

This illustrates how cross-validation enables enables to check the robustness and variability of the model's performance and parameter estimates across different subsets of the data. Comparing the cross-validated mean log loss, AIC or BIC across various models will yield a more robust choice for model selection. Now that the intuition is set, we can create our workflow including cross-validation.

3.1.3 Performance Metrics

To compare the performance of different models, we can consider the following cross-validated metrics:

- Log Loss: Measures predictive accuracy by evaluating the negative log-likelihood (scaled by the number of observations) of the observed outcomes given the predicted probabilities.
- AIC (Akaike Information Criterion): Evaluates model quality by balancing goodness of fit and model complexity. Lower values indicate better models.
- BIC (Bayesian Information Criterion): Similar to AIC, but with a stronger penalty for model complexity. Lower values indicate better models.

Note that the `Log- Loss`, `AIC` and `BIC` are all based on the likelihood of the model, here are how they are computed:

- Log Loss: `-mean(dpois(y, lambda = predicted_lambda, log = TRUE))` or `-logLik(model)/n` where `n` is the number of observations.
- AIC: `2 * k - 2 * logLik(model)` where `k` is the number of parameters.
- BIC: `log(n) * k - 2 * logLik(model)` where `n` is the number of observations and `k` is the number of parameters.

We see that `Log loss`simply does not account for model complexity, unlike `AIC` and `BIC` which include penalties of 2 * k and 11.5566361[8] * k respectively. Because the penalty is so much higher, due to the large size of our portfolio, `BIC` tends to favor simpler models compared to `AIC`.

The models will be gradually complexified by adding covariates. For each step, we will compare the final model with the previous step based on:

- Scaled Residual Deviance: Measures the goodness of fit of a model relative to a saturated model, scaled by the residual deviance of the null model.
- Anova Test: Compares nested models by evaluating whether the addition of new covariates significantly improves the model fit.

For the scaled residual deviance, here are the maximum and minimum deviance values:

- Null Deviance: $D_0 = 2 \times (logLik(SaturatedModel) - logLik(NullModel))$
- Residual Deviance: $D_m = 2 \times (logLik(SaturatedModel) - logLik(FittedModel))$
- Scaled Residual Deviance: $D_s = \frac{D_m}{D_0}$

```
train_data <- mtpl_be %>%
    filter(ID %in% data$ID) %>%
    data_preparation_workflow()
m_null <- glm(NCLAIMS ~ offset(log(EXP)), family = poisson, data = train_data)
m_ex <- glm(NCLAIMS ~ offset(log(EXP)) + BM_binned, family = poisson, data = train_data)
cat(sprintf(
    "Scaled Residual Deviance: %.2f%% (%.0f / %.0f)",
    deviance(m_ex) / deviance(m_null)*100,
    deviance(m_ex),
    deviance(m_null)
))
```

```
Scaled Residual Deviance: 98.28% (70460 / 71692)
```

```
ll_sat <- deviance(m_null)/2 + logLik(m_null)
calc_dm <- 2 * (ll_sat - logLik(m_ex))
cat(sprintf(
    "Calculated D_m = %.2f and Model Deviance = %.0f",
    calc_dm,
    deviance(m_ex)
))
```

```
Calculated D_m = 70460.30 and Model Deviance = 70460
```

[8]The term `log(nrow(data)*(4/5))` comes from the fact that we are using cross-validation with 5 folds, so each training set contains 4/5 of the data.

```r
cat(sprintf("We thus have maximum log likelihood for the saturated model: %.2f", ll_sat))
```

```
We thus have maximum log likelihood for the saturated model: -15022.23
```

```r
cat(sprintf("And minimum log likelihood for the null model: %.2f", logLik(m_null)))
```

```
And minimum log likelihood for the null model: -50868.24
```

The anova test will be comparing the deviance of the nested models to evaluate whether the addition of new covariates significantly improves the model fit.

```r
anova(m_null, m_ex, test = "Chisq")
```

```
Analysis of Deviance Table

Model 1: NCLAIMS ~ offset(log(EXP))
Model 2: NCLAIMS ~ offset(log(EXP)) + BM_binned
  Resid. Df Resid. Dev Df Deviance  Pr(>Chi)
1    130584      71692
2    130582      70460  2   1231.7 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

3.2 Frequency Modelling

Based on all the notions explained in the Validation Framework: data preparation, cross-validation, and performance metrics, we will now create workflows for frequency modelling.

3.2.1 GLM Models

```r
create_wflow_freq_glm <- function(data, formula, v = 5, seed = 123) {

    set.seed(seed)
    folds <- vfold_cv(
        data, v = v,
        strata = NCLAIMS
    )

    rec <- recipe(as.formula(formula), data = data)

    # The Poisson family uses a log link by default
    mod <- poisson_reg() %>%
        set_engine("glm")
```

```
    wflow <- workflow() %>%
        add_recipe(rec) %>%
        add_model(mod)

    cv_results <- fit_resamples(
        wflow,
        resamples = folds,
        control = control_resamples(save_pred = TRUE)
    )

    # Cross-validation does not fit a final model on all the data
    final_fit <- fit(wflow, data = data)
    glm_fit <- extract_fit_engine(final_fit)

    # Cross-validated deviance
    cv_pred <- collect_predictions(cv_results)
    cv_deviance <- cv_pred %>%
        mutate(deviance_i = poisson()$dev.resids(y = NCLAIMS, mu = .pred, wt = 1)) %>%
        summarise(
            total_deviance = sum(deviance_i),
        )
    cv_log_loss <- poisson_log_loss(cv_pred, truth = NCLAIMS, estimate = .pred)

    list(
        workflow = wflow,
        folds = folds,
        resamples = cv_results,
        fit = glm_fit,
        performance_metrics = list(
            AIC = AIC(glm_fit),
            BIC = BIC(glm_fit),
            Deviance = deviance(glm_fit),
            CV_Deviance = cv_deviance$total_deviance,
            CV_LogLoss = cv_log_loss$.estimate
        )
    )
}
```

We created a workflow to:

- Make the n folds (5 by default)
- Add the new levels to a categorical variable “Other”, particularly relevant for Municipalities with low frequency
- Specify the Poisson regression model with a log link
- Fit the model on the entire training dataset
- Perform cross-validation to evaluate model performance
- Store the workflow, cross-validation (Deviance and LogLoss) and final model fit

This workflow will be used for all models in the frequency analysis, ensuring consistency in model fitting, cross-validation, and performance evaluation.

3.2.1.1 Categorical Variables Only

Let us start with adding the categorical variables only to our frequency model:

- COVERAGE_BIN
- FLEET
- SEX
- FUEL

Note that USE was not significant in the preliminary analysis and thus is not included in the categorical variables for the frequency model.

```
perf_metrics_df <- data.frame(
    Model = character(),
    AIC = numeric(),
    BIC = numeric(),
    Deviance = numeric(),
    CV_Deviance = numeric(),
    CV_LogLoss = numeric()
)
append_performance_metrics <- function(name, formula, result, perf_df = perf_metrics_df) {
    perf <- result$performance_metrics
    new_row <- data.frame(
        Model = name,
        AIC = perf$AIC,
        BIC = perf$BIC,
        Deviance = perf$Deviance,
        CV_Deviance = perf$CV_Deviance,
        CV_LogLoss = perf$CV_LogLoss
    )
    rbind(perf_df, new_row)
}

f0 <- "NCLAIMS ~ 1"
fcat <- paste0("NCLAIMS ~ ", paste(original_categorical_features, collapse = " + "))
cat(sprintf("> Categorical formula: %s", fcat))
```

```
> Categorical formula: NCLAIMS ~ COVERAGE_BIN + FUEL + FLEET + SEX
```

```
r0 <- create_wflow_freq_glm(train_data, f0)
perf_metrics_df <- append_performance_metrics(name = "Intercept Only", formula = f0, result = r
rcat <- create_wflow_freq_glm(train_data, fcat)
perf_metrics_df <- append_performance_metrics(name = "Categorical Only", formula = fcat, result
```

```
perf_metrics_df %>% arrange(AIC) %>% knitr::kable(
    booktabs = TRUE, caption = "Performance Metrics for Frequency Models After Adding Categori
)
```

Table 11: Performance Metrics for Frequency Models After Adding Categorical Variables

Model	AIC	BIC	Deviance	CV_Deviance	CV_LogLoss
Categorical Only	101747.0	101795.9	71692.51	71701.73	0.3895784
Intercept Only	101916.8	101926.5	71870.30	71872.46	0.3902322

```
test <- anova(r0$fit, rcat$fit, test = "Chisq")
cat(sprintf("Is the categorical model a significant improvement over the intercept-only model?
```

```
Is the categorical model a significant improvement over the intercept-only model? Yes (p-value
```

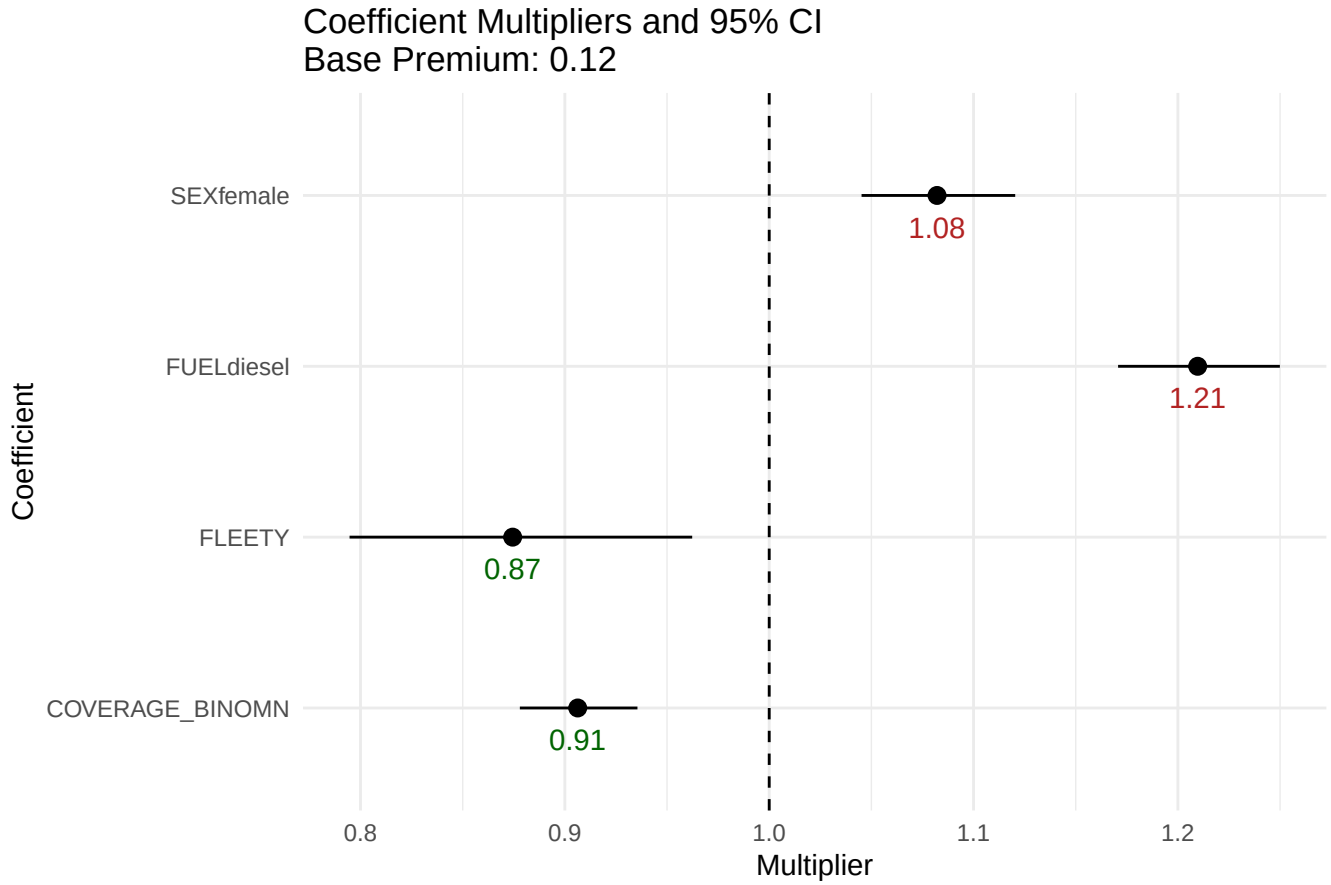
```
delta_deviance <- r0$performance_metrics$CV_Deviance - rcat$performance_metrics$CV_Deviance
diff_degree_of_freedom <- r0$fit$df.residual - rcat$fit$df.residual
pvalue <- pchisq(delta_deviance, diff_degree_of_freedom, lower.tail = FALSE)
cat(sprintf("Anova test based on CV Deviance: delta_deviance = %.4f, diff_degree_of_freedom = %
```

```
Anova test based on CV Deviance: delta_deviance = 170.7362, diff_degree_of_freedom = 4, p-value
```

```
check_summary <- function(result) {
    sum_mod <- summary(result$fit)
    coefficients <- sum_mod$coefficients %>% as.data.frame()
    coefficients$p.value <- coefficients[, "Pr(>|z|)"]
    coefficients <- coefficients %>% select(Estimate, `Std. Error`, p.value)
    coefficients$significant <- ifelse(coefficients$p.value < 0.05, "Yes", "No")
    coefficients$multiplier <- exp(coefficients[, "Estimate"])
    coefficients$multiplier_lb <- exp(coefficients[, "Estimate"] - 1.96 * coefficients[, "Std.
    coefficients$multiplier_ub <- exp(coefficients[, "Estimate"] + 1.96 * coefficients[, "Std.
    return(coefficients)
}

plot_summary <- function(result, add_text=FALSE) {
    coefficients <- check_summary(result)
    mult_df <- coefficients %>% filter(rownames(coefficients) != "(Intercept)")
    ggplot(mult_df, aes(x = rownames(mult_df), y = multiplier, ymin = multiplier_lb, ymax = mul
        geom_pointrange(
            color = ifelse(mult_df$significant == "Yes", "black", "firebrick")
        ) +
        geom_hline(yintercept = 1, linetype = "dashed") +
        {if(add_text) geom_text(
            aes(label = round(multiplier, 2)), vjust = 2,
```

```
            color = ifelse(mult_df$multiplier > 1, "firebrick", "darkgreen")
        ) else NULL} +
        coord_flip() +
        labs(x = "Coefficient", y = "Multiplier", title = sprintf("Coefficient Multipliers and
        theme_minimal()
}
coefs <- check_summary(rcat)
plot_summary(rcat, add_text=TRUE)
```



We see that the anova test indicates that the more complex model with categorical variables is a significant improvement over the intercept-only model. We also observe that all the levels we kept are significant. The multipliers are in line with our expectations based on the data exploration:

- Female drivers have a higher risk compared to male drivers: the multiplier for the female category is of 1.0821361 (95% CI: 1.0451905 - 1.1203878)
- Diesel cars are significantly more risky compared to gasoline cars: the multiplier for the diesel category is of 1.209678 (95% CI: 1.1707423 - 1.2499086)
- Cars that are part of a fleet have a lower risk compared to non-fleet cars: the multiplier for the fleet category is of 0.8744607 (95% CI: 0.7946295 - 0.9623119)

- Drivers with an omnium coverage are at a lower risk compared to those with TPL only: the multiplier for the omnium category is of 0.9062866 (95% CI: 0.8779691 - 0.9355175)

We can conclude that the categorical variables were well-chosen in the EDA and have a significant impact in our model. The new model is better than the null model and needs no adaptation, so we can continue with the `rcat` model as our new baseline for further model development.

```
base_mod <- rcat
```

3.2.1.2 Binned Continuous Variables

Next, we will add the binned continuous variables to our model to capture the potential non-linear effects of continuous predictors on the frequency of claims:

- BM_binned
- AGEPH_binned_merged
- AGEC_binned
- POWER_binned_merged

```
fcontb <- paste0(fcat, " + ", paste(binned_continuous_features, collapse = " + "))
cat(sprintf("> Full formula with binned continuous variables: %s", fcontb))
```

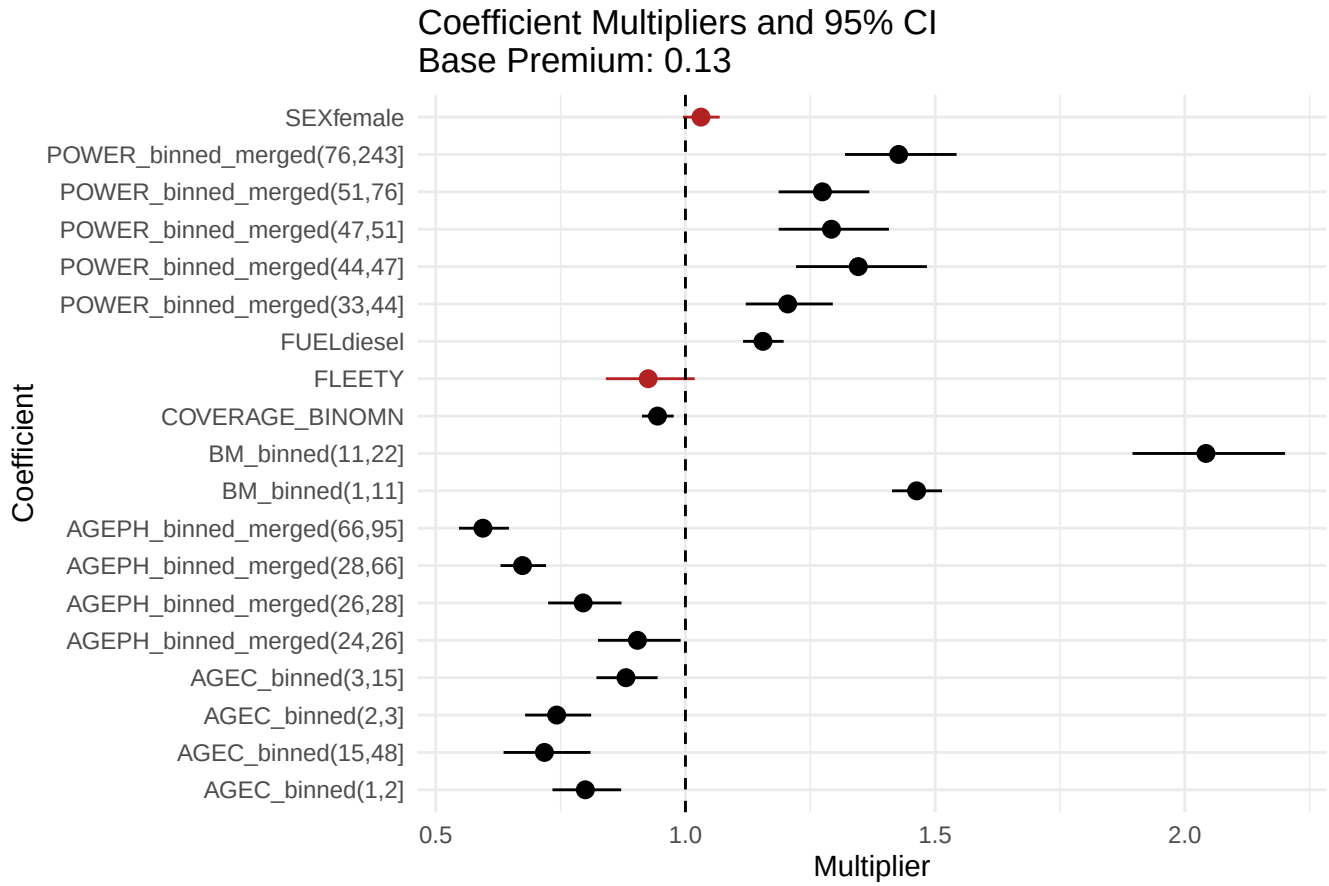
```
> Full formula with binned continuous variables: NCLAIMS ~ COVERAGE_BIN + FUEL + FLEET + SEX +
```

```
rcontb <- create_wflow_freq_glm(train_data, fcontb)
perf_metrics_df <- append_performance_metrics(name = "All Continuous Binned", formula = fcontb

test <- anova(rcat$fit, rcontb$fit, test = "Chisq")
cat(sprintf("Is the full model with binned continuous variables a significant improvement over
```

```
Is the full model with binned continuous variables a significant improvement over the categori
```

```
coefs_contb <- check_summary(rcontb)
plot_summary(rcontb)
```



There are two main takeaways from the analysis of this more complex model:

1. With the addition of binned continuous variables, the `SEXfemale` and `FLEETY` variables are not significant anymore. It suggests that they may have been capturing some of the information that is now explained by the binned continuous variables.
2. The binned continuous variables themselves are all significant at the 95% confidence level. The WoE-based binning and merging strategy we applied is therefore effective in capturing significant non-linear effects of continuous predictors on the frequency of claims.

```
coefs_contb %>% filter(significant == "No") %>%
    arrange(desc(p.value)) %>%
    kable(booktabs = TRUE, caption = "Non-significant Coefficients")
```

Table 12: Non-significant Coefficients

	Estimate	Std. Error	p.value	significant	multiplier	multiplier_lb	multiplier_ub
FLEETY	-0.0775289	0.0489716	0.1133891	No	0.9254003	0.8407061	1.018627
SEXfemale	0.0306299	0.0182467	0.0932195	No	1.0311038	0.9948795	1.068647

We will start by removing the FLEET [^fleet_remove] variable from the model as it is less significant that the SEX variable.

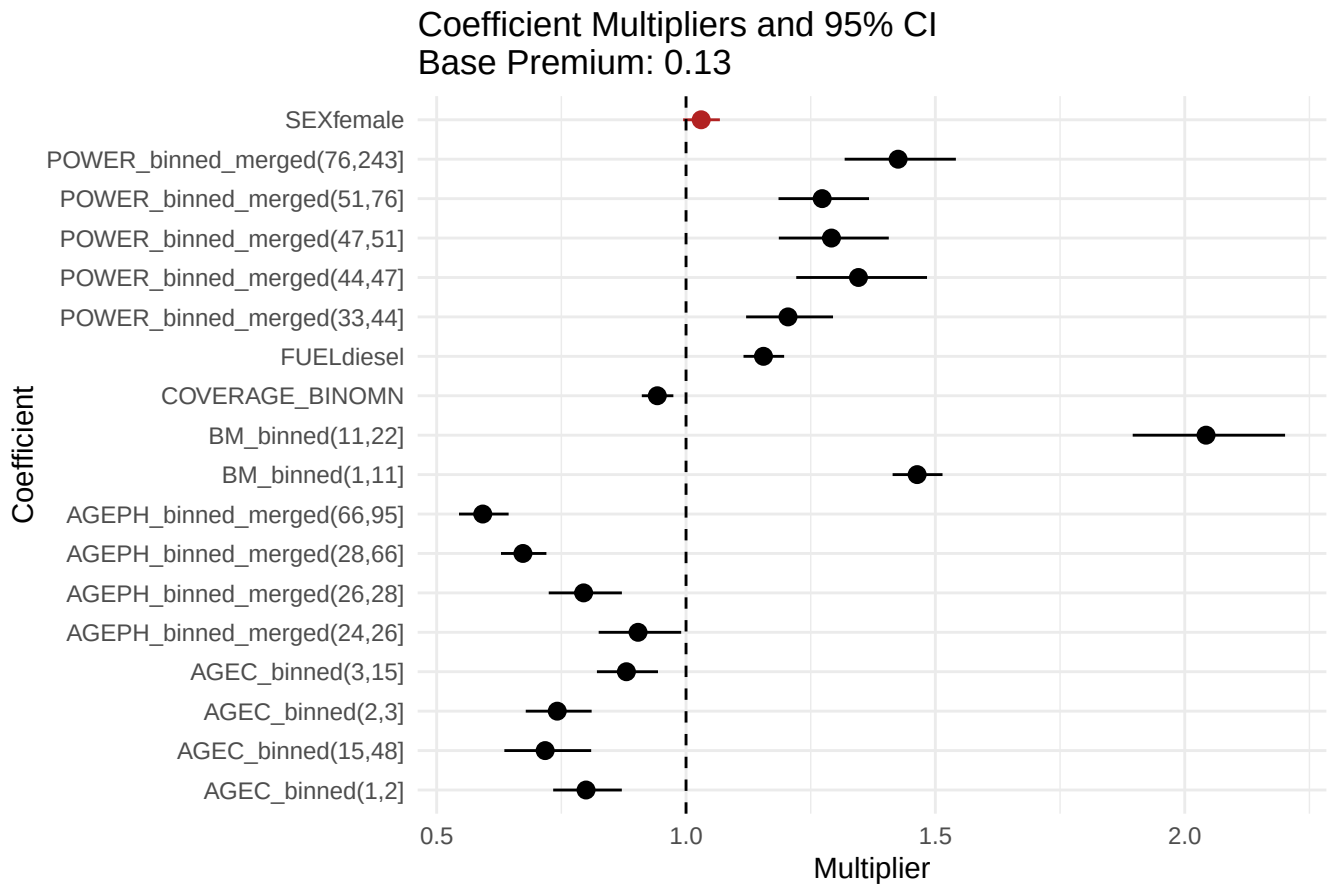
```
large_mod <- rcontb$fit
fcontb_p1 <- gsub(" \\+ FLEET", "", fcontb)
cat(sprintf("> Formula after removing FLEET: %s", fcontb_p1))
```

```
> Formula after removing FLEET: NCLAIMS ~ COVERAGE_BIN + FUEL + SEX + BM_binned + AGEPH_binned
```

```
rcontb_p1 <- create_wflow_freq_glm(train_data, fcontb_p1)
perf_metrics_df <- append_performance_metrics(name = "Pruned-1 Continuous Binned", formula = fc
test_p1 <- anova(large_mod, rcontb_p1$fit, test = "Chisq")
cat(sprintf("Are we loosing significant explanatory power by removing FLEET? %s (p-value = %.4f
```

```
Are we loosing significant explanatory power by removing FLEET? No (p-value = 0.1091)
```

```
coefs_contb_p1 <- check_summary(rcontb_p1)
plot_summary(rcontb_p1)
```



Our conclusions hold and we will now proceed to remove the SEX variable from the model as it is the other non-significant variable.

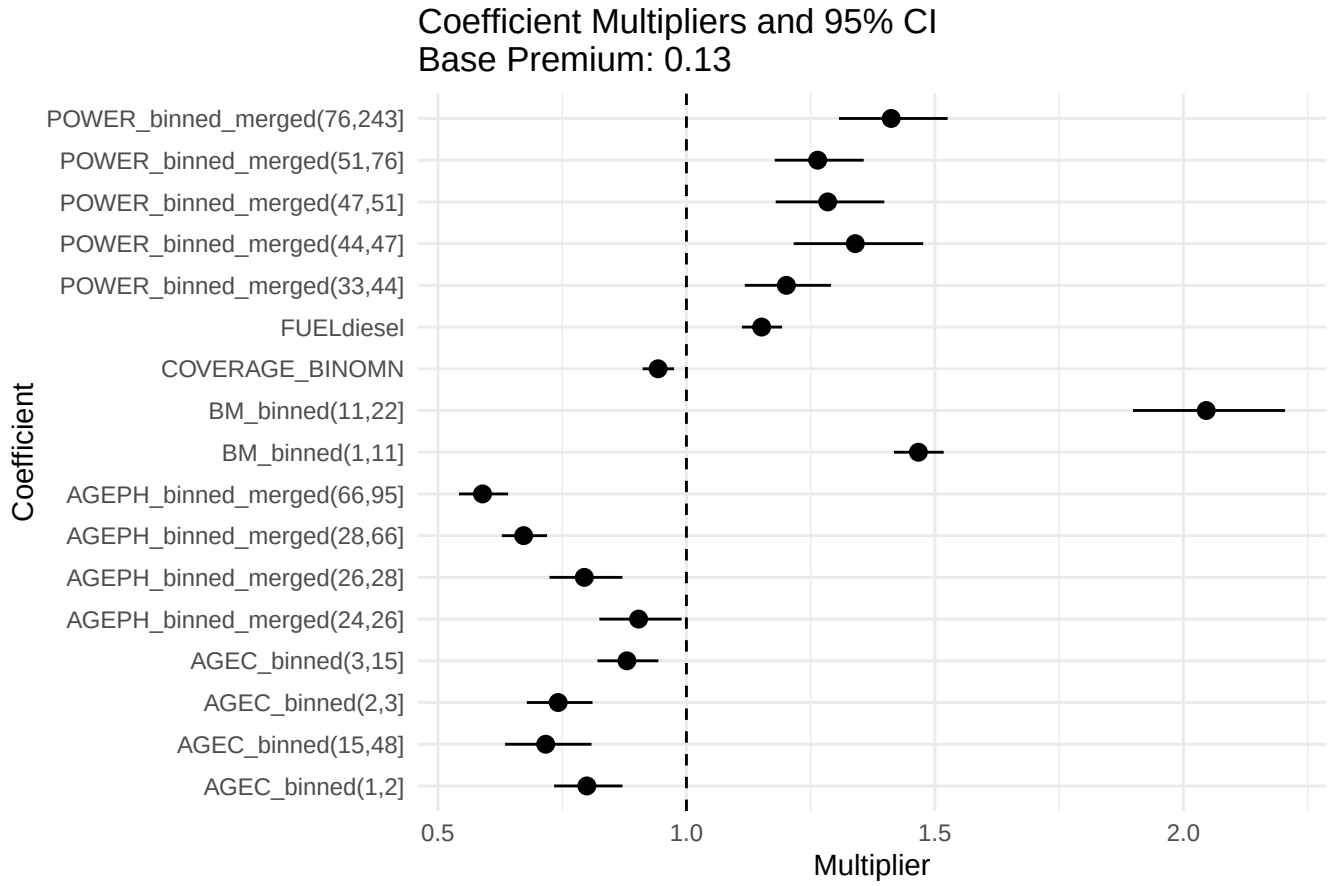
```
large_mod <- rcontb_p1$fit
fcontb_p2 <- gsub(" \\+ SEX", "", fcontb_p1)
cat(sprintf("> Formula after removing SEX: %s", fcontb_p2))
```

```
> Formula after removing SEX: NCLAIMS ~ COVERAGE_BIN + FUEL + BM_binned + AGEPH_binned_merged +
```

```
rcontb_p2 <- create_wflow_freq_glm(train_data, fcontb_p2)
perf_metrics_df <- append_performance_metrics(name = "Pruned-2 Continuous Binned", formula = fc
test_p2 <- anova(large_mod, rcontb_p2$fit, test = "Chisq")
cat(sprintf("Are we loosing significant explanatory power by removing SEX? %s (p-value = %.4f)"
```

```
Are we loosing significant explanatory power by removing SEX? No (p-value = 0.1018)
```

```
coefs_contb_p2 <- check_summary(rcontb_p2)
plot_summary(rcontb_p2)
```



```
base_mod <- rcontb_p2$fit
```

After pruning the non-significant variables `FLEET` and `SEX`, our final continuous binned model includes only the significant predictors. This model will serve as the base for the next enrichment steps in our modeling process.

3.2.1.3 Enriched Features

After including our continuous binned variables, we will now add the new geospatial and enriched features:

- `REGION_merged`
- `MUNICIPALITY_binned`
- `POPULATION_binned`

Because the Municipalities are so small, we will first focus on the two other variables.

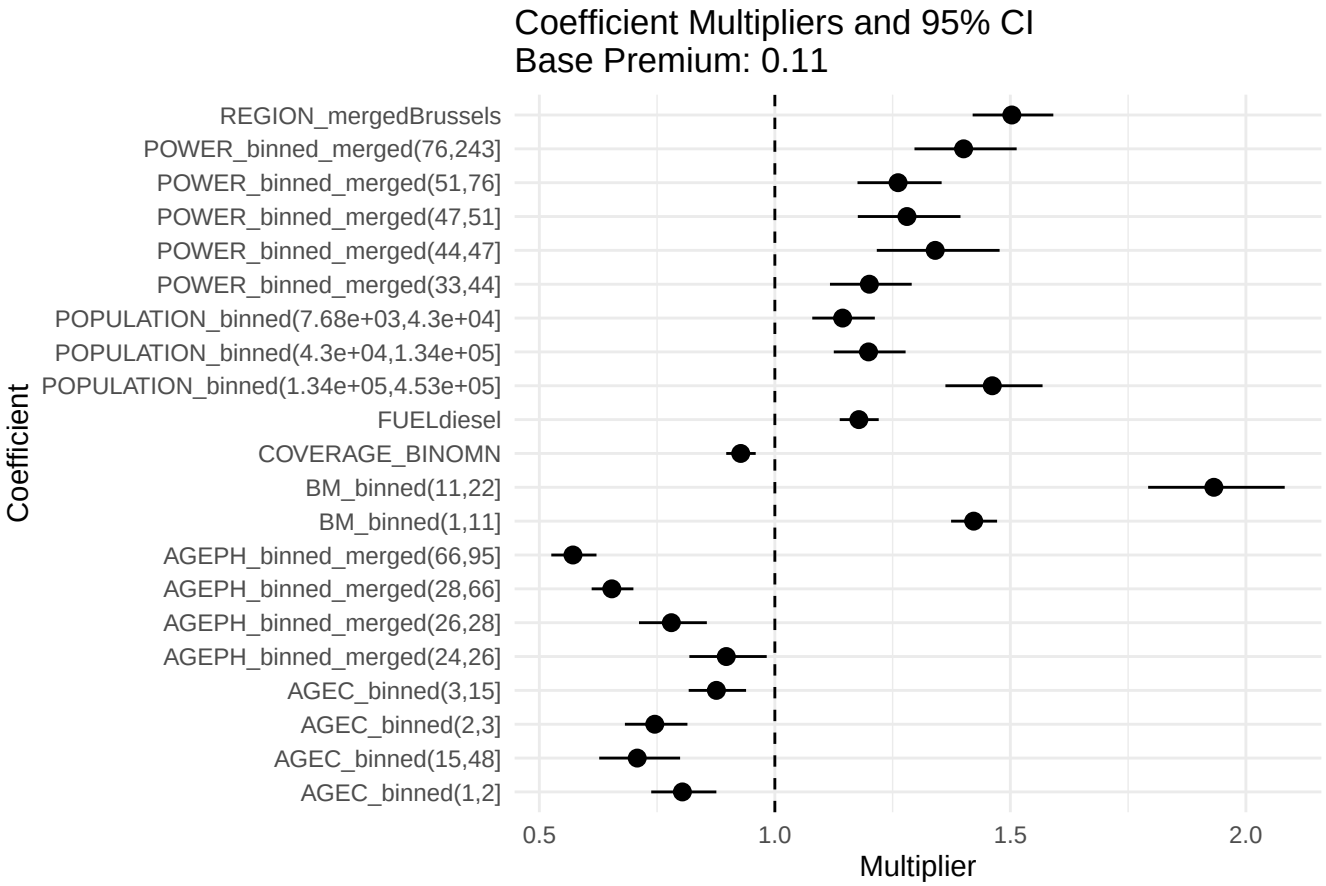
```
f_enr2 <- paste0(fcontb_p2, " + REGION_merged + POPULATION_binned")
cat(sprintf("> Formula after adding two enriched features: %s", f_enr2))
```

```
> Formula after adding two enriched features: NCLAIMS ~ COVERAGE_BIN + FUEL + BM_binned + AGEPI
```

```
r_enr2 <- create_wflow_freq_glm(train_data, f_enr2)
perf_metrics_df <- append_performance_metrics(name = "Enriched Features", formula = f_enr2, res
test_enr2 <- anova(base_mod, r_enr2$fit, test = "Chisq")
cat(sprintf("Are we gaining significant explanatory power by adding enriched features? %s (p-va
```

```
Are we gaining significant explanatory power by adding enriched features? Yes (p-value = 0.000
```

```
coefs_enr2 <- check_summary(r_enr2)
plot_summary(r_enr2)
```



We notice that the `REGION_merged` and `POPULATION_binned` variables contribute significantly to the model, it was therefore worth including them as enriched features in our final model.

- `REGION_mergedBrussels` is actually the second highest multiplier of claim frequency in the model: 1.5031001 (95% CI: 1.4198746 - 1.5912039)
- `POPULATION_binned(1.34e+05,4.53e+05]`, so high population also stand out as third biggest multiplier of claim frequency in the model: 1.4615695 (95% CI: 1.3620979 - 1.5683053)

Note that the largest multiplier of claim frequency is actually an experience-related variable, it should not be available for a-priori pricing. Therefore the enriched features we added are the most relevant for improving the a-priori model. Finally, let's have a quick look at the `MUNICIPALITY_binned` variable to see if it can add anything to the urban and population density we already considered. Note that we won't use our workflow because the bins are too small and we will run into issues with non-observed levels in the cross-validation folds. We'll make a single GLM fit to test our hypothesis.

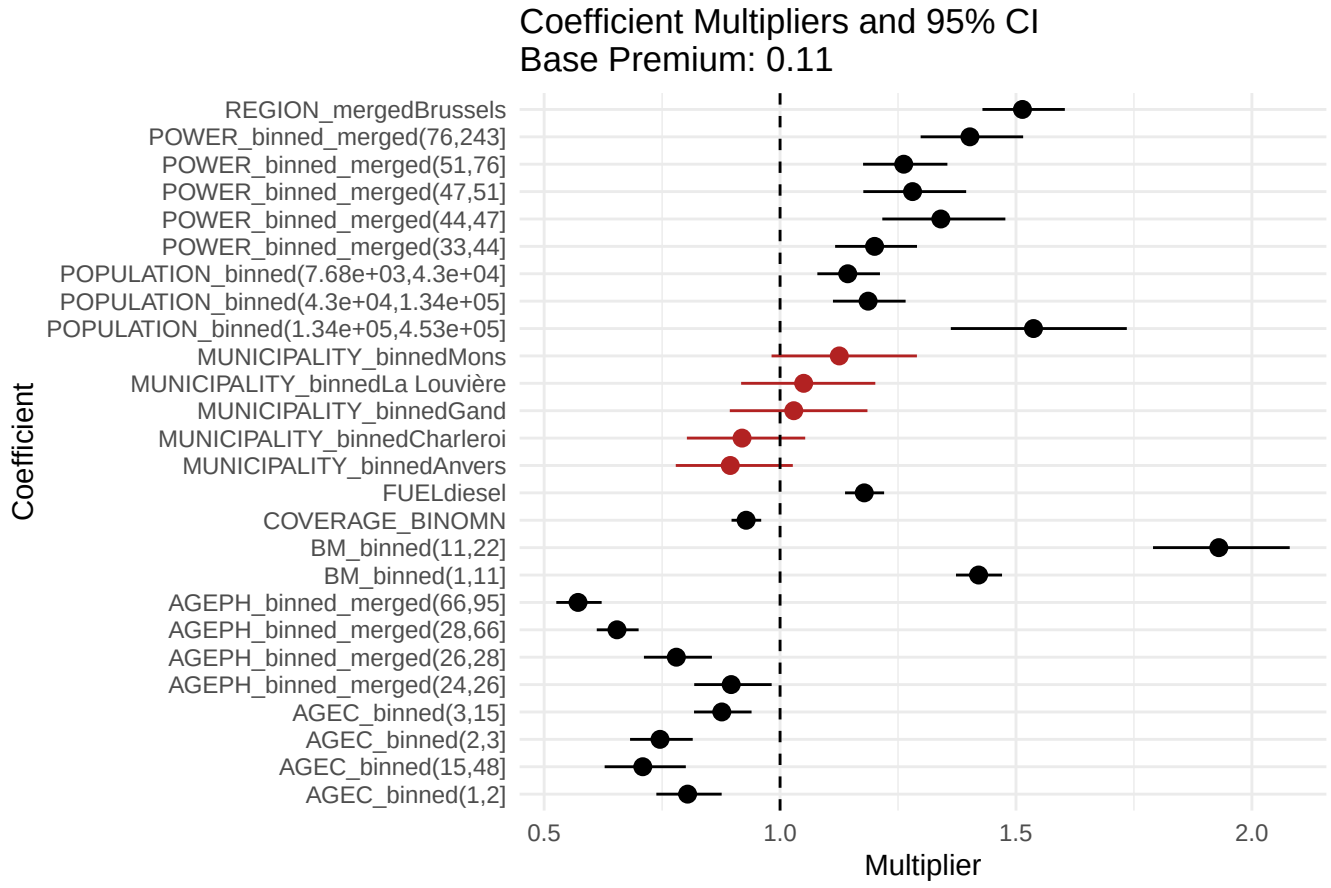
```
f_enr3 <- paste0(f_enr2, " + MUNICIPALITY_binned")
cat(sprintf("> Formula after adding MUNICIPALITY_binned: %s", f_enr3))
```

```
> Formula after adding MUNICIPALITY_binned: NCLAIMS ~ COVERAGE_BIN + FUEL + BM_binned + AGEPH_
```

```
r_enr3 <- list()
r_enr3$fit <- glm(as.formula(f_enr3), data = train_data, family = poisson())
test_enr3 <- anova(r_enr2$fit, r_enr3$fit, test = "Chisq")
cat(sprintf("Does MUNICIPALITY_binned add significant explanatory power? %s (p-value = %.4f)\n"
```

```
Does MUNICIPALITY_binned add significant explanatory power? Yes (p-value = 0.0000)
```

```
coefs_enr3 <- check_summary(r_enr3)
plot_summary(r_enr3)
```




```
base_mod <- r_enr2$fit
```

Even if the more complex MUNICIPALITY_binned variable adds some explanatory power, we see that individual municipalities do not have a significant impact on claim frequency (multiplier close to 1). We will therefore not include `MUNICIPALITY_binned` in our final model.

3.2.1.4 Interaction Effects To close this GLM estimation chapter for the claim frequency model, we will include the interaction effects we identified with the decision tree analysis.

- BM_binned x AGEPH_binned_merged
- REGION_merged x POWER_binned_merged
- BM_binned x REGION_merged
- BM_01_FUEL_not_gasoline
- BM_01_FUEL_gasoline

The interaction based on expert knowledge will also be considered:

- AGEPH_binned_merged x POWER_binned_merged

Because interactions will create very small cross sections, we'll skip cross validation and use GLM straightaway.

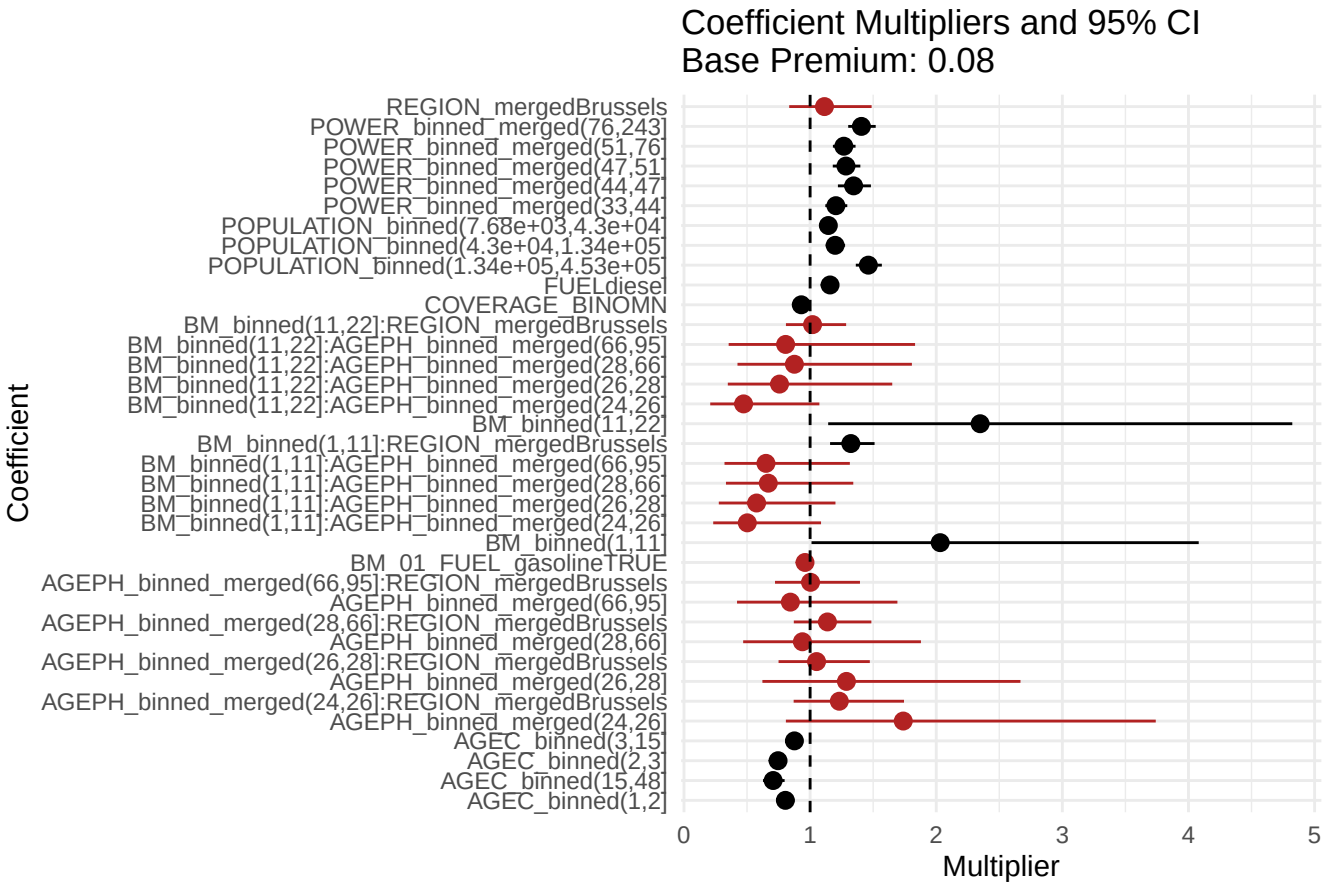
```
f_sp_int <- paste0(f_enr2, " + ", paste(interaction_features, collapse = " + "))
cat(sprintf("> Formula after adding specific interaction: %s", f_sp_int))
```

```
> Formula after adding specific interaction: NCLAIMS ~ COVERAGE_BIN + FUEL + BM_binned + AGEPH
```

```
r_sp_int <- list()
r_sp_int$fit <- glm(as.formula(f_sp_int), data = train_data, family = poisson())
test_sp_int <- anova(base_mod, r_sp_int$fit, test = "Chisq")
cat(sprintf("Are we gaining significant explanatory power by adding interactions? %s (p-value =
```

```
Are we gaining significant explanatory power by adding interactions? Yes (p-value = 0.0000)
```

```
coefs_sp_int <- check_summary(r_sp_int)
plot_summary(r_sp_int)
```



Note that even if the interactions add some explanatory power, they make the model more complex, less stable, and harder to interpret. Besides, only the specific BM between 1 and 11 x Brussels Region has a significant effect on claim frequency. We will therefore not include the interaction terms in our final model.

3.2.1.5 Summary of Performance Metrics and Final GLM Model

We can close the GLM frequency model estimation here by looking at the evolution of our performance metrics across the different modeling stages:

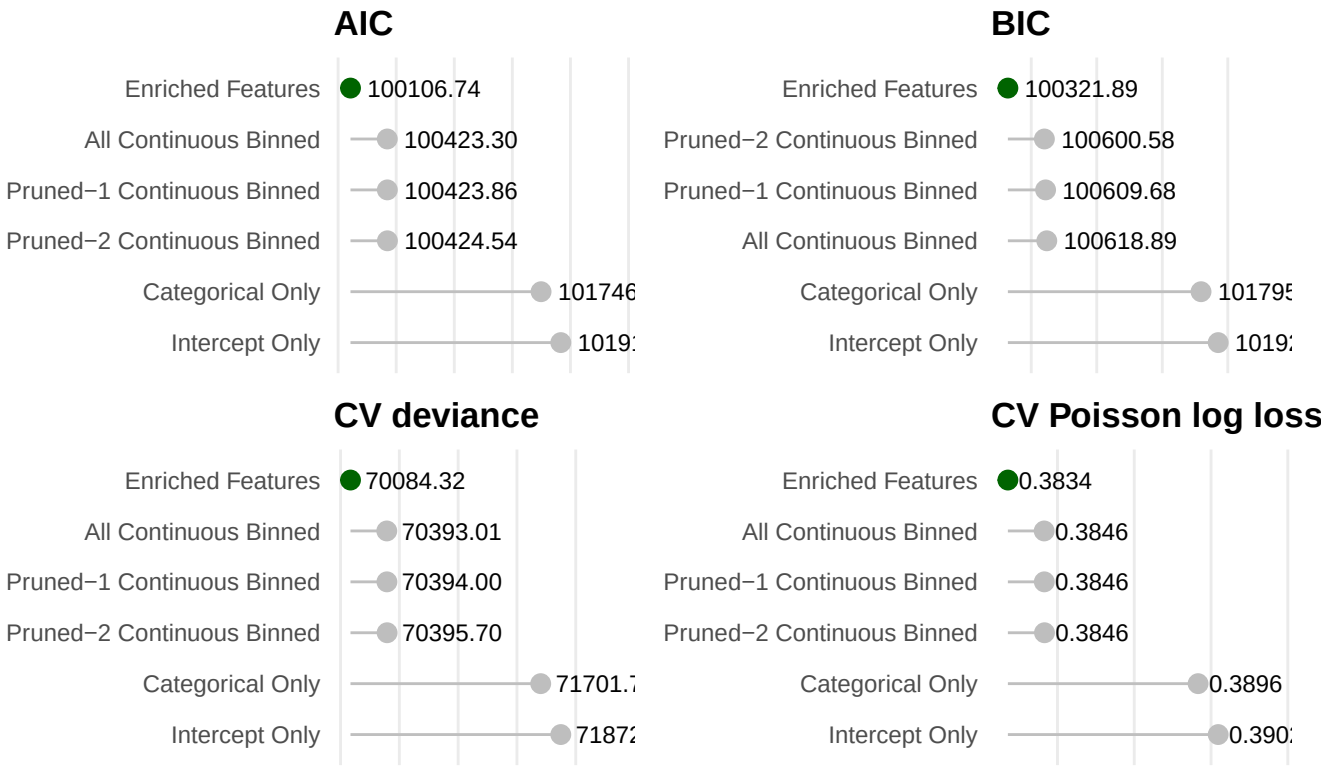
```
plot_leaderboard <- function(data, metric, title, digits = 2) {

    plot_data <- data %>%
        transmute(
            Model = as.character(Model),
            value = .data[[metric]]
        ) %>%
        arrange(desc(value)) %>%
        mutate(
            Model = factor(Model, levels = Model),
            best = value == min(value, na.rm = TRUE)
        )
```

```
    ggplot(plot_data, aes(x = value, y = Model)) +
        geom_segment(
            aes(x = min(value), xend = value, yend = Model),
            colour = "grey75"
        ) +
        geom_point(aes(colour = best), size = 3) +
        geom_text(
            aes(label = formatC(value, format = "f", digits = digits)),
            hjust = -0.15,
            size = 3.3
        ) +
        scale_colour_manual(
            values = c("FALSE" = "grey", "TRUE" = "darkgreen"),
            guide = "none"
        ) +
        scale_x_continuous(
            expand = expansion(mult = c(0.08, 0.35))
        ) +
        labs(title = title, x = NULL, y = NULL) +
        # remove x-axis labels
        theme_minimal(base_size = 11) +
        theme(
            panel.grid.major.y = element_blank(),
            panel.grid.minor = element_blank(),
            plot.title = element_text(face = "bold"),
            axis.text.x = element_blank()
        )
}

p1 <- plot_leaderboard(perf_metrics_df, "AIC", "AIC")
p2 <- plot_leaderboard(perf_metrics_df, "BIC", "BIC")
p3 <- plot_leaderboard(perf_metrics_df, "CV_Deviance", "CV deviance")
p4 <- plot_leaderboard(
    perf_metrics_df, "CV_LogLoss", "CV Poisson log loss", digits = 4
)

(p1 | p2) / (p3 | p4) +
    plot_annotation(
        title = "Model performance leaderboard",
        subtitle = "Best model highlighted in green"
)
```



Based on this leaderboard, we can confirm that our final model with categorical, significant binned continuous features and significant enriched features performs the best across all evaluated metrics. The other models have the same ranking across AIC, cross-validated deviance, and cross-validated Poisson log loss. The only difference we noticed is in the BIC metric that penalizes more model complexity. For the three models with continuous features, this penalization difference is sufficient to alter their ranking and favor the simpler models.

3.2.2 GAM Models

3.2.3 Final Frequency Model

3.3 Severity Modelling

3.3.1 GLM Models

We went through a lot of details in the data preparation and model selection process for the frequency component of our analysis. For the severity component, we will go through a similar process in much lesser depth to keep this report concise. We will nevertheless go through the following steps:

- Filter out non-relevant observations for the severity analysis

- Look at the distribution of the severity across individual features
- Fit step-by-step GLM models to find our best model. We'll skip cross-validation for brevity.

Watch out that we are not using the canonical link function for the Gamma distribution, which is the inverse link. This means that the sum of predicted values might not equal the total observed cost. In a real-world setting, we would need to account for this discrepancy when setting premiums. The reason why we will use the log link instead of the canonical inverse link is to ensure that our predicted values are always positive and to simplify the interpretation of the model coefficients (as multiplicative effects on the expected value of the response variable).

```
train_severity <- train_data[train_data$NCLAIMS > 0, ]
avg_cost_int <- glm(AVG ~ 1, data = train_severity, family = Gamma(link = "log"))
avg_cost_int_sum <- summary(avg_cost_int)
mu_avg <- exp(avg_cost_int_sum$coefficients[1, 1])
sigma_avg <- exp(avg_cost_int_sum$coefficients[1, 2])
cat(sprintf("Estimated mean of average cost: %.2f (95%% CI: %.2f - %.2f)", mu_avg, mu_avg - 1.9
```

3.3.1.1 Data Preparation For Severity Analysis

```
Estimated mean of average cost: 1698.68 (95% CI: 1696.54 - 1700.83)
```

3.3.2 Final Severity Model

4 Global Overview

4.1 Importance of Features

4.2 Final Premium Structure

5 Conclusion